



Πανεπιστήμιο Αιγαίου

Τμήμα Μηχανικών Πληροφοριακών και Επικοινωνιακών Συστημάτων

**Υλοποίηση Πρωτοκόλλου Συμφωνίας
Κλειδιού Ομάδας με Χρήση Blockchain
(Implementation of a Blockchain Group
Key Agreement Protocol)**

Διπλωματική Εργασία

Στοϊλουδής Κωνσταντίνος

Επιβλέπουσα Καθηγήτρια
Κωνσταντίνου Ελισάβετ

Αθήνα, 2026

Περιεχόμενα

1	Εισαγωγή	5
1.1	Στόχοι της Εργασίας	5
1.2	Δομή της Εργασίας	6
2	Μαθηματικό Υπόβαθρο	7
2.1	Ομάδες (Groups)	7
2.1.1	Κυκλικές Ομάδες και Γεννήτορες	7
2.1.2	Πεπερασμένα Σώματα	8
2.1.3	Αριθμητική Modulo	8
2.2	Συναρτήσεις Κατακερματισμού	8
2.3	Πρωτόκολλα Ανταλλαγής Κλειδιών	8
2.3.1	Πρωτόκολλο Diffie--Hellman	8
2.3.2	Ελλειπτική Εκδοχή Diffie--Hellman (ECDH)	9
2.3.3	Group Diffie--Hellman	9
2.3.4	Contributory Key Agreement	10
2.3.5	Υπολογιστική Ασφάλεια και Δύσκολα Μαθηματικά Προβλήματα	10
2.4	Κρυπτογραφία Ελλειπτικών Καμπυλών	10
2.4.1	Ελλειπτικές Καμπύλες και Σημεία	11
2.4.2	Γεωμετρική και Αλγεβρική Δομή	11
2.4.3	Πολλαπλασιασμός με Βαθμωτό (Scalar Multiplication)	12
2.4.4	Ασφάλεια της ECC	12
2.4.5	Πλεονεκτήματα της Κρυπτογραφίας Ελλειπτικών Καμπυλών	12
2.5	Δύσκολα Υπολογιστικά Προβλήματα	13
2.5.1	Πρόβλημα Διακριτού Λογαρίθμου (Discrete Logarithm Problem)	13
2.5.2	Υπολογιστικό Πρόβλημα Diffie--Hellman (Computational Diffie--Hellman)	13
2.5.3	Αποφασιστικό Πρόβλημα Diffie--Hellman (Decisional Diffie--Hellman)	13
2.5.4	Διγραμμικό Πρόβλημα Diffie--Hellman (Bilinear Diffie--Hellman)	14
2.5.5	Υπολογιστικές Υποθέσεις και Προβλήματα: Σχέση CDH και DDH	14
2.6	Pairing-Based Cryptography	15
2.6.1	Bilinear Pairings	15
2.6.2	Ιδιότητες	16
2.6.3	Κατηγοριοποίηση Ζευγοποιήσεων	16
2.6.4	Τύποι Pairings	17
2.6.5	Weil Pairing	17
2.6.6	Tate Pairing	18
2.6.7	Ate Pairing	19
2.6.8	Pairing-Friendly Καμπύλες	19

2.6.9	Πλεονεκτήματα και Μειονεκτήματα των Ζευγοποιήσεων	19
2.7	Group Key Agreement	19
3	Πρωτόκολλα Ομαδικής Συμφωνίας Κλειδιού (Group Key Agreement Protocols)	23
3.1	Εισαγωγή	23
3.2	Βασικές Απαιτήσεις	23
3.3	Κεντροποιημένες Προσεγγίσεις	24
3.4	Κατανομημένες Προσεγγίσεις	24
3.5	Δενδροειδείς Προσεγγίσεις (Tree-Based Approaches)	24
3.6	Pairing-Based Group Key Agreement	25
3.7	Σύγκριση Προσεγγίσεων Group Key Agreement	25
3.8	Δυναμική Συμμετοχή (Dynamic Membership)	26
3.9	Μηχανισμοί Επανακλειδώματος (Rekeying Mechanisms)	26
3.10	Κλασικά Πρωτόκολλα Ομαδικής Συμφωνίας Κλειδιού	26
3.11	Επιθέσεις σε Πρωτόκολλα Group Key Agreement	27
3.12	Συμπεράσματα	27
4	Τεχνολογία Blockchain	28
4.1	Εξέλιξη των Blockchain	28
4.2	Δημόσια, Ιδιωτικά και Permissioned Blockchain	28
4.3	Μηχανισμοί Συναίνεσης	29
4.4	Smart Contracts	29
4.5	Identity Management	30
4.5.1	Proof of Work (PoW)	30
4.5.2	Proof of Stake (PoS)	30
4.5.3	Practical Byzantine Fault Tolerance (PBFT)	30
4.5.4	Raft	31
4.6	Identity Management	31
4.7	Δομή ενός Block	31
4.8	Merkle Trees	31
4.9	Ασφάλεια Blockchain	32
4.10	Blockchain και Διαχείριση Κλειδιών	32
5	Hyperledger Fabric	33
5.1	Εισαγωγή	33
5.2	Αρχιτεκτονική Hyperledger Fabric	33
5.3	Συστατικά του Fabric	33
5.3.1	Peers	33
5.3.2	Orderers	33
5.3.3	MSP (Membership Service Provider)	34
5.3.4	Certificate Authorities	34
5.4	Channels	34
5.5	Δομή του Ledger	34
5.6	Endorsement Policies	34
5.7	Chaincode	35
5.8	Private Data Collections	35
5.9	Μοντέλο Ασφάλειας του Fabric	35

6	Άρθρο Υλοποίησης	36
6.1	Μοντέλο Συστήματος	36
6.2	Κρυπτογραφική Αρχικοποίηση	37
6.3	Δημιουργία και Διαχείριση Κλειδίων	39
6.4	Πρωτόκολλο Συμφωνίας Ομαδικού Κλειδιού	41
6.4.1	Παραγωγή Τυχαίων Τιμών	41
6.4.2	Υπολογισμός Δημόσιων Συνεισφορών	41
6.4.3	Δημιουργία Κρυπτογραφικής Δέσμευσης	42
6.4.4	Δημιουργία Ψηφιακής Υπογραφής	42
6.4.5	Ανταλλαγή Μηνυμάτων	42
6.5	Επαλήθευση Μηνυμάτων	43
6.6	Παραγωγή Ομαδικού Κλειδιού	44
6.6.1	Παραγωγή των Τιμών f_i	44
6.6.2	Υπολογισμός της Παραμέτρου β	45
6.6.3	Τοπικός Υπολογισμός της Παραμέτρου α_i	45
6.6.4	Υπολογισμός του Τελικού Ομαδικού Κλειδιού	46
6.6.5	Αυτοπιστοποίηση του Κλειδιού	46
6.7	Ενσωμάτωση Blockchain	46
6.7.1	Καταγραφή Δημοσίων Παραμέτρων	47
6.7.2	Καταγραφή Δημοσίων Συνεισφορών	47
6.7.3	Καταγραφή Υπογραφών	47
6.7.4	Χρήση Chaincode	47
6.7.5	Ρόλος των Peers	48
6.7.6	Ρόλος των Orderers	48
6.7.7	Ιχνηλασιμότητα και Μη Αποποίηση Ευθύνης	48
6.7.8	Πλεονεκτήματα της Ενσωμάτωσης Blockchain	49
6.8	Ανάλυση Ορθότητας	49
6.9	Ανάλυση Ασφάλειας	51
6.9.1	Αυθεντικοποίηση Συμμετεχόντων	51
6.9.2	Ακεραιότητα Δεδομένων	52
6.9.3	Forward Secrecy	52
6.9.4	Backward Secrecy	52
6.9.5	Key Independence	53
6.9.6	Ανθεκτικότητα σε Replay Attacks	53
6.9.7	Ανθεκτικότητα σε Man-in-the-Middle Επιθέσεις	53
6.9.8	Ανθεκτικότητα σε Επιθέσεις Συνωμοσίας	53
6.9.9	Ακεραιότητα του Blockchain	54
6.9.10	Σχέση με την Υπόθεση Bilinear Diffie--Hellman	54
7	Τεχνολογίες Υλοποίησης	55
7.1	Spring Boot	55
7.2	RESTful Web Services	55
7.3	Apache ActiveMQ Artemis	55
7.4	Java Message Service (JMS)	56
7.5	Docker	56
7.6	JPBC Library	56

8	Σχεδιασμός Εφαρμογής	58
8.1	Hyperledger Fabric Network	58
8.2	Organizations	58
8.3	Ροή Εκτέλεσης του Πρωτοκόλλου	59
8.3.1	Τύποι Μηνυμάτων	59
8.3.2	Φάση Εκκίνησης (START)	60
8.3.3	Φάση Καθορισμού Συμμετεχόντων	60
8.3.4	Φάση Ανταλλαγής Δημοσίων Κλειδιών	60
8.3.5	Φάση Δημιουργίας Blocks	60
8.3.6	Φάση Υπολογισμού Group Key	61
8.3.7	Υπολογισμός του Τελικού Κλειδιού	61
8.3.8	Φάση Τελικής Επαλήθευσης	61
8.3.9	Υλοποίηση Chaincode	61
8.4	Repository Layer	62
8.5	Service Layer	62
8.6	Αρχικοποίηση Pairings	62
8.6.1	Υλοποίηση BLS Υπογραφών	62
9	Πειράματα και Μετρήσεις	63
9.1	Περιβάλλον Πειραμάτων	63
9.2	Μεθοδολογία Μετρήσεων	64
9.3	Επεκτασιμότητα πρωτοκόλλου	64
9.4	Επικοινωνιακό κόστος	64
9.5	Συμπεράσματα	65
10	Πηγαίος Κώδικας	66
11	Βιβλιογραφία	84

Κεφάλαιο 1

Εισαγωγή

Η ασφαλής επικοινωνία σε κατανεμημένα συστήματα αποτελεί ένα από τα θεμελιώδη προβλήματα της σύγχρονης κρυπτογραφίας. Σε περιβάλλοντα όπου πολλοί χρήστες ή κόμβοι πρέπει να συνεργαστούν, η δημιουργία ενός κοινού μυστικού κλειδιού με ασφαλή και αποδοτικό τρόπο είναι απαραίτητη για την προστασία της επικοινωνίας.

Τα πρωτόκολλα συμφωνίας ομαδικού κλειδιού (Group Key Agreement protocols) επιτρέπουν σε ένα σύνολο συμμετεχόντων να καταλήξουν σε ένα κοινό κρυπτογραφικό κλειδί μέσω ενός μη ασφαλούς καναλιού επικοινωνίας, χωρίς να απαιτείται πλήρης εμπιστοσύνη σε έναν κεντρικό φορέα. Ωστόσο, η υλοποίηση τέτοιων πρωτοκόλλων σε πραγματικά αποκεντρωμένα περιβάλλοντα συνοδεύεται από προκλήσεις που σχετίζονται με το επικοινωνιακό κόστος, την υπολογιστική πολυπλοκότητα και την ανθεκτικότητα σε κακόβουλες ενέργειες.

Τα τελευταία χρόνια, η τεχνολογία blockchain έχει χρησιμοποιηθεί ως μηχανισμός ενίσχυσης της ασφάλειας σε κατανεμημένα συστήματα, παρέχοντας ιδιότητες όπως ακεραιότητα, διαφάνεια και μη αποποίηση ευθύνης. Πλατφόρμες όπως το Hyperledger Fabric επιτρέπουν την ανάπτυξη permissioned blockchain δικτύων, τα οποία είναι κατάλληλα για επιχειρησιακές και ελεγχόμενες εφαρμογές.

Στο πλαίσιο αυτό, η παρούσα διπλωματική εργασία υλοποιεί και αξιολογεί το πρωτόκολλο Blockchain Dynamic Group Key Agreement (BDGKA), το οποίο έχει προταθεί στη βιβλιογραφία για τη δημιουργία ασφαλούς ομαδικού κλειδιού σε αποκεντρωμένα περιβάλλοντα. Το πρωτόκολλο συνδυάζει τεχνικές βασισμένες σε bilinear pairings με υποδομή blockchain, με στόχο την εξασφάλιση επαληθεύσιμης και ανθεκτικής διαδικασίας ανταλλαγής μηνυμάτων.

Η κύρια συμβολή της παρούσας εργασίας δεν έγκειται στον σχεδιασμό νέου πρωτοκόλλου, αλλά στην υλοποίηση, ανάλυση και πειραματική αξιολόγηση της απόδοσης του BDGKA σε πραγματικό περιβάλλον. Συγκεκριμένα, μελετάται η συμπεριφορά του πρωτοκόλλου ως προς τον χρόνο εκτέλεσης, το επικοινωνιακό κόστος καθώς και η κλιμάκωσή του σε συνάρτηση με τον αριθμό των συμμετεχόντων κόμβων.

Η υλοποίηση πραγματοποιήθηκε σε περιβάλλον Linux με χρήση Docker containers για την ανάπτυξη δικτύου Hyperledger Fabric. Οι κόμβοι του συστήματος υλοποιήθηκαν σε Java με χρήση του Spring Boot, ενώ οι κρυπτογραφικές λειτουργίες βασίστηκαν στη βιβλιοθήκη JPBC για την υποστήριξη bilinear pairings και πράξεων σε ελλειπτικές καμπύλες.

1.1 Στόχοι της Εργασίας

Στόχος της παρούσας διπλωματικής εργασίας είναι η υλοποίηση και η πειραματική αξιολόγηση του πρωτοκόλλου Blockchain Dynamic Group Key Agreement σε ένα πραγματικό περιβάλλον

blockchain βασισμένο στο Hyperledger Fabric.

Αρχικά πραγματοποιείται η μελέτη του θεωρητικού και μαθηματικού υποβάθρου που απαιτείται για την κατανόηση του πρωτοκόλλου. Συγκεκριμένα παρουσιάζονται βασικές έννοιες της κρυπτογραφίας δημοσίου κλειδιού, των ελλειπτικών καμπυλών, των διγραμμικών απεικονίσεων, καθώς και των πρωτοκόλλων συμφωνίας κλειδιού ομάδας.

Στη συνέχεια αναλύεται η λειτουργία του πρωτοκόλλου BDGKA και παρουσιάζονται τα επιμέρους στάδια εκτέλεσής του. Ιδιαίτερη έμφαση δίνεται στον τρόπο δημιουργίας του ομαδικού κλειδιού, στους μηχανισμούς επαλήθευσης των συμμετεχόντων και στις ιδιότητες ασφαλείας που παρέχει το πρωτόκολλο.

Έπειτα σχεδιάζεται και αναπτύσσεται μία ολοκληρωμένη υλοποίηση του πρωτοκόλλου με χρήση της γλώσσας Java και του πλαισίου Spring Boot. Η υλοποίηση ενσωματώνεται σε δίκτυο Hyperledger Fabric, όπου τα παραγόμενα δεδομένα αποθηκεύονται στο blockchain μέσω κατάλληλου έξυπνου συμβολαίου.

Τέλος, πραγματοποιείται πειραματική αξιολόγηση της υλοποίησης μέσω μετρήσεων απόδοσης.

1.2 Δομή της Εργασίας

Η παρούσα εργασία οργανώνεται σε δέκα κεφάλαια.

Στο δεύτερο κεφάλαιο παρουσιάζεται το μαθηματικό και κρυπτογραφικό υπόβαθρο που απαιτείται για την κατανόηση του πρωτοκόλλου. Αναλύονται οι αλγεβρικές δομές, οι συναρτήσεις κατακερματισμού, τα πρωτόκολλα ανταλλαγής κλειδιών, οι ελλειπτικές καμπύλες, τα υπολογιστικά προβλήματα ασφάλειας και οι διγραμμικές απεικονίσεις.

Στο τρίτο κεφάλαιο παρουσιάζονται τα πρωτόκολλα ομαδικής συμφωνίας κλειδιού, οι βασικές απαιτήσεις ασφαλείας τους, καθώς και οι σημαντικότερες κατηγορίες και προσεγγίσεις που έχουν προταθεί στη βιβλιογραφία.

Στο τέταρτο κεφάλαιο παρουσιάζονται οι βασικές έννοιες της τεχνολογίας blockchain, οι κατηγορίες blockchain δικτύων, οι μηχανισμοί συναίνεσης, τα έξυπνα συμβόλαια και οι μηχανισμοί διαχείρισης ταυτότητας.

Στο πέμπτο κεφάλαιο αναλύεται η πλατφόρμα Hyperledger Fabric. Περιγράφονται η αρχιτεκτονική της, τα βασικά συστατικά της, οι μηχανισμοί ασφάλειας και οι λειτουργίες που προσφέρει για την ανάπτυξη permissioned blockchain εφαρμογών.

Στο έκτο κεφάλαιο παρουσιάζεται το πρωτόκολλο Blockchain Dynamic Group Key Agreement (BDGKA). Αναλύεται το μοντέλο λειτουργίας του, η διαδικασία δημιουργίας και διαχείρισης κλειδιών, η παραγωγή του ομαδικού κλειδιού, η ενσωμάτωση του blockchain καθώς και η ανάλυση ορθότητας και ασφάλειας του πρωτοκόλλου.

Στο έβδομο κεφάλαιο παρουσιάζονται οι τεχνολογίες που χρησιμοποιήθηκαν για την ανάπτυξη της υλοποίησης, όπως το Spring Boot, το Apache ActiveMQ Artemis, το Java Message Service, το Docker και η βιβλιοθήκη JPBC.

Στο όγδοο κεφάλαιο περιγράφεται ο σχεδιασμός της εφαρμογής και η αρχιτεκτονική του συστήματος. Παρουσιάζεται το δίκτυο Hyperledger Fabric, η ροή εκτέλεσης του πρωτοκόλλου, η δομή των λογισμικών επιπέδων και η υλοποίηση των κρυπτογραφικών μηχανισμών.

Στο ένατο κεφάλαιο παρουσιάζεται η πειραματική αξιολόγηση της υλοποίησης. Περιγράφεται το περιβάλλον εκτέλεσης, η μεθοδολογία των μετρήσεων και αναλύονται τα αποτελέσματα που προέκυψαν από τα πειράματα.

Τέλος, στο δέκατο κεφάλαιο παρατίθενται ενδεικτικά τμήματα του πηγαίου κώδικα που αναπτύχθηκαν στο πλαίσιο της εργασίας.

Κεφάλαιο 2

Μαθηματικό Υπόβαθρο

Το παρόν κεφάλαιο παρουσιάζει το απαραίτητο μαθηματικό και κρυπτογραφικό υπόβαθρο στο οποίο βασίζεται η προτεινόμενη υλοποίηση του πρωτοκόλλου. Συγκεκριμένα, αναλύονται βασικές έννοιες από τη θεωρία αριθμών, τις αλγεβρικές δομές, τις ελλειπτικές καμπύλες, τις συναρτήσεις κατακερματισμού, το πρωτόκολλο Diffie-Hellman, τις bilinear pairings και την τεχνολογία blockchain.

2.1 Ομάδες (Groups)

Έστω G ένα μη κενό σύνολο εφοδιασμένο με μια δυαδική πράξη $\cdot : G \times G \rightarrow G$. Το ζεύγος $(G, \cdot)$ καλείται *ομάδα* εάν ικανοποιούνται τα ακόλουθα αξιώματα:

Υπάρχει στοιχείο $e \in G$, το οποίο ονομάζεται *ουδέτερο στοιχείο*, τέτοιο ώστε για κάθε $a \in G$ να ισχύει

$$e \cdot a = a \cdot e = a.$$

Για κάθε $a \in G$ υπάρχει στοιχείο $a^{-1} \in G$, το οποίο ονομάζεται *αντίστροφο του a* , τέτοιο ώστε

$$a \cdot a^{-1} = a^{-1} \cdot a = e.$$

Επιπλέον, η πράξη είναι προσεταιριστική, δηλαδή για κάθε $a, b, c \in G$ ισχύει

$$(a \cdot b) \cdot c = a \cdot (b \cdot c).$$

Μια ομάδα G λέγεται *αβελιανή* όταν η πράξη είναι αντιμεταθετική, δηλαδή

$$a \cdot b = b \cdot a, \quad \forall a, b \in G.$$

2.1.1 Κυκλικές Ομάδες και Γεννήτορες

Έστω G μια ομάδα και $g \in G$. Το υποσύνολο

$$\langle g \rangle = \{g^n \mid n \in \mathbb{Z}\}$$

είναι η μικρότερη υποομάδα του G που περιέχει το g και καλείται *κυκλική υποομάδα* που παράγεται από το g .

Αν ισχύει ότι $G = \langle g \rangle$ για κάποιο $g \in G$, τότε η ομάδα G καλείται *κυκλική ομάδα* και το στοιχείο g ονομάζεται *γεννήτορας* της G .

Γενικότερα, αν $S \subseteq G$, τότε η υποομάδα που παράγεται από το S , συμβολίζεται με $\langle S \rangle$, και ορίζεται ως η τομή όλων των υποομάδων του G που περιέχουν το S .

2.1.2 Πεπερασμένα Σώματα

Ένα σώμα F είναι ένα σύνολο εφοδιασμένο με δύο πράξεις $+$ και $\cdot$ έτσι ώστε:

Το $(F, +)$ να είναι αβελιανή ομάδα με ουδέτερο στοιχείο το 0, και το $(F \setminus \{0\}, \cdot)$ να είναι επίσης αβελιανή ομάδα με ουδέτερο στοιχείο το 1. Επιπλέον, για κάθε $a, b, c \in F$ ισχύει η επιμεριστική ιδιότητα

$$a(b + c) = ab + ac.$$

Ένα σώμα F καλείται *πεπερασμένο σώμα* όταν το σύνολό του είναι πεπερασμένο. Είναι γνωστό ότι η τάξη ενός πεπερασμένου σώματος είναι της μορφής

$$|F| = p^n,$$

όπου p είναι πρώτος αριθμός και $n \in \mathbb{N}$. Σε αυτή την περίπτωση το σώμα συμβολίζεται συνήθως ως $\mathbb{F}_{p^n}$.

2.1.3 Αριθμητική Modulo

Έστω $n \geq 2$ ακέραιος. Ορίζουμε τη σχέση ισοδυναμίας στο $\mathbb{Z}$ ως

$$a \equiv b \pmod{n} \iff n \mid (a - b).$$

Η σχέση αυτή διαμερίζει το σύνολο των ακεραίων σε κλάσεις ισοδυναμίας. Το σύνολο των κλάσεων συμβολίζεται με $\mathbb{Z}_n$ και γράφεται

$$\mathbb{Z}_n = \{[0], [1], \dots, [n-1]\}.$$

Οι πράξεις πρόσθεσης και πολλαπλασιασμού ορίζονται μέσω αντιπροσώπων ως

$$[a] + [b] = [a + b], \quad [a] \cdot [b] = [ab].$$

Με τις πράξεις αυτές, το $\mathbb{Z}_n$ αποτελεί δακτύλιο με μοναδιαίο στοιχείο. Επιπλέον, το $\mathbb{Z}_n$ είναι σώμα αν και μόνο αν ο n είναι πρώτος αριθμός.

2.2 Συναρτήσεις Κατακερματισμού

Μια κρυπτογραφική συνάρτηση κατακερματισμού H δέχεται ως είσοδο ένα μήνυμα αυθαίρετου μεγέθους και παράγει μια έξοδο σταθερού μήκους. Η ιδιότητα αυτή επιτρέπει τη συμπίεση δεδομένων σε σταθερή αναπαράσταση ανεξαρτήτως μεγέθους εισόδου.

Επιπλέον, οι συναρτήσεις κατακερματισμού είναι μονόδρομες, δηλαδή είναι υπολογιστικά εύκολο να υπολογιστεί η τιμή $H(x)$ για δεδομένο x , ενώ είναι πρακτικά αδύνατη η αντίστροφη διαδικασία εύρεσης του x από την τιμή $H(x)$. Για τον λόγο αυτό χρησιμοποιούνται ευρέως σε ψηφιακές υπογραφές, πρωτόκολλα ταυτοποίησης και συστήματα blockchain.

2.3 Πρωτόκολλα Ανταλλαγής Κλειδιών

2.3.1 Πρωτόκολλο Diffie--Hellman

Το πρωτόκολλο Diffie--Hellman (DH) επιτρέπει σε δύο μέρη, έστω A και B , να συμφωνήσουν σε ένα κοινό μυστικό κλειδί μέσω ενός μη ασφαλούς καναλιού.

Έστω G μια κυκλική ομάδα τάξης πρώτου q με γεννήτορα g . Τα βήματα του πρωτοκόλλου είναι τα εξής:

Ο χρήστης A επιλέγει τυχαίο $a \in \mathbb{Z}_q$ και υπολογίζει:

$$A = g^a,$$

το οποίο αποστέλλει στον B .

Ο χρήστης B επιλέγει τυχαίο $b \in \mathbb{Z}_q$ και υπολογίζει:

$$B = g^b,$$

το οποίο αποστέλλει στον A .

Το κοινό μυστικό κλειδί υπολογίζεται ως:

$$K = B^a = (g^b)^a = g^{ab} = (g^a)^b = A^b.$$

Η ασφάλεια του πρωτοκόλλου βασίζεται στην υπόθεση Computational Diffie--Hellman (CDH).

2.3.2 Ελλειπτική Εκδοχή Diffie--Hellman (ECDH)

Στο Elliptic Curve Diffie--Hellman (ECDH), η ομάδα G αντικαθίσταται από την αβελιανή ομάδα σημείων μιας ελλειπτικής καμπύλης $E(F)$.

Έστω $P \in E(F)$ σημείο γεννήτορας. Ο χρήστης A επιλέγει μυστικό $a \in \mathbb{Z}_q$ και υπολογίζει:

$$A = aP,$$

ενώ ο χρήστης B επιλέγει $b \in \mathbb{Z}_q$ και υπολογίζει:

$$B = bP.$$

Το κοινό μυστικό κλειδί προκύπτει ως:

$$K = aB = a(bP) = abP = b(aP) = bA.$$

Η ασφάλεια του πρωτοκόλλου βασίζεται στην υπολογιστική δυσκολία του Elliptic Curve Discrete Logarithm Problem (ECDLP).

2.3.3 Group Diffie--Hellman

Το Group Diffie--Hellman επεκτείνει το κλασικό DH σε ομάδα n συμμετεχόντων $P_1, P_2, \dots, P_n$ σε κυκλική ομάδα G .

Κάθε συμμετέχων P_i επιλέγει ιδιωτικό μυστικό $x_i \in \mathbb{Z}_q$ και υπολογίζει g^{x_i} .

Μέσω διαδοχικών ανταλλαγών και επαναληπτικών εκθετικοποιήσεων, οι συμμετέχοντες καταλήγουν σε κοινό μυστικό:

$$K = g^{x_1 x_2 \dots x_n}.$$

Η πολυπλοκότητα του πρωτοκόλλου αυξάνεται με τον αριθμό των συμμετεχόντων, τόσο σε επικοινωνιακό όσο και σε υπολογιστικό επίπεδο.

2.3.4 Contributory Key Agreement

Τα πρωτόκολλα *contributory key agreement* αποτελούν γενίκευση των group key exchange μηχανισμών, όπου το τελικό κλειδί εξαρτάται ισοδύναμα από τις συνεισφορές όλων των συμμετεχόντων και όχι από έναν κεντρικό παράγοντα.

Τυπικά, κάθε συμμετέχων P_i συνεισφέρει μια τιμή x_i , και το τελικό κλειδί K είναι συνάρτηση όλων των συνεισφορών:

$$K = f(x_1, x_2, \dots, x_n),$$

όπου η συνάρτηση f είναι τέτοια ώστε κανένα υποσύνολο λιγότερο από όλους τους συμμετέχοντες να μην μπορεί να υπολογίσει το K .

Παραδείγματα τέτοιων πρωτοκόλλων περιλαμβάνουν επεκτάσεις του Diffie--Hellman σε ομάδες καθώς και πρωτόκολλα βασισμένα σε ελλειπτικές καμπύλες.

Η ασφάλεια βασίζεται συνήθως σε παραλλαγές του CDH ή του ECDLP, ανάλογα με το αλγεβρικό πλαίσιο στο οποίο υλοποιούνται.

2.3.5 Υπολογιστική Ασφάλεια και Δύσκολα Μαθηματικά Προβλήματα

Η ασφάλεια των σύγχρονων κρυπτογραφικών πρωτοκόλλων βασίζεται στην ύπαρξη μαθηματικών προβλημάτων των οποίων η επίλυση θεωρείται υπολογιστικά μη εφικτή με τα διαθέσιμα υπολογιστικά μέσα. Αντί να στηρίζεται στη μυστικότητα του αλγορίθμου, η σύγχρονη κρυπτογραφία βασίζεται στη δυσκολία αντιστροφής συγκεκριμένων μαθηματικών πράξεων.

Ένα από τα σημαντικότερα προβλήματα είναι το Πρόβλημα του Διακριτού Λογαρίθμου (Discrete Logarithm Problem - DLP). Δοθέντων μιας κυκλικής ομάδας G , ενός γεννήτορα g και ενός στοιχείου h , ζητείται ο υπολογισμός του εκθέτη x έτσι ώστε:

$$h = g^x \tag{2.1}$$

Παρότι ο υπολογισμός του h από το x είναι εύκολος, η αντίστροφη διαδικασία θεωρείται εξαιρετικά δύσκολη όταν χρησιμοποιούνται κατάλληλες παράμετροι.

Πάνω σε αυτή τη δυσκολία βασίζεται και το πρόβλημα Computational Diffie-Hellman (CDH). Δοθέντων των τιμών g , g^a και g^b , ένας επιτιθέμενος καλείται να υπολογίσει την τιμή:

$$g^{ab} \tag{2.2}$$

χωρίς να γνωρίζει τα μυστικά a και b .

Μία ακόμη ισχυρότερη υπόθεση είναι το Decisional Diffie-Hellman (DDH), σύμφωνα με το οποίο ένας επιτιθέμενος καλείται να αποφασίσει αν μία τιμή g^c ισούται με g^{ab} ή αποτελεί τυχαίο στοιχείο της ομάδας.

Οι παραπάνω υποθέσεις αποτελούν τη θεωρητική βάση πολλών πρωτοκόλλων ανταλλαγής κλειδιών και χρησιμοποιούνται ευρέως τόσο στην κλασική κρυπτογραφία όσο και στα σχήματα που βασίζονται σε διγραμμικά ζευγαρώματα.

2.4 Κρυπτογραφία Ελλειπτικών Καμπυλών

Οι ελλειπτικές καμπύλες αποτελούν μία από τις σημαντικότερες εξελίξεις της σύγχρονης κρυπτογραφίας. Η χρήση τους επιτρέπει την επίτευξη υψηλού επιπέδου ασφάλειας με σημαντικά μικρότερα μεγέθη κλειδιών σε σύγκριση με κλασικά συστήματα όπως το RSA.

Μία ελλειπτική καμπύλη πάνω από ένα πεπερασμένο σώμα περιγράφεται από την εξίσωση:

$$y^2 = x^3 + ax + b \quad (2.3)$$

όπου οι παράμετροι a και b επιλέγονται κατά τρόπο ώστε η καμπύλη να μην παρουσιάζει ιδιομορφίες.

Τα σημεία της καμπύλης μαζί με ένα ειδικό σημείο στο άπειρο σχηματίζουν μία αβελιανή ομάδα. Η ιδιότητα αυτή επιτρέπει την εκτέλεση βασικών πράξεων όπως η πρόσθεση σημείων και ο βαθμωτός πολλαπλασιασμός, οι οποίες αποτελούν το θεμέλιο των κρυπτογραφικών πρωτοκόλλων που βασίζονται σε ελλειπτικές καμπύλες.

Η ασφάλεια των συστημάτων αυτών βασίζεται στο πρόβλημα του διακριτού λογαρίθμου πάνω σε ελλειπτικές καμπύλες (Elliptic Curve Discrete Logarithm Problem - ECDLP). Το πρόβλημα αυτό θεωρείται σημαντικά δυσκολότερο ανά bit σε σχέση με το αντίστοιχο πρόβλημα σε κλασικές ομάδες πεπερασμένων σωμάτων.

Για τον λόγο αυτό, ένα κλειδί ECC μεγέθους 256 bit θεωρείται ότι προσφέρει επίπεδο ασφάλειας αντίστοιχο με ένα κλειδί RSA μεγέθους 3072 bit, γεγονός που καθιστά την ECC ιδιαίτερα αποδοτική σε εφαρμογές με περιορισμένους πόρους.

2.4.1 Ελλειπτικές Καμπύλες και Σημεία

Μια ελλειπτική καμπύλη E ορισμένη πάνω από ένα σώμα F (συνήθως πεπερασμένο σώμα $\mathbb{F}_p$ ή $\mathbb{F}_{p^n}$) δίνεται σε απλουστευμένη μορφή Weierstrass ως:

$$E : y^2 = x^3 + ax + b,$$

όπου $a, b \in F$ και ικανοποιείται η συνθήκη μη εκφυλισμού:

$$4a^3 + 27b^2 \neq 0.$$

Το σύνολο των σημείων της καμπύλης ορίζεται ως:

$$E(F) = \{(x, y) \in F \times F \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\},$$

όπου $\mathcal{O}$ είναι το ουδέτερο στοιχείο, το λεγόμενο *σημείο στο άπειρο*.

2.4.2 Γεωμετρική και Αλγεβρική Δομή

Το σύνολο $E(F)$ εφοδιασμένο με μια κατάλληλα ορισμένη πράξη σημείων σχηματίζει αβελιανή ομάδα με ουδέτερο στοιχείο το $\mathcal{O}$. Η πράξη αυτή ορίζεται γεωμετρικά ως εξής:

Για δύο σημεία $P, Q \in E(F)$, η ευθεία που τα ενώνει τέμνει την καμπύλη σε ένα τρίτο σημείο. Το συμμετρικό αυτού του σημείου ως προς τον άξονα x ορίζει το άθροισμα:

$$P + Q = R.$$

Αν $P = Q$, τότε η πράξη ορίζεται μέσω της εφαπτομένης της καμπύλης στο σημείο P . Το $\mathcal{O}$ λειτουργεί ως ουδέτερο στοιχείο:

$$P + \mathcal{O} = P, \quad \forall P \in E(F).$$

Επιπλέον, για κάθε $P = (x, y)$, το αντίστροφο είναι:

$$-P = (x, -y).$$

2.4.3 Πολλαπλασιασμός με Βαθμωτό (Scalar Multiplication)

Για ένα ακέραιο $k \in \mathbb{Z}$ και ένα σημείο $P \in E(F)$, ορίζουμε τον βαθμωτό πολλαπλασιασμό ως:

$$kP = \underbrace{P + P + \dots + P}_{k \text{ φορές}}.$$

Ο υπολογισμός αυτός υλοποιείται αποδοτικά μέσω της μεθόδου διπλασιασμού και πρόσθεσης (*double-and-add*).

Ο scalar multiplication αποτελεί τον θεμελιώδη αλγόριθμο πάνω στον οποίο βασίζεται η κρυπτογραφία ελλειπτικών καμπυλών.

2.4.4 Ασφάλεια της ECC

Η ασφάλεια των συστημάτων ελλειπτικών καμπυλών βασίζεται στο υπολογιστικά δύσκολο πρόβλημα του Διακριτού Λογαρίθμου Ελλειπτικής Καμπύλης (*Elliptic Curve Discrete Logarithm Problem - ECDLP*).

Πιο συγκεκριμένα, δεδομένων των σημείων $P, Q \in E(F)$, όπου:

$$Q = kP,$$

είναι υπολογιστικά δύσκολο να βρεθεί ο ακέραιος k , ακόμη και αν είναι γνωστά τα P και Q .

Η δυσκολία του προβλήματος ECDLP αποτελεί τη βάση της ασφάλειας της :contentReference[oaicite:0]in και επιτρέπει μικρότερα κλειδιά σε σχέση με κλασικά κρυπτοσυστήματα όπως το RSA, διατηρώντας ισοδύναμο επίπεδο ασφάλειας.

2.4.5 Πλεονεκτήματα της Κρυπτογραφίας Ελλειπτικών Καμπυλών

Η κρυπτογραφία ελλειπτικών καμπυλών παρουσιάζει σημαντικά πλεονεκτήματα σε σχέση με παραδοσιακές προσεγγίσεις δημόσιου κλειδιού.

Αρχικά, προσφέρει ισοδύναμο επίπεδο ασφάλειας με σημαντικά μικρότερα κλειδιά. Η ιδιότητα αυτή μειώνει τις απαιτήσεις αποθήκευσης και περιορίζει τον όγκο των δεδομένων που μεταδίδονται κατά την εκτέλεση κρυπτογραφικών πρωτοκόλλων.

Επιπλέον, οι μικρότερες παράμετροι οδηγούν σε ταχύτερες κρυπτογραφικές πράξεις και μειωμένη κατανάλωση υπολογιστικών πόρων. Το χαρακτηριστικό αυτό είναι ιδιαίτερα σημαντικό σε περιβάλλοντα περιορισμένων δυνατοτήτων, όπως οι κινητές συσκευές, τα ενσωματωμένα συστήματα και οι συσκευές του Διαδικτύου των Πραγμάτων (Internet of Things - IoT).

Ένα ακόμη σημαντικό πλεονέκτημα είναι η ευρεία υιοθέτησή της από σύγχρονα πρότυπα ασφαλείας. Η ECC χρησιμοποιείται σε πρωτόκολλα όπως το TLS, σε συστήματα ψηφιακών υπογραφών, σε εικονικά ιδιωτικά δίκτυα (VPN), καθώς και σε πολλές πλατφόρμες blockchain.

Η αυξανόμενη χρήση των ελλειπτικών καμπυλών οδήγησε στην ανάπτυξη πιο προηγμένων κρυπτογραφικών τεχνικών, όπως η Pairing-Based Cryptography. Οι τεχνικές αυτές αξιοποιούν ειδικές κατηγορίες ελλειπτικών καμπυλών και διγραμμικών απεικονίσεων για την υλοποίηση σύνθετων λειτουργιών, όπως η ανταλλαγή ομαδικών κλειδιών, η κρυπτογραφία βασισμένη στην ταυτότητα και οι σύντομες ψηφιακές υπογραφές.

2.5 Δύσκολα Υπολογιστικά Προβλήματα

2.5.1 Πρόβλημα Διακριτού Λογαρίθμου (Discrete Logarithm Problem)

Έστω G μια κυκλική ομάδα πολλαπλασιαστικής γραφής, γεννήτορας $g \in G$ και στοιχείο $h \in G$. Το Πρόβλημα Διακριτού Λογαρίθμου (Discrete Logarithm Problem - DLP) ορίζεται ως η εύρεση ενός ακεραίου $x \in \mathbb{Z}$ τέτοιου ώστε:

$$g^x = h.$$

Η υπολογιστική δυσκολία του προβλήματος εξαρτάται από την επιλογή της ομάδας G και αποτελεί θεμελιώδη βάση για πλήθος κρυπτογραφικών πρωτοκόλλων.

2.5.2 Υπολογιστικό Πρόβλημα Diffie--Hellman (Computational Diffie--Hellman)

Έστω G κυκλική ομάδα τάξης q με γεννήτορα g . Δίνονται τα στοιχεία:

$$(g, g^a, g^b),$$

όπου $a, b \in \mathbb{Z}_q$ είναι άγνωστα.

Το *Computational Diffie--Hellman problem (CDH)* συνίσταται στον υπολογισμό του:

$$g^{ab}.$$

Το πρόβλημα θεωρείται υπολογιστικά δύσκολο σε κατάλληλες ομάδες, ακόμη και όταν τα g^a και g^b είναι γνωστά.

2.5.3 Αποφασιστικό Πρόβλημα Diffie--Hellman (Decisional Diffie--Hellman)

Έστω G κυκλική ομάδα τάξης q με γεννήτορα g . Δίνονται τα στοιχεία:

$$(g, g^a, g^b, g^c),$$

όπου $a, b, c \in \mathbb{Z}_q$.

Το *Decisional Diffie--Hellman problem (DDH)* είναι το πρόβλημα απόφασης αν:

$$c = ab \pmod{q}$$

ή ισοδύναμα αν:

$$g^c = g^{ab}.$$

Το πρόβλημα απαιτεί τη διάκριση μεταξύ του g^{ab} και ενός τυχαίου στοιχείου της ομάδας G .

2.5.4 Διγραμμικό Πρόβλημα Diffie--Hellman (Bilinear Diffie--Hellman)

Έστω (G_1, G_2, G_T) κυκλικές ομάδες τάξης πρώτου p , και μια διγραμμική απεικόνιση:

$$e : G_1 \times G_2 \rightarrow G_T.$$

Έστω $g \in G_1$ και $a, b, c \in \mathbb{Z}_p$. Δίνονται τα στοιχεία:

$$(g, g^a, g^b, g^c).$$

Το *Bilinear Diffie--Hellman problem (BDH)* συνίσταται στον υπολογισμό του:

$$e(g, g)^{abc} \in G_T.$$

Η δυσκολία του προβλήματος εξαρτάται από τις ιδιότητες της διγραμμικής απεικόνισης και αποτελεί θεμελιώδες υπόβαθρο για ζευγοποιήσεις (pairings) σε σύγχρονα κρυπτογραφικά πρωτόκολλα.

2.5.5 Υπολογιστικές Υποθέσεις και Προβλήματα: Σχέση CDH και DDH

Στην κρυπτογραφία βασισμένη σε ομάδες, είναι θεμελιώδες να διακρίνουμε μεταξύ υπολογιστικών προβλημάτων και υπολογιστικών υποθέσεων (assumptions). Ένα υπολογιστικό πρόβλημα ορίζεται τυπικά ως μια συνάρτηση ή σχέση που πρέπει να υπολογιστεί ή να αποφασιστεί, ενώ μια υπόθεση αναφέρεται στην παραδοχή ότι κάποιο τέτοιο πρόβλημα είναι υπολογιστικά ανεπίλυτο εντός πολυωνυμικού χρόνου για κατάλληλες παραμέτρους ασφάλειας.

Στο πλαίσιο των κυκλικών ομάδων G τάξης πρώτου q , με γεννήτορα $g \in G$, τα προβλήματα Computational Diffie--Hellman (CDH) και Decisional Diffie--Hellman (DDH) αποτελούν δύο θεμελιώδεις έννοιες με στενή αλλά μη ισοδύναμη σχέση.

Το πρόβλημα CDH ως υπολογιστική υπόθεση

Το *Computational Diffie--Hellman problem (CDH)* δίνεται από τριάδα εισόδου:

$$(g, g^a, g^b), \quad a, b \in \mathbb{Z}_q,$$

και απαιτεί τον υπολογισμό του στοιχείου:

$$g^{ab}.$$

Η υπόθεση CDH (CDH assumption) διατυπώνεται ως η παραδοχή ότι δεν υπάρχει αποδοτικός (πολυωνυμικός) αλγόριθμος που να επιλύει το παραπάνω πρόβλημα με μη αμελητέα πιθανότητα επιτυχίας σε κατάλληλα επιλεγμένες ομάδες.

Το πρόβλημα DDH ως πρόβλημα διάκρισης

Το *Decisional Diffie--Hellman problem (DDH)* είναι πρόβλημα απόφασης. Δεδομένου του τετραπλού:

$$(g, g^a, g^b, g^c),$$

ο στόχος είναι να αποφασιστεί αν ισχύει:

$$c = ab \pmod{q} \quad \text{ή ισοδύναμα} \quad g^c = g^{ab},$$

ή αν το g^c είναι τυχαίο στοιχείο της ομάδας G .

Η υπόθεση DDH (DDH assumption) διατυπώνει ότι δεν υπάρχει αποδοτικός αλγόριθμος που να διακρίνει το g^{ab} από ένα τυχαίο στοιχείο του G με μη αμελητέα διαφορά πιθανότητας.

Σχέση μεταξύ CDH και DDH

Οι δύο υποθέσεις σχετίζονται ιεραρχικά ως προς την ισχύ τους. Συγκεκριμένα, η ευκολία επίλυσης του DDH συνεπάγεται επίλυση του CDH σε κατάλληλες κατασκευές, ενώ το αντίστροφο δεν ισχύει γενικά.

Πιο τυπικά, ισχύει η σχέση:

$$\text{DDH} \Rightarrow \text{CDH},$$

υπό την έννοια ότι ένας αλγόριθμος που λύνει το DDH μπορεί να χρησιμοποιηθεί για την κατασκευή αλγορίθμου που λύνει το CDH σε κατάλληλες ομάδες.

Ωστόσο, υπάρχουν ομάδες όπου το CDH θεωρείται δύσκολο, ενώ το DDH είναι εύκολο ή άκυρο ως υπόθεση, γεγονός που αναδεικνύει ότι οι δύο υποθέσεις δεν είναι ισοδύναμες.

Η επιλογή κατάλληλης ομάδας είναι συνεπώς κρίσιμη για την εγκυρότητα των υποθέσεων ασφάλειας σε πρωτόκολλα που βασίζονται σε Diffie--Hellman, όπως το Diffie--Hellman key exchange και σύγχρονα συστήματα βασισμένα σε elliptic curves.

2.6 Pairing-Based Cryptography

Η κρυπτογραφία που βασίζεται σε διγραμμικές απεικονίσεις (Pairing-Based Cryptography) αποτελεί έναν από τους σημαντικότερους κλάδους της σύγχρονης κρυπτογραφίας δημοσίου κλειδιού. Η ανάπτυξή της ξεκίνησε στις αρχές της δεκαετίας του 2000, όταν διαπιστώθηκε ότι συγκεκριμένες διγραμμικές απεικονίσεις σε ελλειπτικές καμπύλες μπορούν να χρησιμοποιηθούν για την κατασκευή νέων κρυπτογραφικών πρωτοκόλλων που δεν ήταν εφικτά με τις παραδοσιακές τεχνικές.

Η βασική ιδέα της Pairing-Based Cryptography είναι η χρήση μιας ειδικής συνάρτησης η οποία συνδέει δύο στοιχεία μιας ελλειπτικής καμπύλης και παράγει ένα στοιχείο σε διαφορετική ομάδα. Η ιδιότητα της διγραμμικότητας επιτρέπει την εκτέλεση σύνθετων κρυπτογραφικών υπολογισμών, καθιστώντας δυνατή την υλοποίηση πρωτοκόλλων ταυτοποίησης, ψηφιακών υπογραφών, κρυπτογραφίας βασισμένης στην ταυτότητα και πρωτοκόλλων ομαδικής συμφωνίας κλειδιών.

Στην παρούσα εργασία οι διγραμμικές απεικονίσεις αποτελούν το θεμέλιο του πρωτοκόλλου BDGKA, καθώς επιτρέπουν τον ασφαλή υπολογισμό ενός κοινού μυστικού κλειδιού μεταξύ πολλών συμμετεχόντων χωρίς την ύπαρξη κεντρικού διαχειριστή.

2.6.1 Bilinear Pairings

Έστω δύο κυκλικές ομάδες πρώτης τάξης G_1 και G_T με κοινή τάξη q . Μία διγραμμική απεικόνιση ορίζεται ως η συνάρτηση

$$e : G_1 \times G_1 \rightarrow G_T$$

η οποία αντιστοιχίζει δύο στοιχεία της ομάδας G_1 σε ένα στοιχείο της ομάδας G_T .

Η απεικόνιση αυτή πρέπει να είναι αποδοτικά υπολογίσιμη και να ικανοποιεί συγκεκριμένες μαθηματικές ιδιότητες. Η σημαντικότερη ιδιότητά της είναι ότι μετατρέπει πολλαπλασιασμούς εκθετών σε πράξεις στην ομάδα στόχο, γεγονός που επιτρέπει την κατασκευή προηγμένων κρυπτογραφικών μηχανισμών.

Για στοιχεία $P, Q \in G_1$ και εκθέτες $a, b \in \mathbb{Z}_q$ ισχύει:

$$e(aP, bQ) = e(P, Q)^{ab}$$

Η παραπάνω σχέση αποτελεί τη θεμελιώδη ιδιότητα των bilinear pairings και είναι η βάση όλων των pairing-based πρωτοκόλλων.

2.6.2 Ιδιότητες

Μία συνάρτηση e θεωρείται κατάλληλη για κρυπτογραφική χρήση μόνο εάν ικανοποιεί τις παρακάτω ιδιότητες.

Διγραμμικότητα (Bilinearity)

Για όλα τα $u \in G_1$, $v \in G_2$ και $a, b \in \mathbb{Z}_p$ ισχύει:

$$e(u^a, v^b) = e(u, v)^{ab}.$$

Ισοδύναμα:

$$e(u_1 u_2, v) = e(u_1, v) e(u_2, v), \quad e(u, v_1 v_2) = e(u, v_1) e(u, v_2).$$

Μη εκφυλισμός (Non-degeneracy)

Η απεικόνιση e δεν είναι τετριμμένη, δηλαδή:

$$\exists u \in G_1, v \in G_2 \text{ τέτοια ώστε } e(u, v) \neq 1.$$

Αυτό συνεπάγεται ότι η ζευγοποίηση διατηρεί μη τετριμμένη αλγεβρική πληροφορία.

Υπολογισιμότητα (Computability)

Υπάρχει αποδοτικός αλγόριθμος που υπολογίζει το $e(u, v)$ για κάθε $u \in G_1$, $v \in G_2$ σε πολυωνυμικό χρόνο ως προς το μέγεθος των παραμέτρων ασφάλειας.

2.6.3 Κατηγοριοποίηση Ζευγοποιήσεων

Οι ζευγοποιήσεις ταξινομούνται σε τρεις τύπους ανάλογα με τη σχέση μεταξύ των ομάδων G_1 και G_2 .

Type 1 (Συμμετρικές ζευγοποιήσεις)

Ισχύει $G_1 = G_2$. Πρόκειται για συμμετρικές ζευγοποιήσεις, οι οποίες είναι ιστορικά οι πρώτες που μελετήθηκαν, αλλά συχνά δεν είναι αποδοτικές σε σύγχρονες υλοποιήσεις.

Type 2

Υπάρχει ισομορφισμός μεταξύ G_1 και G_2 , αλλά ο ισομορφισμός δεν είναι αποδοτικά υπολογίσιμος. Αυτό δημιουργεί περιορισμούς στη σχεδίαση πρωτοκόλλων.

Type 3

Οι ομάδες G_1 και G_2 είναι διακριτές και δεν υπάρχει αποδοτικός ισομορφισμός μεταξύ τους. Αυτός ο τύπος θεωρείται ο πιο αποδοτικός και χρησιμοποιείται ευρέως στη σύγχρονη κρυπτογραφία.

2.6.4 Τύποι Pairings

Οι διγραμμικές απεικονίσεις διακρίνονται σε διαφορετικές κατηγορίες ανάλογα με τη δομή και τη σχέση μεταξύ των εμπλεκόμενων ομάδων. Στην περίπτωση όπου οι ομάδες εισόδου και στόχου ταυτίζονται, δηλαδή $G_1 = G_2$, η υλοποίηση είναι πιο απλή και συχνά πιο αποδοτική, καθώς δεν απαιτούνται πρόσθετες μετατροπές μεταξύ διαφορετικών αλγεβρικών δομών.

Σε μια πιο γενική περίπτωση, οι ομάδες G_1 και G_2 είναι διαφορετικές αλλά υπάρχει αποδοτικός υπολογισμός ισομορφισμός μεταξύ τους, συνήθως της μορφής

$$\phi : G_2 \rightarrow G_1.$$

Η ύπαρξη αυτής της απεικόνισης επιτρέπει τη μεταφορά υπολογισμών μεταξύ των ομάδων, χωρίς όμως να απαιτείται η ύπαρξη αντίστροφου ισομορφισμού.

Στην πιο γενική και κρυπτογραφικά ισχυρή περίπτωση, οι ομάδες G_1 και G_2 είναι διακριτές και δεν υπάρχει αποδοτικός υπολογισμός ισομορφισμός ούτε προς τη μία ούτε προς την άλλη κατεύθυνση. Η κατηγορία αυτή προσφέρει μεγαλύτερη ευελιξία και χρησιμοποιείται συχνά σε σύγχρονα pairing-based πρωτόκολλα, καθώς αποφεύγεται η ύπαρξη ανεπιθύμητων δομικών σχέσεων μεταξύ των ομάδων.

2.6.5 Weil Pairing

Το Weil pairing αποτελεί μία από τις πρώτες και θεμελιώδεις κατασκευές διγραμμικών απεικονίσεων πάνω σε ελλειπτικές καμπύλες και εισήχθη από τον André Weil στο πλαίσιο της αλγεβρικής γεωμετρίας. Η σημασία του έγκειται στο ότι παρέχει ένα θεωρητικό υπόβαθρο για την ανάπτυξη σύγχρονων κρυπτογραφικών pairings.

Στο πλαίσιο των ελλειπτικών καμπυλών, το Weil pairing ορίζεται πάνω σε σημεία πεπερασμένης τάξης και ικανοποιεί ιδιότητες διγραμμικότητας και εναλλαγής, οι οποίες το καθιστούν κατάλληλο για χρήση σε κρυπτογραφικά πρωτόκολλα.

Για τον υπολογισμό του Weil pairing στην πράξη χρησιμοποιείται ο αλγόριθμος του Miller, ο οποίος επιτρέπει τον αποδοτικό υπολογισμό συναρτήσεων της μορφής $f_{r,P}(Q)$. Ο υπολογισμός του pairing πραγματοποιείται συνήθως μέσω δύο εφαρμογών του Miller algorithm, ώστε να υπολογιστούν οι ποσότητες

$$f_{r,P}(Q) \quad \text{και} \quad f_{r,Q}(P),$$

οι οποίες στη συνέχεια συνδυάζονται για την παραγωγή του τελικού αποτελέσματος του ζευγαρώματος.

Κατά την εκτέλεση του αλγορίθμου μπορεί να εμφανιστούν ενδιάμεσα σημεία μη ορισμού (poles) ή μηδενισμοί στις υπολογιζόμενες συναρτήσεις. Η εμφάνιση τέτοιων περιπτώσεων συνδέεται με την αλγεβρική δομή των σημείων και υποδεικνύει ότι τα εμπλεκόμενα σημεία ανήκουν στην ίδια κυκλική υποομάδα.

Συγκεκριμένα, εάν ένα σημείο Q ανήκει στην υποομάδα που παράγεται από το P , δηλαδή αν υπάρχει $\lambda \in \mathbb{Z}_q^*$ τέτοιο ώστε $Q = \lambda P$, τότε λόγω της διγραμμικότητας και της αντισυμμετρίας

του Weil pairing ισχύει ότι

$$e_{W,r}(P, Q) = e_{W,r}(P, \lambda P) = e_{W,r}(P, P)^\lambda.$$

Ωστόσο, το Weil pairing είναι εναλλασσόμενο (alternating), γεγονός που συνεπάγεται ότι

$$e_{W,r}(P, P) = 1.$$

Κατά συνέπεια προκύπτει ότι

$$e_{W,r}(P, Q) = 1,$$

γεγονός που αντικατοπτρίζει τη δομική εξάρτηση των δύο σημείων μέσα στην ίδια κυκλική υποομάδα. Για τον υπολογισμό του Weil pairing στην πράξη, χρησιμοποιείται ο Αλγόριθμος του Miller δύο φορές, ώστε να υπολογιστούν οι τιμές:

$$f_{r,P}(Q) \quad \text{και} \quad f_{r,Q}(P)$$

Στη συνέχεια, από τον συνδυασμό των παραπάνω μεγεθών προκύπτει το τελικό αποτέλεσμα του ζευγαρώματος.

Κατά την εκτέλεση του αλγορίθμου, είναι πιθανό να παρουσιαστούν ενδιάμεσα μηδενικά ή πόλοι στις υπολογιζόμενες συναρτήσεις. Σε αυτή την περίπτωση, η διαδικασία υποδεικνύει ότι το σημείο Q ανήκει στον κυκλικό υποομάδα που παράγεται από το P , δηλαδή ότι είναι πολλαπλάσιο του P . Τότε προκύπτει ότι:

$$e_{W,r}(P, Q) = 1$$

Πράγματι, αν ισχύει ότι:

$$Q = \lambda P$$

τότε από τη γραμμικότητα και την εναλλαγή του Weil pairing έχουμε:

$$e_{W,r}(P, Q) = e_{W,r}(P, P)^\lambda$$

Ωστόσο, επειδή το Weil pairing είναι εναλλασσόμενο, ισχύει ότι:

$$e_{W,r}(P, P) = 1$$

και συνεπώς καταλήγουμε στο συμπέρασμα ότι:

$$e_{W,r}(P, Q) = 1$$

2.6.6 Tate Pairing

Η Tate pairing βασίζεται στη θεωρία των divisor classes και ορίζεται ως απεικόνιση:

$$e_r : E[r] \times E(F_q)/rE(F_q) \rightarrow F_q^*/(F_q^*)^r.$$

Σε πρακτικές εφαρμογές χρησιμοποιείται η βελτιωμένη εκδοχή (reduced Tate pairing), η οποία αποδίδει τιμές σε κυκλική υποομάδα.

Η Tate pairing είναι πιο αποδοτική από τη Weil pairing σε υπολογιστικό επίπεδο.

2.6.7 Ate Pairing

Η Ate pairing αποτελεί βελτιστοποίηση της Tate pairing και μειώνει το υπολογιστικό κόστος μειώνοντας τον εκθέτη της τελικής εκθετικοποίησης.

Χρησιμοποιείται ευρέως σε σύγχρονες βιβλιοθήκες ζευγοποιήσεων λόγω της υψηλής αποδοτικότητας.

2.6.8 Pairing-Friendly Καμπύλες

Barreto--Naehrig (BN) Curves

Οι καμπύλες BN είναι οικογένεια ελλειπτικών καμπυλών κατάλληλων για αποδοτικές ζευγοποιήσεις με embedding degree $k = 12$. Έχουν ευρεία χρήση σε πρακτικά συστήματα λόγω καλής ισορροπίας ασφάλειας και απόδοσης.

BLS Curves

Οι BLS καμπύλες αποτελούν άλλη σημαντική οικογένεια pairing-friendly καμπυλών και χρησιμοποιούνται ευρέως σε πρωτόκολλα υπογραφών βασισμένων σε ζευγοποιήσεις (BLS signatures).

2.6.9 Πλεονεκτήματα και Μειονεκτήματα των Ζευγοποιήσεων

Οι διγραμμικές ζευγοποιήσεις επιτρέπουν την κατασκευή προηγμένων κρυπτογραφικών πρωτοκόλλων, όπως identity-based encryption, attribute-based encryption και short signatures.

Ωστόσο, παρουσιάζουν αυξημένο υπολογιστικό κόστος σε σχέση με κλασικά συστήματα βασισμένα σε discrete logarithm problems. Επιπλέον, απαιτούν προσεκτική επιλογή καμπυλών ώστε να αποφεύγονται γνωστές επιθέσεις που εκμεταλλεύονται το embedding degree και τις ιδιότητες της ομάδας στόχου G_T .

2.7 Group Key Agreement

Το *Group Key Agreement (GKA)* αποτελεί ένα θεμελιώδες κρυπτογραφικό πρωτόκολλο, το οποίο επιτρέπει σε ένα σύνολο από συμμετέχοντες κόμβους $U = \{U_1, U_2, \dots, U_n\}$ να συμφωνήσουν και να υπολογίσουν από κοινού ένα κοινό μυστικό κλειδί GK , το οποίο χρησιμοποιείται στη συνέχεια για την ασφαλή επικοινωνία μεταξύ τους. Η διαδικασία αυτή πραγματοποιείται μέσω ενός μη ασφαλούς καναλιού επικοινωνίας, γεγονός που σημαίνει ότι οποιοσδήποτε τρίτος μπορεί δυνητικά να παρακολουθεί την ανταλλαγή μηνυμάτων, χωρίς όμως να μπορεί να ανακτήσει το τελικό κλειδί.

Ένα βασικό χαρακτηριστικό του πρωτοκόλλου είναι η απουσία κεντρικής αρχής κατά τη φάση της δημιουργίας του κλειδιού. Δηλαδή, δεν υπάρχει ένας μοναδικός εξουσιοδοτημένος φορέας που να υπολογίζει ή να διανέμει το GK , αλλά αντίθετα όλοι οι συμμετέχοντες συμβάλλουν ενεργά στη διαδικασία παραγωγής του. Ως αποτέλεσμα, η τελική τιμή του κλειδιού προκύπτει από την αλληλεπίδραση όλων των συμμετεχόντων.

Ο στόχος του πρωτοκόλλου είναι όλοι οι νόμιμοι συμμετέχοντες να καταλήγουν στο ίδιο κοινό αποτέλεσμα, ανεξάρτητα από τη σειρά επικοινωνίας ή τον χρόνο συμμετοχής τους. Αυτό εκφράζεται μαθηματικά ως:

$$GK_{U_1} = GK_{U_2} = \dots = GK_{U_n} = GK$$

Η ασφάλεια του πρωτοκόλλου εξετάζεται υπό την παρουσία ενός ισχυρού και ενεργού αντιπάλου. Ο αντίπαλος αυτός θεωρείται ότι έχει τη δυνατότητα να παρακολουθεί πλήρως την επικοινωνία του δικτύου, να καταγράφει όλα τα μηνύματα που ανταλλάσσονται μεταξύ των κόμβων, καθώς και να επιχειρεί ενεργές επιθέσεις, όπως η τροποποίηση ή η επαναποστολή παλαιών μηνυμάτων (replay attacks). Επιπλέον, θεωρείται ότι μπορεί να αποκτήσει πρόσβαση σε ένα υποσύνολο των συμμετεχόντων κόμβων, οι οποίοι χαρακτηρίζονται ως παραβιασμένοι (corrupted nodes).

Παρά αυτές τις ισχυρές υποθέσεις επίθεσης, το πρωτόκολλο σχεδιάζεται έτσι ώστε να διατηρεί κρίσιμες ιδιότητες ασφάλειας. Πρώτον, εξασφαλίζεται η εμπιστευτικότητα (confidentiality), δηλαδή το κοινό μυστικό κλειδί GK δεν αποκαλύπτεται σε καμία μη εξουσιοδοτημένη οντότητα, ακόμη και αν αυτή παρακολουθεί το σύνολο της επικοινωνίας. Δεύτερον, παρέχεται forward secrecy, πράγμα που σημαίνει ότι ένας κόμβος ο οποίος αποχωρεί από το δίκτυο δεν μπορεί να ανακτήσει ή να υπολογίσει μελλοντικά κλειδιά που παράγονται μετά την αποχώρησή του. Αντίστοιχα, εξασφαλίζεται backward secrecy, ώστε ένας νέος κόμβος που εισέρχεται στο σύστημα να μην μπορεί να ανακτήσει προηγούμενα κοινά κλειδιά τα οποία χρησιμοποιήθηκαν πριν τη συμμετοχή του. Τέλος, το πρωτόκολλο παρουσιάζει ανθεκτικότητα σε συνεργασία κακόβουλων κόμβων (resistance to collusion), δηλαδή ακόμη και αν πολλοί παραβιασμένοι συμμετέχοντες συνεργαστούν, δεν είναι σε θέση να αποκαλύψουν το GK .

Η ασφάλεια του Group Key Agreement βασίζεται σε θεμελιώδη και ευρέως αποδεκτά υπολογιστικά δύσκολα προβλήματα της κρυπτογραφίας. Συγκεκριμένα, στηρίζεται στη δυσκολία επίλυσης του Discrete Logarithm Problem (DLP), καθώς και στο Computational Bilinear Diffie-Hellman (CBDH) πρόβλημα, τα οποία θεωρούνται υπολογιστικά μη εφικτά για επαρκώς μεγάλες παραμέτρους.

Με βάση τα παραπάνω, κάθε κόμβος του συστήματος συμμετέχει ενεργά στη διαδικασία παραγωγής και επεξεργασίας των ενδιάμεσων τιμών, με αποτέλεσμα, παρότι οι υπολογισμοί πραγματοποιούνται κατανεμημένα και ανεξάρτητα, όλοι οι συμμετέχοντες να καταλήγουν τελικά στο ίδιο κοινό μυστικό κλειδί GK .

Algorithm 1 Miller's algorithm (general version)

Input: $r \in \mathbb{N}$, $I = \lfloor \log r \rfloor$, $P = (x_P, y_P) \in E[r](K)$, $Q = (x_Q, y_Q) \in E(K)$

Output: $f_{r,P}(Q)$

1: Compute the binary decomposition:

$$r := \sum_{i=0}^I b_i 2^i$$

Let $T = P$, $f = 1$
2: **for** i in $[I - 1..0]$ **do**
3: $f = f^2 \mu_{T,T}(Q)$
4: $T = 2T$
5: **if** $b_i = 1$ **then**
6: $f = f \mu_{T,P}(Q)$
7: $T = T + P$
8: **end if**
9: **end for**
10: **return** f

Algorithm 2 Miller's Algorithm for Affine Short Weierstrass Coordinates

Require: $r \in \mathbb{N}$, $I = \lfloor \log r \rfloor$, $P = (x_P, y_P) \in E[r](K)$, $Q = (x_Q, y_Q) \in E(K)$

Ensure: $f_{r,P}(Q)$

1: Compute the binary decomposition

$$r = \sum_{i=0}^I b_i 2^i$$

2: $T \leftarrow P$

3: $f_1 \leftarrow 1$

4: $f_2 \leftarrow 1$

5: **for** $i = I - 1$ **downto** 0 **do**

6:

$$\alpha \leftarrow \frac{3x_T^2 + a}{2y_T}$$

▷ slope of the tangent of E at T

7:

$$x_{2T} \leftarrow \alpha^2 - 2x_T$$

8:

$$y_{2T} \leftarrow -y_T - \alpha(x_{2T} - x_T)$$

9:

$$f_1 \leftarrow f_1^2(y_Q - y_T - \alpha(x_Q - x_T))$$

10:

$$f_2 \leftarrow f_2^2(x_Q + 2x_T - \alpha^2)$$

11: $T \leftarrow 2T$

12: **if** $b_i = 1$ **then**

13:

$$\alpha \leftarrow \frac{y_T - y_P}{x_T - x_P}$$

▷ slope of the line through P and T

14:

$$x_{T+P} \leftarrow \alpha^2 - x_T - x_P$$

15:

$$y_{T+P} \leftarrow -y_T - \alpha(x_{T+P} - x_T)$$

16:

$$f_1 \leftarrow f_1(y_Q - y_T - \alpha(x_Q - x_T))$$

17:

$$f_2 \leftarrow f_2(x_Q + (x_P + x_T) - \alpha^2)$$

18: $T \leftarrow T + P$

19: **end if**

20: **end for**

21:

$$f_1 \leftarrow f_1(x_Q - x_T)$$

22: **return**

$$\frac{f_1}{22f_2}$$

Κεφάλαιο 3

Πρωτόκολλα Ομαδικής Συμφωνίας Κλειδιού (Group Key Agreement Protocols)

3.1 Εισαγωγή

Τα πρωτόκολλα ομαδικής συμφωνίας κλειδιού (Group Key Agreement Protocols) επιτρέπουν σε μια ομάδα n συμμετεχόντων να καταλήξουν σε ένα κοινό μυστικό κλειδί μέσω ενός μη ασφαλούς καναλιού. Σε αντίθεση με τα πρωτόκολλα ανταλλαγής κλειδιών δύο μερών, η πολυπλοκότητα και οι απαιτήσεις ασφάλειας αυξάνονται σημαντικά λόγω της δυναμικής φύσης των ομάδων.

3.2 Βασικές Απαιτήσεις

Τα πρωτόκολλα ομαδικής συμφωνίας κλειδιού πρέπει να ικανοποιούν συγκεκριμένες απαιτήσεις ασφάλειας και λειτουργικότητας, ώστε να μπορούν να χρησιμοποιηθούν σε πραγματικά κατανεμημένα περιβάλλοντα. Σε αντίθεση με τα απλά πρωτόκολλα ανταλλαγής κλειδιού μεταξύ δύο χρηστών, στην περίπτωση των ομάδων το πλήθος των συμμετεχόντων μπορεί να μεταβάλλεται δυναμικά, γεγονός που αυξάνει σημαντικά την πολυπλοκότητα του προβλήματος.

Βασική απαίτηση αποτελεί η εμπιστευτικότητα του ομαδικού κλειδιού. Το τελικό κλειδί πρέπει να είναι γνωστό μόνο στα έγκυρα μέλη της ομάδας και να μην μπορεί να υπολογιστεί από εξωτερικούς χρήστες ή από κόμβους που δεν συμμετέχουν νόμιμα στην εκτέλεση του πρωτοκόλλου. Η ιδιότητα αυτή είναι απαραίτητη, καθώς το ομαδικό κλειδί χρησιμοποιείται στη συνέχεια για την προστασία της επικοινωνίας μεταξύ των μελών.

Μία ακόμη σημαντική απαίτηση είναι η εμπρόσθια μυστικότητα (forward secrecy). Η ιδιότητα αυτή εξασφαλίζει ότι όταν ένα μέλος αποχωρεί από την ομάδα, δεν μπορεί να υπολογίσει μελλοντικά ομαδικά κλειδιά. Με τον τρόπο αυτό αποτρέπεται η πρόσβαση αποχωρησάντων χρηστών σε μελλοντικές επικοινωνίες της ομάδας. Η απαίτηση αυτή είναι ιδιαίτερα σημαντική σε δυναμικά συστήματα, όπου οι συμμετέχοντες μπορούν να αλλάζουν συχνά.

Αντίστοιχα, η αντίστροφη μυστικότητα (backward secrecy) εξασφαλίζει ότι ένα νέο μέλος που εισέρχεται στην ομάδα δεν μπορεί να αποκτήσει πρόσβαση σε παλαιότερα ομαδικά κλειδιά. Έτσι, προστατεύεται το ιστορικό της επικοινωνίας της ομάδας και αποτρέπεται η αποκάλυψη προηγούμενων δεδομένων σε νέους συμμετέχοντες.

Επιπλέον, σημαντική ιδιότητα είναι η ανεξαρτησία κλειδιών (key independence). Σύμφωνα με αυτή την απαίτηση, η γνώση ενός παλαιού ή μελλοντικού κλειδιού δεν πρέπει να επιτρέπει τον υπολογισμό άλλων κλειδιών της ομάδας. Κάθε ομαδικό κλειδί πρέπει να θεωρείται ανεξάρτητο από τα προηγούμενα και τα επόμενα, ώστε η παραβίαση μιας εκτέλεσης του πρωτοκόλλου να μην επηρεάζει συνολικά την ασφάλεια του συστήματος.

Τα πρωτόκολλα Group Key Agreement πρέπει επίσης να προσφέρουν αυθεντικοποίηση των συμμετεχόντων. Κάθε κόμβος που συμμετέχει στη διαδικασία συμφωνίας κλειδιού πρέπει να μπορεί να αποδείξει την ταυτότητά του, ώστε να αποτρέπονται επιθέσεις πλαστοπροσωπίας. Η αυθεντικοποίηση μπορεί να επιτευχθεί μέσω ψηφιακών υπογραφών, πιστοποιητικών ή μηχανισμών βασισμένων σε ταυτότητες.

Τέλος, ιδιαίτερη σημασία έχει η αποδοτικότητα του πρωτοκόλλου. Καθώς ο αριθμός των συμμετεχόντων αυξάνεται, το πρωτόκολλο πρέπει να παραμένει πρακτικά εφαρμόσιμο ως προς τον αριθμό των μηνυμάτων, τον χρόνο υπολογισμού και τη συνολική κατανάλωση πόρων. Η επεκτασιμότητα αποτελεί κρίσιμη ιδιότητα για την εφαρμογή τέτοιων πρωτοκόλλων σε πραγματικά κατανεμημένα συστήματα και σε περιβάλλοντα blockchain.

3.3 Κεντριοποιημένες Προσεγγίσεις

Στα κεντριοποιημένα πρωτόκολλα, ένας κεντρικός εξυπηρετητής ή διαχειριστής ομάδας (Group Controller) είναι υπεύθυνος για τη δημιουργία και διανομή του κλειδιού σε όλους τους συμμετέχοντες.

Έστω ένας controller C και συμμετέχοντες $P_1, \dots, P_n$. Ο C παράγει ένα μυστικό κλειδί K και το διανέμει σε κάθε P_i μέσω ασφαλούς καναλιού ή μέσω κρυπτογράφησης με δημόσια κλειδιά.

Παρότι απλά στην υλοποίηση, τα πρωτόκολλα αυτά παρουσιάζουν πρόβλημα single point of failure και περιορισμένη κλιμάκωση.

3.4 Κατανεμημένες Προσεγγίσεις

Στα κατανεμημένα πρωτόκολλα, δεν υπάρχει κεντρική αρχή. Όλοι οι συμμετέχοντες συνεισφέρουν ισότιμα στον υπολογισμό του τελικού κλειδιού.

Ένα γενικό μοντέλο βασίζεται σε επαναληπτικές εκθετικοποιήσεις της μορφής:

$$K = g^{x_1 x_2 \dots x_n},$$

όπου κάθε x_i είναι ιδιωτική τιμή του συμμετέχοντα P_i .

Η ασφάλεια βασίζεται σε παραλλαγές του Computational Diffie–Hellman problem.

3.5 Δενδροειδείς Προσεγγίσεις (Tree-Based Approaches)

Οι tree-based προσεγγίσεις οργανώνουν τους συμμετέχοντες σε ιεραρχική δομή δέντρου, μειώνοντας την πολυπλοκότητα επικοινωνίας.

Σε ένα δυαδικό δέντρο, κάθε εσωτερικός κόμβος αντιπροσωπεύει μερικό υπο-κλειδί, ενώ η ρίζα αντιστοιχεί στο τελικό κλειδί ομάδας.

Η πολυπλοκότητα μειώνεται από $O(n)$ σε $O(\log n)$ ως προς τον αριθμό των απαιτούμενων βημάτων επανακλειδώματος.

3.6 Pairing-Based Group Key Agreement

Τα πρωτόκολλα που βασίζονται σε διγραμμικές ζευγοποιήσεις επιτρέπουν αποδοτικές κατασκευές group key agreement με χρήση της απεικόνισης:

$$e : G_1 \times G_2 \rightarrow G_T.$$

Ένα τυπικό σχήμα επιτρέπει σε κάθε συμμετέχοντα P_i με μυστικό x_i να συμβάλει στο τελικό κλειδί μέσω εκφράσεων της μορφής:

$$K = e(g, g)^{x_1 x_2 \dots x_n}.$$

Τα pairing-based πρωτόκολλα προσφέρουν αποδοτικές λύσεις σε σενάρια πολλαπλών συμμετεχόντων, αλλά αυξάνουν το υπολογιστικό κόστος σε σχέση με κλασικά DH-based συστήματα.

3.7 Σύγκριση Προσεγγίσεων Group Key Agreement

Οι διαφορετικές προσεγγίσεις ομαδικής συμφωνίας κλειδιού παρουσιάζουν διαφορετικά χαρακτηριστικά ως προς την ασφάλεια, την αποδοτικότητα και την καταλληλότητά τους για δυναμικά περιβάλλοντα. Οι κεντριοποιημένες προσεγγίσεις βασίζονται συνήθως σε έναν έμπιστο διαχειριστή, ο οποίος αναλαμβάνει τη δημιουργία και διανομή του ομαδικού κλειδιού. Το βασικό τους πλεονέκτημα είναι η απλότητα σχεδίασης και υλοποίησης, καθώς τα μέλη δεν χρειάζεται να εκτελέσουν πολύπλοκες διαδικασίες συμφωνίας μεταξύ τους.

Ωστόσο, οι κεντριοποιημένες προσεγγίσεις εισάγουν ένα κρίσιμο σημείο αποτυχίας. Αν ο κεντρικός διαχειριστής παραβιαστεί ή καταστεί μη διαθέσιμος, ολόκληρη η ασφάλεια και η λειτουργικότητα του συστήματος επηρεάζεται. Επιπλέον, η ύπαρξη κεντρικού κόμβου έρχεται σε αντίθεση με τη φιλοσοφία των αποκεντρωμένων συστημάτων και των blockchain υποδομών.

Αντίθετα, οι κατακεντρωμένες προσεγγίσεις επιδιώκουν να εξαλείψουν την ανάγκη ύπαρξης ενός πλήρως έμπιστου κεντρικού φορέα. Σε αυτές, το τελικό κλειδί προκύπτει από τη συμβολή όλων των συμμετεχόντων. Το χαρακτηριστικό αυτό ενισχύει την ασφάλεια, καθώς κανένας μεμονωμένος κόμβος δεν μπορεί να καθορίσει μόνος του το τελικό κλειδί. Παράλληλα, η απουσία κεντρικού σημείου ελέγχου καθιστά τις κατακεντρωμένες προσεγγίσεις πιο κατάλληλες για αποκεντρωμένα περιβάλλοντα.

Οι δενδροειδείς προσεγγίσεις αποτελούν έναν ενδιάμεσο μηχανισμό που στοχεύει κυρίως στη βελτίωση της αποδοτικότητας. Οργανώνοντας τους συμμετέχοντες σε δενδροειδή δομή, η διαδικασία επανακλειδώματος μπορεί να περιοριστεί σε συγκεκριμένα τμήματα του δένδρου όταν ένα μέλος εισέρχεται ή αποχωρεί από την ομάδα. Έτσι μειώνεται το υπολογιστικό και επικοινωνιακό κόστος σε σύγκριση με προσεγγίσεις όπου κάθε αλλαγή επηρεάζει όλα τα μέλη.

Οι pairing-based προσεγγίσεις αξιοποιούν τις ιδιότητες των διγραμμικών απεικονίσεων για την κατασκευή πιο σύνθετων και αποδοτικών πρωτοκόλλων. Η δυνατότητα των bilinear pairings να συνδέουν στοιχεία διαφορετικών ομάδων επιτρέπει την υλοποίηση μηχανισμών αυθεντικοποίησης και παραγωγής κοινών μυστικών με μικρότερο αριθμό γύρων επικοινωνίας. Παρόλα αυτά, οι πράξεις pairing είναι υπολογιστικά ακριβότερες από απλές πράξεις σε ελλειπτικές καμπύλες, γεγονός που πρέπει να λαμβάνεται υπόψη κατά την υλοποίηση.

Στο πλαίσιο της παρούσας εργασίας, το BDGKA συνδυάζει χαρακτηριστικά δυναμικής ομαδικής συμφωνίας κλειδιού, pairing-based κρυπτογραφίας και blockchain υποδομής. Η

χρήση blockchain προσθέτει ιδιότητες ιχνηλασιμότητας, ακεραιότητας και μη αποποίησης ευθύνης, ενώ οι διγραμμικές απεικονίσεις χρησιμοποιούνται για την ασφαλή παραγωγή του τελικού ομαδικού κλειδιού.

3.8 Δυναμική Συμμετοχή (Dynamic Membership)

Σε πραγματικά συστήματα, η ομάδα μεταβάλλεται δυναμικά με εισαγωγή και αποχώρηση μελών.

Κατά την είσοδο νέου μέλους απαιτείται *join rekeying*, ενώ κατά την αποχώρηση απαιτείται *leave rekeying* ώστε να διατηρείται η forward και backward secrecy.

Η αποτυχία επανακλειδώματος οδηγεί σε παραβίαση της ασφάλειας του συστήματος.

3.9 Μηχανισμοί Επανακλειδώματος (Rekeying Mechanisms)

Οι μηχανισμοί rekeying διασφαλίζουν ότι το μυστικό κλειδί ανανεώνεται μετά από κάθε αλλαγή στη σύνθεση της ομάδας.

Οι κύριες κατηγορίες είναι:

- πλήρης επαναυπολογισμός του κλειδιού,
- μερική ενημέρωση μέσω broadcast μηνυμάτων,
- ιεραρχική ανανέωση σε tree-based δομές.

Η επιλογή μηχανισμού επηρεάζει άμεσα την απόδοση και την ασφάλεια του πρωτοκόλλου.

3.10 Κλασικά Πρωτόκολλα Ομαδικής Συμφωνίας Κλειδιού

Η έρευνα στον τομέα της συμφωνίας ομαδικού κλειδιού ξεκίνησε ως επέκταση του πρωτοκόλλου Diffie--Hellman για περισσότερους από δύο συμμετέχοντες. Στόχος των πρωτοκόλλων αυτών είναι η δημιουργία ενός κοινού μυστικού κλειδιού μεταξύ πολλών χρηστών χωρίς την ύπαρξη κεντρικού διαχειριστή.

Ένα από τα πρώτα και σημαντικότερα πρωτόκολλα είναι το Burmester--Desmedt Group Key Agreement Protocol. Το πρωτόκολλο αυτό επιτρέπει σε όλους τους συμμετέχοντες να συμβάλουν ισότιμα στη δημιουργία του τελικού κλειδιού, απαιτώντας μικρό αριθμό γύρων επικοινωνίας και παρουσιάζοντας καλή επεκτασιμότητα.

Μία ακόμη σημαντική κατηγορία αποτελεί το Tree Group Diffie--Hellman (TGDH), το οποίο οργανώνει τους συμμετέχοντες σε δενδροειδή δομή. Η προσέγγιση αυτή επιτρέπει αποδοτικότερη διαχείριση δυναμικών αλλαγών στην ομάδα, καθώς η αποχώρηση ή η είσοδος ενός μέλους επηρεάζει μόνο ένα τμήμα του δένδρου.

Η εμφάνιση των bilinear pairings οδήγησε στην ανάπτυξη νέων pairing-based πρωτοκόλλων, όπως το πρωτόκολλο Joux και οι επεκτάσεις του για πολλαπλούς συμμετέχοντες. Τα πρωτόκολλα αυτά αξιοποιούν τις ιδιότητες των διγραμμικών απεικονίσεων ώστε να μειώσουν τον αριθμό των απαιτούμενων ανταλλαγών μηνυμάτων και να προσφέρουν πρόσθετες δυνατότητες αυθεντικοποίησης.

Τα σύγχρονα πρωτόκολλα Group Key Agreement βασίζονται συχνά σε ελλειπτικές καμπύλες και διγραμμικές απεικονίσεις, επιδιώκοντας υψηλότερο επίπεδο ασφάλειας και καλύτερη απόδοση σε περιβάλλοντα μεγάλης κλίμακας.

3.11 Επιθέσεις σε Πρωτόκολλα Group Key Agreement

Η ασφάλεια ενός πρωτοκόλλου συμφωνίας ομαδικού κλειδιού εξαρτάται από την ικανότητά του να προστατεύεται απέναντι σε διάφορες κατηγορίες επιθέσεων. Για τον λόγο αυτό, κατά τον σχεδιασμό ενός πρωτοκόλλου λαμβάνονται υπόψη διαφορετικά μοντέλα επιτιθέμενων και σενάρια παραβίασης.

Μία από τις συνηθέστερες επιθέσεις είναι η επίθεση επανάληψης (Replay Attack), κατά την οποία ένας επιτιθέμενος καταγράφει έγκυρα μηνύματα μιας προηγούμενης εκτέλεσης του πρωτοκόλλου και τα επαναμεταδίδει με στόχο την παραπλάνηση των συμμετεχόντων.

Μία ακόμη σημαντική κατηγορία είναι η επίθεση Man-in-the-Middle, όπου ο επιτιθέμενος παρεμβάλλεται μεταξύ των συμμετεχόντων και επιχειρεί να τροποποιήσει ή να αντικαταστήσει τα ανταλλασσόμενα μηνύματα χωρίς να γίνει αντιληπτός.

Επιπλέον, ιδιαίτερη σημασία έχει η προστασία απέναντι σε επιθέσεις πλαστοπροσωπίας (Impersonation Attacks), στις οποίες ένας κακόβουλος χρήστης προσπαθεί να παρουσιαστεί ως νόμιμο μέλος της ομάδας.

Στα δυναμικά πρωτόκολλα συμφωνίας κλειδιού εξετάζονται επίσης επιθέσεις συνωμοσίας (Collusion Attacks), όπου πρώην και τρέχοντα μέλη συνεργάζονται προκειμένου να αποκτήσουν πρόσβαση σε ομαδικά κλειδιά που δεν δικαιούνται να γνωρίζουν.

Η αντιμετώπιση των παραπάνω επιθέσεων επιτυγχάνεται μέσω μηχανισμών αυθεντικοποίησης, ψηφιακών υπογραφών, χρήσης χρονοσφραγίδων και κατάλληλων διαδικασιών επανακλειδώματος της ομάδας.

3.12 Συμπεράσματα

Τα πρωτόκολλα ομαδικής συμφωνίας κλειδιού αποτελούν κρίσιμο στοιχείο σε κατανεμημένα συστήματα ασφαλείας. Οι κεντριοποιημένες προσεγγίσεις είναι απλές αλλά μη κλιμακώσιμες, ενώ οι κατανεμημένες και pairing-based λύσεις προσφέρουν μεγαλύτερη ευελιξία και ασφάλεια με αυξημένο υπολογιστικό κόστος.

Κεφάλαιο 4

Τεχνολογία Blockchain

4.1 Εξέλιξη των Blockchain

Η τεχνολογία blockchain αποτελεί μια κατακεντρωμένη δομή δεδομένων η οποία επιτρέπει την αμετάβλητη καταγραφή συναλλαγών χωρίς την ανάγκη κεντρικής αρχής εμπιστοσύνης. Η πρώτη πρακτική υλοποίηση εμφανίστηκε με το Bitcoin, εισάγοντας έναν μηχανισμό Proof of Work για την επίτευξη συναίνεσης σε ένα πλήρως αποκεντρωμένο δίκτυο.

Στη συνέχεια, η εξέλιξη των blockchain συστημάτων οδήγησε σε πιο γενικευμένες πλατφόρμες, όπως το Ethereum, οι οποίες υποστηρίζουν προγραμματιζόμενη λογική μέσω smart contracts, καθώς και σε permissioned συστήματα με αυξημένες απαιτήσεις απόδοσης και ελέγχου πρόσβασης.

4.2 Δημόσια, Ιδιωτικά και Permissioned Blockchain

Τα blockchain δίκτυα μπορούν να κατηγοριοποιηθούν με βάση τον τρόπο πρόσβασης, τη διαδικασία συμμετοχής των κόμβων και το μοντέλο εμπιστοσύνης που υιοθετούν. Η διάκριση αυτή είναι σημαντική, καθώς διαφορετικές κατηγορίες blockchain είναι κατάλληλες για διαφορετικά είδη εφαρμογών.

Στα δημόσια blockchain, οποιοσδήποτε χρήστης μπορεί να συμμετάσχει στο δίκτυο, να δημιουργήσει συναλλαγές και, ανάλογα με τον μηχανισμό συναίνεσης, να λάβει μέρος στη διαδικασία επικύρωσης. Παραδείγματα δημόσιων blockchain αποτελούν το Bitcoin και το Ethereum. Το βασικό τους πλεονέκτημα είναι ο υψηλός βαθμός αποκέντρωσης και η ανοικτή συμμετοχή. Ωστόσο, η πλήρης απουσία ελέγχου συμμετεχόντων οδηγεί συνήθως σε υψηλότερο κόστος συναίνεσης, αυξημένες καθυστερήσεις και περιορισμένη απόδοση.

Αντίθετα, στα ιδιωτικά blockchain η πρόσβαση στο δίκτυο ελέγχεται από έναν οργανισμό ή μία περιορισμένη ομάδα διαχειριστών. Οι συμμετέχοντες είναι γνωστοί εκ των προτέρων και η επικύρωση των συναλλαγών πραγματοποιείται από εξουσιοδοτημένους κόμβους. Η προσέγγιση αυτή προσφέρει καλύτερη απόδοση και μεγαλύτερο έλεγχο, αλλά περιορίζει τον βαθμό αποκέντρωσης.

Τα permissioned blockchain αποτελούν μία ειδική κατηγορία blockchain δικτύων, όπου οι συμμετέχοντες πρέπει να διαθέτουν κατάλληλα δικαιώματα πρόσβασης. Σε τέτοια δίκτυα, η ταυτότητα των κόμβων είναι γνωστή και ελέγχεται μέσω μηχανισμών πιστοποίησης. Το Hyperledger Fabric ανήκει σε αυτή την κατηγορία και είναι κατάλληλο για επιχειρησιακές εφαρμογές, όπου απαιτείται έλεγχος πρόσβασης, διαχείριση ταυτοτήτων και υψηλή απόδοση.

Η επιλογή permissioned blockchain είναι ιδιαίτερα σημαντική για την παρούσα εργασία, καθώς το πρωτόκολλο BDGKA εκτελείται σε περιβάλλον όπου οι συμμετέχοντες κόμβοι

πρέπει να είναι γνωστοί και αυθεντικοποιημένοι. Με τον τρόπο αυτό, η υποδομή blockchain δεν λειτουργεί μόνο ως μηχανισμός αποθήκευσης δεδομένων, αλλά και ως μέσο ενίσχυσης της εμπιστοσύνης μεταξύ των συμμετεχόντων.

4.3 Μηχανισμοί Συναίνεσης

Οι μηχανισμοί συναίνεσης αποτελούν βασικό στοιχείο κάθε blockchain συστήματος, καθώς καθορίζουν τον τρόπο με τον οποίο οι κόμβοι του δικτύου συμφωνούν για την εγκυρότητα και τη σειρά των συναλλαγών. Χωρίς έναν τέτοιο μηχανισμό, δεν θα ήταν δυνατό να διατηρηθεί μία κοινή και συνεπής κατάσταση του ledger μεταξύ των διαφορετικών κόμβων.

Στα δημόσια blockchain χρησιμοποιούνται συχνά μηχανισμοί όπως το Proof of Work και το Proof of Stake. Το Proof of Work βασίζεται στην επίλυση υπολογιστικά δύσκολων προβλημάτων, απαιτώντας σημαντική κατανάλωση υπολογιστικών πόρων. Αντίθετα, το Proof of Stake επιλέγει επικυρωτές με βάση το ποσοστό συμμετοχής τους στο δίκτυο, μειώνοντας την ενεργειακή κατανάλωση σε σχέση με το Proof of Work.

Σε permissioned blockchain δίκτυα, όπως το Hyperledger Fabric, η συναίνεση μπορεί να επιτευχθεί με πιο αποδοτικούς μηχανισμούς, επειδή οι συμμετέχοντες είναι γνωστοί και πιστοποιημένοι. Έτσι δεν απαιτούνται ενεργοβόροι μηχανισμοί όπως το Proof of Work. Αντίθετα, μπορούν να χρησιμοποιηθούν μηχανισμοί όπως το Raft ή άλλες μορφές ordering service, οι οποίες προσφέρουν καλύτερη απόδοση και μικρότερη καθυστέρηση.

Η επιλογή του κατάλληλου μηχανισμού συναίνεσης επηρεάζει άμεσα την απόδοση του συστήματος. Σε εφαρμογές όπου απαιτείται γρήγορη καταχώριση δεδομένων, όπως στην περίπτωση ενός πρωτοκόλλου συμφωνίας κλειδιού, η καθυστέρηση συναίνεσης μπορεί να επηρεάσει τον συνολικό χρόνο εκτέλεσης. Για τον λόγο αυτό, η χρήση Hyperledger Fabric είναι κατάλληλη, καθώς διαχωρίζει τη διαδικασία εκτέλεσης συναλλαγών, επικύρωσης και ταξινόμησης, προσφέροντας μεγαλύτερη ευελιξία.

4.4 Smart Contracts

Τα έξυπνα συμβόλαια αποτελούν προγράμματα τα οποία εκτελούνται πάνω σε ένα blockchain δίκτυο και καθορίζουν τη λογική διαχείρισης των δεδομένων. Μέσω αυτών, οι χρήστες μπορούν να αλληλεπιδρούν με το ledger με ελεγχόμενο και προκαθορισμένο τρόπο.

Σε ένα blockchain σύστημα, το smart contract καθορίζει ποιες λειτουργίες επιτρέπονται, ποια δεδομένα μπορούν να καταχωριστούν και υπό ποιες προϋποθέσεις μπορεί να τροποποιηθεί η κατάσταση του ledger. Με τον τρόπο αυτό, η επιχειρησιακή λογική της εφαρμογής μεταφέρεται στο ίδιο το blockchain δίκτυο.

Στο Hyperledger Fabric, τα smart contracts υλοποιούνται με τη μορφή chaincode. Το chaincode εγκαθίσταται στους peers του δικτύου και εκτελείται όταν ένας client υποβάλλει μία συναλλαγή. Η εκτέλεση του chaincode παράγει αποτελέσματα τα οποία στη συνέχεια επικυρώνονται και καταχωρίζονται στο ledger.

Στην παρούσα εργασία, το chaincode χρησιμοποιείται για την καταγραφή δεδομένων που σχετίζονται με την εκτέλεση του πρωτοκόλλου BDGKA. Τέτοια δεδομένα περιλαμβάνουν δημόσιες παραμέτρους, δημόσιες συνεισφορές κόμβων, υπογραφές και πληροφορίες που είναι απαραίτητες για την επαλήθευση της διαδικασίας. Με τον τρόπο αυτό, το blockchain λειτουργεί ως αξιόπιστο και αμετάβλητο μέσο καταγραφής των κρίσιμων βημάτων του πρωτοκόλλου.

4.5 Identity Management

Η διαχείριση ταυτοτήτων αποτελεί κρίσιμο ζήτημα σε κάθε blockchain σύστημα, ιδιαίτερα όταν το δίκτυο δεν είναι δημόσιο αλλά permissioned. Σε τέτοια περιβάλλοντα, κάθε συμμετέχων πρέπει να διαθέτει μία επαληθεύσιμη ταυτότητα, ώστε να είναι δυνατό να ελεγχθεί ποιος εκτελεί κάθε ενέργεια.

Στα δημόσια blockchain, η ταυτότητα ενός χρήστη συνδέεται συνήθως με ένα ζεύγος δημόσιου και ιδιωτικού κλειδιού. Η διεύθυνση του χρήστη προκύπτει από το δημόσιο κλειδί, ενώ το ιδιωτικό κλειδί χρησιμοποιείται για την υπογραφή συναλλαγών. Το μοντέλο αυτό προσφέρει ψευδωνυμία, αλλά δεν παρέχει απαραίτητα άμεση σύνδεση με πραγματικές οργανωτικές ταυτότητες.

Αντίθετα, σε permissioned blockchain όπως το Hyperledger Fabric, οι ταυτότητες βασίζονται σε ψηφιακά πιστοποιητικά. Κάθε οργανισμός διαθέτει δική του Certificate Authority, η οποία εκδίδει πιστοποιητικά για τους χρήστες και τους κόμβους του δικτύου. Τα πιστοποιητικά αυτά χρησιμοποιούνται για την αυθεντικοποίηση των συμμετεχόντων και τον έλεγχο των δικαιωμάτων πρόσβασης.

Η ύπαρξη μηχανισμού διαχείρισης ταυτοτήτων είναι ιδιαίτερα σημαντική για την υλοποίηση ενός πρωτοκόλλου συμφωνίας κλειδιού. Οι κόμβοι που συμμετέχουν στη διαδικασία πρέπει να είναι γνωστοί και να μπορούν να αποδείξουν την ταυτότητά τους. Έτσι αποτρέπονται επιθέσεις πλαστοπροσωπίας και διασφαλίζεται ότι μόνο εξουσιοδοτημένα μέλη μπορούν να συμμετέχουν στην παραγωγή του ομαδικού κλειδιού.

4.5.1 Proof of Work (PoW)

Το Proof of Work απαιτεί από τους miners να επιλύσουν ένα υπολογιστικά δύσκολο πρόβλημα της μορφής:

$$H(B \parallel \text{nonce}) < T,$$

όπου H είναι μια κρυπτογραφική συνάρτηση κατακερματισμού και T το όριο δυσκολίας.

Η ασφάλεια βασίζεται στο υψηλό υπολογιστικό κόστος αναδημιουργίας της αλυσίδας.

4.5.2 Proof of Stake (PoS)

Στο Proof of Stake, η επιλογή του δημιουργού ενός block εξαρτάται από το ποσοστό συμμετοχής (stake) του κόμβου στο δίκτυο. Η πιθανότητα επιλογής είναι ανάλογη του stake, μειώνοντας σημαντικά την ενεργειακή κατανάλωση σε σχέση με το PoW.

4.5.3 Practical Byzantine Fault Tolerance (PBFT)

Το PBFT είναι ένα σύστημα συναίνεσης σχεδιασμένο για permissioned δίκτυα, το οποίο αντέχει την παρουσία κακόβουλων κόμβων μέχρι ένα όριο f , όπου το σύνολο των κόμβων ικανοποιεί:

$$n \geq 3f + 1.$$

Το πρωτόκολλο λειτουργεί μέσω φάσεων επικοινωνίας μεταξύ primary και backup nodes, εξασφαλίζοντας συμφωνία ακόμη και υπό Byzantine αποτυχίες.

4.5.4 Raft

Το Raft είναι ένας αλγόριθμος συναίνεσης σχεδιασμένος για απλότητα και κατανοησιμότητα. Βασίζεται στην εκλογή leader και στην αναπαραγωγή log entries στους follower nodes.

Σε αντίθεση με το PBFT, το Raft δεν είναι Byzantine fault tolerant, αλλά παρέχει ισχυρή συνέπεια σε περιβάλλοντα όπου οι κόμβοι θεωρούνται αξιόπιστοι.

4.6 Identity Management

Η διαχείριση ταυτότητας σε blockchain συστήματα στοχεύει στην αποκέντρωση της ψηφιακής ταυτότητας, μειώνοντας την εξάρτηση από κεντρικούς παρόχους.

Τα Decentralized Identity (DID) συστήματα επιτρέπουν στους χρήστες να ελέγχουν τα ίδια τα κρυπτογραφικά τους αναγνωριστικά μέσω δημόσιων κλειδιών και αποδείξεων ταυτότητας.

Η βασική αρχή είναι ότι η ταυτότητα ενός χρήστη συνδέεται με ένα ζεύγος κλειδιών (sk, pk) , όπου:

$$pk = sk \cdot G,$$

και οι συναλλαγές ή πιστοποιήσεις επαληθεύονται μέσω ψηφιακών υπογραφών.

Η ενσωμάτωση blockchain σε identity management ενισχύει την ακεραιότητα, την ιχνηλασιμότητα και την ανθεκτικότητα σε παραποίηση δεδομένων.

4.7 Δομή ενός Block

Το blockchain αποτελείται από μία αλληλουχία blocks τα οποία συνδέονται μεταξύ τους μέσω κρυπτογραφικών συναρτήσεων κατακερματισμού. Κάθε block περιέχει ένα σύνολο συναλλαγών ή δεδομένων καθώς και πληροφορίες που επιτρέπουν την επαλήθευση της ακεραιότητάς του.

Ένα τυπικό block αποτελείται από δύο κύρια μέρη: την επικεφαλίδα (block header) και το σώμα (block body). Η επικεφαλίδα περιλαμβάνει πληροφορίες όπως το hash του προηγούμενου block, τη χρονοσφραγίδα δημιουργίας, τον μηχανισμό συναίνεσης και τη ρίζα Merkle. Το σώμα περιέχει το σύνολο των συναλλαγών ή εγγραφών που αποθηκεύονται στο block.

Η ύπαρξη του hash του προηγούμενου block δημιουργεί μία αλυσίδα εξαρτήσεων μεταξύ όλων των blocks. Ως αποτέλεσμα, οποιαδήποτε τροποποίηση σε παλαιότερο block μεταβάλλει το αντίστοιχο hash και καθιστά άμεσα ανιχνεύσιμη την παραποίηση των δεδομένων.

4.8 Merkle Trees

Τα Merkle Trees αποτελούν ειδικές δενδροειδείς δομές δεδομένων που χρησιμοποιούνται ευρέως στα blockchain συστήματα για την αποδοτική επαλήθευση της ακεραιότητας μεγάλου όγκου δεδομένων.

Κάθε φύλλο του δένδρου αντιστοιχεί στο αποτέλεσμα μιας συνάρτησης κατακερματισμού που εφαρμόζεται σε μία συναλλαγή. Στη συνέχεια, τα γειτονικά hashes συνδυάζονται και κατακερματίζονται εκ νέου, μέχρι να παραχθεί μία μοναδική τιμή η οποία ονομάζεται Merkle Root.

Η τιμή αυτή αποθηκεύεται στην επικεφαλίδα του block και λειτουργεί ως κρυπτογραφικό αποτύπωμα όλων των συναλλαγών που περιέχει. Με τον τρόπο αυτό καθίσταται δυνατή η

αποδοτική επαλήθευση συγκεκριμένων συναλλαγών χωρίς να απαιτείται η ανάγνωση ολόκληρου του block.

Η χρήση των Merkle Trees μειώνει σημαντικά το κόστος αποθήκευσης και επικοινωνίας, ενώ παράλληλα ενισχύει την ακεραιότητα των δεδομένων που διατηρούνται στο blockchain.

4.9 Ασφάλεια Blockchain

Η ασφάλεια αποτελεί μία από τις σημαντικότερες ιδιότητες των blockchain συστημάτων. Η προστασία των δεδομένων επιτυγχάνεται μέσω του συνδυασμού κρυπτογραφικών τεχνικών, μηχανισμών συναίνεσης και κατανεμημένης αποθήκευσης.

Μία γνωστή κατηγορία απειλών είναι οι επιθέσεις 51%, κατά τις οποίες ένας επιτιθέμενος αποκτά τον έλεγχο της πλειονότητας των υπολογιστικών ή επικυρωτικών πόρων του δικτύου. Σε τέτοια περίπτωση μπορεί να επιχειρήσει την αναδιοργάνωση της αλυσίδας και τη διπλή δαπάνη συναλλαγών.

Επιπλέον, συναντώνται επιθέσεις Sybil, κατά τις οποίες ένας χρήστης δημιουργεί μεγάλο αριθμό ψευδών ταυτοτήτων με σκοπό να επηρεάσει τη λειτουργία του δικτύου. Παράλληλα, ιδιαίτερη προσοχή απαιτείται στην ασφάλεια των έξυπνων συμβολαίων, καθώς σφάλματα στον πηγαίο κώδικα μπορούν να οδηγήσουν σε σοβαρές παραβιάσεις ασφαλείας.

Στα permissioned blockchain δίκτυα, όπως το Hyperledger Fabric, οι κίνδυνοι αυτοί περιορίζονται μέσω μηχανισμών ελέγχου ταυτότητας, ψηφιακών πιστοποιητικών και αυστηρών πολιτικών πρόσβασης.

4.10 Blockchain και Διαχείριση Κλειδιών

Η διαχείριση κρυπτογραφικών κλειδιών αποτελεί κρίσιμο ζήτημα σε κατανεμημένα συστήματα. Η χρήση blockchain προσφέρει έναν ασφαλή και αμετάβλητο μηχανισμό αποθήκευσης πληροφοριών που σχετίζονται με τη διαδικασία δημιουργίας και διαχείρισης κλειδιών.

Στο πλαίσιο των πρωτοκόλλων Group Key Agreement, το blockchain μπορεί να χρησιμοποιηθεί για την καταγραφή δημοσίων παραμέτρων, ψηφιακών υπογραφών και στοιχείων επαλήθευσης. Με τον τρόπο αυτό παρέχεται πλήρης ιχνηλασιμότητα όλων των ενεργειών που πραγματοποιούνται κατά την εκτέλεση του πρωτοκόλλου.

Επιπλέον, η αμεταβλητότητα του blockchain επιτρέπει την αποτροπή μη εξουσιοδοτημένων τροποποιήσεων, ενώ η αποκεντρωμένη φύση του εξαλείφει την ανάγκη ύπαρξης ενός κεντρικού σημείου εμπιστοσύνης.

Για τους λόγους αυτούς, η ενσωμάτωση τεχνολογιών blockchain σε πρωτόκολλα συμφωνίας κλειδιού αποτελεί μία ιδιαίτερα ελκυστική προσέγγιση για σύγχρονα κατανεμημένα συστήματα.

Κεφάλαιο 5

Hyperledger Fabric

5.1 Εισαγωγή

Το Hyperledger Fabric αποτελεί μια permissioned blockchain πλατφόρμα γενικού σκοπού, σχεδιασμένη για επιχειρησιακές εφαρμογές με υψηλές απαιτήσεις σε απόδοση, εμπιστευτικότητα και διαχείριση ταυτότητας. Σε αντίθεση με τα δημόσια blockchain, το Fabric υποστηρίζει ελεγχόμενη συμμετοχή κόμβων και ευέλικτες πολιτικές συναίνεσης και πρόσβασης.

5.2 Αρχιτεκτονική Hyperledger Fabric

Η αρχιτεκτονική του Hyperledger Fabric βασίζεται σε μια modular σχεδίαση, όπου τα επιμέρους συστατικά του δικτύου διαχωρίζονται σε διακριτούς ρόλους. Το δίκτυο αποτελείται από οργανισμούς, κόμβους (peers), υπηρεσίες ordering και μηχανισμούς διαχείρισης ταυτότητας.

Η βασική αρχή σχεδίασης είναι ο διαχωρισμός των φάσεων εκτέλεσης, ordering και επικύρωσης συναλλαγών, επιτρέποντας υψηλή κλιμάκωση και παραλληλισμό.

5.3 Συστατικά του Fabric

5.3.1 Peers

Οι peer nodes αποτελούν τους βασικούς συμμετέχοντες του δικτύου. Κάθε peer διατηρεί ένα αντίγραφο του ledger και εκτελεί smart contracts (chaincode).

Οι peers διακρίνονται σε endorsing peers και committing peers, ανάλογα με τον ρόλο τους στην επεξεργασία συναλλαγών.

5.3.2 Orderers

Οι orderer nodes είναι υπεύθυνοι για τη συνολική διάταξη των συναλλαγών στο δίκτυο. Συλλέγουν συναλλαγές από τους clients και τις οργανώνουν σε blocks, τα οποία στη συνέχεια διανέμονται στους peers.

Η λειτουργία τους είναι ανεξάρτητη από την εκτέλεση smart contracts, εξασφαλίζοντας διαχωρισμό καθηκόντων.

5.3.3 MSP (Membership Service Provider)

Το MSP αποτελεί τον μηχανισμό διαχείρισης ταυτότητας στο Fabric. Ορίζει τις πολιτικές ταυτοποίησης και εξουσιοδότησης των συμμετεχόντων στο δίκτυο.

Κάθε οργανισμός στο δίκτυο διαθέτει το δικό του MSP, το οποίο καθορίζει ποιοι χρήστες και κόμβοι θεωρούνται έγκυροι.

5.3.4 Certificate Authorities

Οι Certificate Authorities (CA) είναι υπεύθυνες για την έκδοση, ανανέωση και ανάκληση ψηφιακών πιστοποιητικών. Τα πιστοποιητικά αυτά βασίζονται συνήθως σε υποδομή δημόσιου κλειδιού (PKI) με χρήση ελλειπτικών καμπυλών.

Οι CA αποτελούν θεμέλιο για την αυθεντικοποίηση και την εμπιστοσύνη στο δίκτυο.

5.4 Channels

Τα channels στο Hyperledger Fabric επιτρέπουν τη δημιουργία απομονωμένων υπο-δικτύων εντός του ίδιου blockchain συστήματος. Κάθε channel διαθέτει το δικό του ledger και πολιτικές πρόσβασης.

Αυτό επιτρέπει την ιδιωτικότητα των δεδομένων μεταξύ υποομάδων συμμετεχόντων χωρίς να απαιτείται ξεχωριστό blockchain.

5.5 Δομή του Ledger

Το ledger στο Fabric αποτελείται από δύο βασικά μέρη:

- το blockchain, το οποίο περιέχει την αμετάβλητη αλυσίδα blocks,
- το world state, το οποίο αποτελεί τη βάση δεδομένων τρέχουσας κατάστασης.

Το world state υλοποιείται συνήθως μέσω key-value αποθήκης, όπως LevelDB ή CouchDB.

5.6 Endorsement Policies

Οι endorsement policies καθορίζουν ποιοι peers πρέπει να υπογράψουν μια συναλλαγή ώστε να θεωρηθεί έγκυρη.

Τυπικά, μια πολιτική μπορεί να οριστεί ως:

$$E = (P_1 \wedge P_2) \vee P_3,$$

όπου P_i αντιστοιχούν σε οργανισμούς ή peers.

Οι πολιτικές αυτές ενισχύουν την εμπιστοσύνη χωρίς την ανάγκη πλήρους ομοφωνίας.

5.7 Chaincode

Το chaincode αποτελεί την υλοποίηση των smart contracts στο Hyperledger Fabric. Εκτελείται από τους peers και καθορίζει τη λογική των συναλλαγών.

Τυπικά, ένα chaincode ορίζεται ως συνάρτηση μετάβασης κατάστασης:

$$f : S \times T \rightarrow S,$$

όπου S είναι το state και T οι συναλλαγές.

Η εκτέλεση του chaincode είναι deterministic, ώστε όλοι οι peers να καταλήγουν στο ίδιο αποτέλεσμα.

5.8 Private Data Collections

Οι private data collections επιτρέπουν την αποθήκευση ευαίσθητων δεδομένων που είναι ορατά μόνο σε υποσύνολο των οργανισμών του δικτύου.

Τα δεδομένα αποθηκεύονται τοπικά στους authorized peers, ενώ στο blockchain καταγράφεται μόνο hash για επαλήθευση ακεραιότητας.

5.9 Μοντέλο Ασφάλειας του Fabric

Το μοντέλο ασφάλειας του Hyperledger Fabric βασίζεται σε τρία επίπεδα:

- **Ταυτοποίηση:** μέσω MSP και PKI υποδομών,
- **Εξουσιοδότηση:** μέσω endorsement policies και ACLs,
- **Ακεραιότητα:** μέσω ordering service και κρυπτογραφικών hashes.

Η αρχιτεκτονική του Fabric επιτρέπει την αποφυγή των περιορισμών των δημόσιων blockchain, προσφέροντας υψηλή απόδοση και λεπτομερή έλεγχο πρόσβασης σε επιχειρησιακά περιβάλλοντα.

Κεφάλαιο 6

Άρθρο Υλοποίησης

6.1 Μοντέλο Συστήματος

Το προτεινόμενο πρωτόκολλο σχεδιάστηκε για την ασφαλή δημιουργία και διαχείριση ομαδικού κλειδιού σε ένα αποκεντρωμένο περιβάλλον επικοινωνίας. Το σύστημα αποτελείται από ένα σύνολο χρηστών που επιθυμούν να δημιουργήσουν ένα κοινό μυστικό κλειδί χωρίς να απαιτείται η συνεχής παρουσία μιας κεντρικής αρχής κατά τη διάρκεια της εκτέλεσης του πρωτοκόλλου.

Η αρχιτεκτονική του συστήματος περιλαμβάνει έναν κόμβο αρχικοποίησης, ο οποίος αναφέρεται ως *Key Generation Center (KGC)*, καθώς και ένα σύνολο χρηστών:

$$U = \{u_1, u_2, \dots, u_n\}$$

όπου n είναι ο συνολικός αριθμός των συμμετεχόντων στην ομάδα.

Ο ρόλος του *KGC* περιορίζεται αποκλειστικά στη φάση εγκατάστασης του συστήματος. Συγκεκριμένα, ο *KGC* είναι υπεύθυνος για:

- τη δημιουργία των δημόσιων παραμέτρων του συστήματος,
- τον ορισμό των κρυπτογραφικών ομάδων που χρησιμοποιούνται,
- την επιλογή των συναρτήσεων κατακερματισμού,
- την αρχική εγγραφή των χρηστών,
- τη διανομή των απαραίτητων στοιχείων εκκίνησης.

Μετά την ολοκλήρωση της διαδικασίας αυτής, ο *KGC* δεν συμμετέχει στη δημιουργία του ομαδικού κλειδιού ούτε παρεμβαίνει στις επικοινωνίες μεταξύ των χρηστών. Ως αποτέλεσμα, το προτεινόμενο σχήμα δεν εξαρτάται από μία μόνιμα διαθέσιμη κεντρική αρχή, μειώνοντας το πρόβλημα του single point of failure.

Οι χρήστες θεωρούνται ισότιμοι συμμετέχοντες στο πρωτόκολλο και συνεισφέρουν όλοι στη διαδικασία παραγωγής του τελικού ομαδικού κλειδιού. Η συμμετοχή κάθε χρήστη είναι απαραίτητη για την παραγωγή του τελικού μυστικού, γεγονός που προσδίδει στο πρωτόκολλο χαρακτηριστικά contributory group key agreement.

Το υποκείμενο δίκτυο επικοινωνίας θεωρείται μη αξιόπιστο. Ένας επιτιθέμενος μπορεί να παρακολουθεί, να καταγράφει, να τροποποιεί ή να επαναμεταδίδει μηνύματα τα οποία ανταλλάσσονται μεταξύ των κόμβων. Ωστόσο, θεωρείται ότι δεν μπορεί να παραβιάσει τις

βασικές υπολογιστικές υποθέσεις πάνω στις οποίες στηρίζεται η ασφάλεια του συστήματος, όπως η δυσκολία επίλυσης του προβλήματος Bilinear Diffie--Hellman.

Για την ενίσχυση της διαφάνειας και της επαλήθευσimότητας, το πρωτόκολλο ενσωματώνει ένα permissioned blockchain βασισμένο στο Hyperledger Fabric. Το blockchain λειτουργεί ως αμετάβλητος μηχανισμός καταγραφής των κρίσιμων γεγονότων της εκτέλεσης του πρωτοκόλλου.

Συγκεκριμένα, στο καθολικό καταγράφονται:

- τα δημόσια κλειδιά των συμμετεχόντων,
- οι δημόσιες τυχαίες συνεισφορές των χρηστών,
- οι ψηφιακές υπογραφές που παράγονται κατά τη διάρκεια του πρωτοκόλλου,
- οι χρονικές σφραγίδες των ενεργειών,
- μεταδεδομένα απαραίτητα για την επαλήθευση της εκτέλεσης.

Η χρήση του Hyperledger Fabric επιτρέπει τον έλεγχο πρόσβασης στους συμμετέχοντες μέσω πιστοποιητικών και πολιτικών εξουσιοδότησης, ενώ παράλληλα εξασφαλίζει ότι τα καταγεγραμμένα δεδομένα δεν μπορούν να αλλοιωθούν χωρίς να γίνει αντιληπτό από το σύνολο του δικτύου.

Από κρυπτογραφικής πλευράς, το πρωτόκολλο βασίζεται σε διγραμμικές ζευγοποιήσεις (bilinear pairings) ορισμένες πάνω σε κυκλικές ομάδες πρώτης τάξης. Οι ζευγοποιήσεις αυτές επιτρέπουν την κατασκευή μηχανισμών επαλήθευσης και συμφωνίας κλειδιού που δεν είναι εφικτοί με τα κλασικά πρωτόκολλα Diffie--Hellman.

Η βασική υπόθεση ασφάλειας του συστήματος είναι ότι ένας αντίπαλος δεν μπορεί να επιλύσει αποδοτικά το πρόβλημα Bilinear Diffie--Hellman (BDH), ακόμη και αν διαθέτει πλήρη γνώση όλων των δημόσιων παραμέτρων και των μηνυμάτων που ανταλλάσσονται κατά τη διάρκεια του πρωτοκόλλου.

Υπό τις παραπάνω παραδοχές, το προτεινόμενο μοντέλο συστήματος παρέχει ένα κατάλληλο περιβάλλον για την ασφαλή δημιουργία κοινού ομαδικού κλειδιού, συνδυάζοντας τις ιδιότητες των bilinear pairings με τους μηχανισμούς ακεραιότητας και ιχνηλασιμότητας που προσφέρει το Hyperledger Fabric.

6.2 Κρυπτογραφική Αρχικοποίηση

Η φάση αρχικοποίησης αποτελεί το θεμέλιο του προτεινόμενου πρωτοκόλλου, καθώς καθορίζει το μαθηματικό περιβάλλον μέσα στο οποίο θα πραγματοποιηθούν οι κρυπτογραφικές πράξεις. Η διαδικασία εκτελείται από τον κόμβο KGC πριν από την ένταξη των χρηστών στο σύστημα και ολοκληρώνεται μία μόνο φορά κατά τη δημιουργία του δικτύου.

Στόχος της φάσης αυτής είναι η δημιουργία ενός συνόλου δημόσιων παραμέτρων οι οποίες θα χρησιμοποιηθούν από όλους τους συμμετέχοντες κατά την εκτέλεση του πρωτοκόλλου συμφωνίας ομαδικού κλειδιού.

Αρχικά, ο KGC επιλέγει δύο κυκλικές ομάδες πρώτης τάξης G_1 και G_2 , οι οποίες έχουν κοινή τάξη ίση με έναν μεγάλο πρώτο αριθμό q .

$$|G_1| = |G_2| = q$$

Η ομάδα G_1 χρησιμοποιείται για την αναπαράσταση σημείων ελλειπτικής καμπύλης, ενώ η ομάδα G_2 χρησιμοποιείται ως ομάδα στόχος της διγραμμικής απεικόνισης. Στη συνέχεια επιλέγεται ένας γεννήτορας

$$g \in G_1$$

τέτοιος ώστε κάθε στοιχείο της ομάδας να μπορεί να παραχθεί μέσω επαναλαμβανόμενων πράξεων του g .

Πάνω στις ομάδες αυτές ορίζεται μια διγραμμική απεικόνιση

$$e : G_1 \times G_1 \rightarrow G_2.$$

Η απεικόνιση αυτή πρέπει να ικανοποιεί τρεις θεμελιώδεις ιδιότητες.

Η πρώτη ιδιότητα είναι η διγραμμικότητα (bilinearity). Για κάθε $P, Q \in G_1$ και για κάθε $a, b \in \mathbb{Z}_q$ ισχύει:

$$e(aP, bQ) = e(P, Q)^{ab}.$$

Η ιδιότητα αυτή αποτελεί τον βασικό μηχανισμό μέσω του οποίου επιτυγχάνεται η παραγωγή του κοινού ομαδικού κλειδιού.

Η δεύτερη ιδιότητα είναι ο μη εκφυλισμός (non-degeneracy). Συγκεκριμένα πρέπει να ισχύει:

$$e(g, g) \neq 1_{G_2}.$$

Η απαίτηση αυτή διασφαλίζει ότι η ζευγοποίηση δεν καταρρέει σε τετριμμένες τιμές και συνεπώς διατηρεί την κρυπτογραφική της χρησιμότητα.

Η τρίτη ιδιότητα είναι η υπολογισσιμότητα (computability), σύμφωνα με την οποία υπάρχει αποδοτικός αλγόριθμος υπολογισμού της τιμής

$$e(P, Q)$$

για οποιαδήποτε σημεία $P, Q \in G_1$.

Μετά τον ορισμό της ζευγοποίησης, ο KGC επιλέγει δύο κρυπτογραφικές συναρτήσεις κατακερματισμού.

Η πρώτη συνάρτηση χρησιμοποιείται για τη μετατροπή ταυτοτήτων ή συμβολοσειρών σε στοιχεία της ομάδας G_1 :

$$H_1 : \{0, 1\}^* \rightarrow G_1.$$

Η συνάρτηση αυτή μοντελοποιείται ως random oracle και χρησιμοποιείται για τη χαρτογράφηση δεδομένων σε σημεία της ελλειπτικής καμπύλης.

Η δεύτερη συνάρτηση κατακερματισμού ορίζεται ως:

$$H_2 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*.$$

Η συνάρτηση αυτή χρησιμοποιείται για τη δημιουργία τιμών δέσμευσης και για την παραγωγή παραμέτρων επαλήθευσης κατά τη διάρκεια του πρωτοκόλλου.

Στη συνέχεια ορίζεται το σύνολο των δημόσιων παραμέτρων του συστήματος:

$$params = (G_1, G_2, q, g, e, H_1, H_2).$$

Οι παράμετροι αυτές δημοσιεύονται σε όλους τους συμμετέχοντες και θεωρούνται κοινή γνώση.

Αξίζει να σημειωθεί ότι οι δημόσιες παράμετροι δεν περιέχουν καμία μυστική πληροφορία. Η αποκάλυψή τους δεν επηρεάζει την ασφάλεια του συστήματος, καθώς η προστασία βασίζεται στις υπολογιστικές υποθέσεις δυσκολίας των υποκείμενων μαθηματικών προβλημάτων.

Η ασφάλεια του προτεινόμενου πρωτοκόλλου εξαρτάται κυρίως από τη δυσκολία επίλυσης του προβλήματος Bilinear Diffie--Hellman (BDH). Δοθέντων των στοιχείων

$$(g, ag, bg, cg)$$

για τυχαίες τιμές

$$a, b, c \in \mathbb{Z}_q,$$

θεωρείται υπολογιστικά ανέφικτος ο υπολογισμός της ποσότητας

$$e(g, g)^{abc}.$$

Η παραδοχή αυτή αποτελεί μία από τις βασικότερες υποθέσεις ασφάλειας των pairing-based κρυπτογραφικών συστημάτων και χρησιμοποιείται ευρέως στη βιβλιογραφία για την απόδειξη της ασφάλειας πρωτοκόλλων ομαδικής συμφωνίας κλειδιού.

Με την ολοκλήρωση της φάσης αρχικοποίησης, όλοι οι συμμετέχοντες διαθέτουν ένα κοινό σύνολο κρυπτογραφικών παραμέτρων και είναι πλέον δυνατή η εκτέλεση των επόμενων σταδίων του πρωτοκόλλου, τα οποία αφορούν τη δημιουργία των κλειδιών χρηστών και την παραγωγή του κοινού ομαδικού κλειδιού.

6.3 Δημιουργία και Διαχείριση Κλειδιών

Μετά την ολοκλήρωση της φάσης αρχικοποίησης, κάθε χρήστης που επιθυμεί να συμμετάσχει στο σύστημα πρέπει να αποκτήσει τα απαραίτητα κρυπτογραφικά διαπιστευτήρια. Η διαδικασία αυτή πραγματοποιείται μία φορά κατά την εγγραφή του χρήστη και έχει ως στόχο τη δημιουργία ενός μοναδικού ζεύγους δημόσιου και ιδιωτικού κλειδιού.

Έστω ότι ένας χρήστης u_i επιθυμεί να ενταχθεί στο δίκτυο. Αρχικά δημιουργεί μια μοναδική ταυτότητα:

$$ID_i$$

η οποία μπορεί να αντιστοιχεί σε όνομα χρήστη, διεύθυνση ηλεκτρονικού ταχυδρομείου, αναγνωριστικό συσκευής ή οποιοδήποτε άλλο μοναδικό στοιχείο ταυτοποίησης.

Η ταυτότητα αυτή χρησιμοποιείται για την αναγνώριση του χρήστη εντός του συστήματος και συνδέεται με τα κρυπτογραφικά του κλειδιά.

Στη συνέχεια ο χρήστης επιλέγει τυχαία ένα ιδιωτικό κλειδί:

$$sk_i \in \mathbb{Z}_q^*.$$

Το ιδιωτικό κλειδί παραμένει γνωστό αποκλειστικά στον ίδιο τον χρήστη και δεν μεταδίδεται ποτέ μέσω του δικτύου.

Με βάση το ιδιωτικό κλειδί, υπολογίζεται το αντίστοιχο δημόσιο κλειδί:

$$pk_i = sk_i \cdot g.$$

Η πράξη αυτή αντιστοιχεί σε scalar multiplication πάνω στην ελλειπτική καμπύλη και παράγει ένα σημείο της ομάδας G_1 .

Το δημόσιο κλειδί μπορεί να δημοσιοποιηθεί χωρίς να αποκαλύπτεται το ιδιωτικό κλειδί, λόγω της δυσκολίας επίλυσης του προβλήματος του διακριτού λογαρίθμου σε ελλειπτικές καμπύλες.

Μετά τη δημιουργία του ζεύγους κλειδιών, ο χρήστης αποστέλλει προς το δίκτυο τις πληροφορίες:

$$(ID_i, pk_i).$$

Οι πληροφορίες αυτές αποθηκεύονται στο blockchain και καθίστανται διαθέσιμες σε όλους τους εξουσιοδοτημένους συμμετέχοντες.

Η αποθήκευση στο blockchain εξασφαλίζει ότι:

- τα δημόσια κλειδιά δεν μπορούν να αλλοιωθούν,
- η ταυτότητα κάθε συμμετέχοντα είναι επαληθεύσιμη,
- όλες οι μεταβολές καταγράφονται με χρονική σήμανση,
- παρέχεται πλήρης ιχνηλασιμότητα των ενεργειών.

Στο περιβάλλον Hyperledger Fabric, οι ταυτότητες των χρηστών συνδέονται επιπλέον με ψηφιακά πιστοποιητικά που εκδίδονται από την αντίστοιχη Certificate Authority (CA).

Κάθε πιστοποιητικό περιλαμβάνει:

- το αναγνωριστικό του χρήστη,
- το δημόσιο κλειδί του,
- πληροφορίες οργανισμού,
- χρονική περίοδο ισχύος,
- ψηφιακή υπογραφή της CA.

Η χρήση πιστοποιητικών επιτρέπει στο Membership Service Provider (MSP) του Hyperledger Fabric να πραγματοποιεί διαδικασίες αυθεντικοποίησης και εξουσιοδότησης πριν από οποιαδήποτε αλληλεπίδραση με το δίκτυο.

Μετά την ολοκλήρωση της διαδικασίας εγγραφής όλων των συμμετεχόντων, πραγματοποιείται μια κοινά αποδεκτή ταξινόμηση των χρηστών. Η ταξινόμηση αυτή είναι απαραίτητη ώστε όλοι οι κόμβοι να ακολουθούν την ίδια σειρά κατά την εκτέλεση του πρωτοκόλλου.

Έστω ότι η τελική διάταξη των χρηστών είναι:

$$U = \{u_1, u_2, \dots, u_n\}.$$

Η σειρά αυτή μπορεί να προκύπτει από:

- λεξικογραφική ταξινόμηση των ταυτοτήτων,
- ταξινόμηση βάσει δικτυακής διεύθυνσης,
- ταξινόμηση βάσει hash των δημόσιων κλειδιών.

Η ντετερμινιστική ταξινόμηση εξασφαλίζει ότι όλοι οι συμμετέχοντες καταλήγουν στην ίδια ακολουθία εκτέλεσης χωρίς να απαιτείται πρόσθετος συντονισμός.

Μετά την ολοκλήρωση της διαδικασίας δημιουργίας και δημοσίευσης των κλειδιών, κάθε χρήστης διαθέτει:

$$(ID_i, sk_i, pk_i).$$

Το ιδιωτικό κλειδί sk_i χρησιμοποιείται για την παραγωγή υπογραφών και τη συμμετοχή στον υπολογισμό του ομαδικού κλειδιού, ενώ το δημόσιο κλειδί pk_i χρησιμοποιείται για την επαλήθευση των μηνυμάτων και των κρυπτογραφικών αποδείξεων που ανταλλάσσονται κατά την εκτέλεση του πρωτοκόλλου.

Η ολοκλήρωση του σταδίου αυτού σηματοδοτεί την ετοιμότητα των συμμετεχόντων για την εκκίνηση της διαδικασίας συμφωνίας ομαδικού κλειδιού, η οποία παρουσιάζεται στην επόμενη ενότητα.

6.4 Πρωτόκολλο Συμφωνίας Ομαδικού Κλειδιού

Μετά την ολοκλήρωση της διαδικασίας εγγραφής και δημοσίευσης των δημόσιων κλειδιών, οι συμμετέχοντες είναι έτοιμοι να εκτελέσουν το πρωτόκολλο συμφωνίας ομαδικού κλειδιού. Σκοπός του πρωτοκόλλου είναι η παραγωγή ενός κοινού μυστικού κλειδιού το οποίο θα είναι γνωστό αποκλειστικά στα νόμιμα μέλη της ομάδας.

Η διαδικασία βασίζεται στη συνεισφορά όλων των συμμετεχόντων και ανήκει στην κατηγορία των contributory group key agreement protocols, καθώς κάθε χρήστης εισάγει τη δική του τυχαία πληροφορία στον τελικό υπολογισμό του κλειδιού.

Έστω ότι η ομάδα αποτελείται από τους χρήστες:

$$U = \{u_1, u_2, \dots, u_n\}.$$

Η σειρά εκτέλεσης θεωρείται ήδη γνωστή από το προηγούμενο στάδιο και είναι κοινή για όλους τους συμμετέχοντες.

6.4.1 Παραγωγή Τυχαίων Τιμών

Για κάθε νέα εκτέλεση του πρωτοκόλλου, κάθε χρήστης u_i επιλέγει ανεξάρτητα μία νέα τυχαία τιμή:

$$m_i \in \mathbb{Z}_q^*.$$

Η τιμή αυτή αποτελεί την προσωρινή μυστική συνεισφορά του χρήστη στη συγκεκριμένη συνεδρία.

Η επιλογή νέας τυχαίας τιμής σε κάθε εκτέλεση είναι απαραίτητη για την επίτευξη forward secrecy, καθώς αποτρέπει τη συσχέτιση διαφορετικών συνεδριών μεταξύ τους.

Οι τιμές m_i παραμένουν ιδιωτικές και δεν αποκαλύπτονται ποτέ στο δίκτυο.

6.4.2 Υπολογισμός Δημόσιων Συνεισφορών

Αφού επιλεγεί η τυχαία τιμή m_i , ο χρήστης υπολογίζει:

$$M_i = m_i g.$$

Η τιμή M_i αποτελεί τη δημόσια συνεισφορά του χρήστη στην τρέχουσα εκτέλεση. Όλοι οι συμμετέχοντες δημοσιεύουν τις αντίστοιχες τιμές τους:

$$M_1, M_2, \dots, M_n.$$

Οι τιμές αυτές αποστέλλονται μέσω του δικτύου και καταγράφονται στο blockchain ώστε να είναι διαθέσιμες για μελλοντική επαλήθευση.

Η γνώση του M_i δεν επιτρέπει τον υπολογισμό του m_i , λόγω της δυσκολίας επίλυσης του προβλήματος διακριτού λογαρίθμου πάνω στην ελλειπτική καμπύλη.

6.4.3 Δημιουργία Κρυπτογραφικής Δέσμευσης

Για την προστασία της ακεραιότητας των μηνυμάτων, κάθε κόμβος δημιουργεί μια κρυπτογραφική δέσμευση.

Έστω ότι ο χρήστης u_i έχει στη διάθεσή του το ιστορικό ανταλλαγής μηνυμάτων μέχρι τη χρονική στιγμή παραγωγής της δέσμευσης.

Το ιστορικό αυτό συμβολίζεται ως:

$$transcript_i.$$

Στη συνέχεια υπολογίζεται:

$$h_i = H_2(M_i \parallel pk_i \parallel transcript_i).$$

Η τιμή h_i δεσμεύει κρυπτογραφικά όλα τα στοιχεία που συμμετέχουν στην εκτέλεση.

Οποιαδήποτε τροποποίηση των ανταλλασσόμενων δεδομένων θα οδηγήσει σε διαφορετική τιμή hash και κατά συνέπεια σε αποτυχία της διαδικασίας επαλήθευσης.

6.4.4 Δημιουργία Ψηφιακής Υπογραφής

Μετά τον υπολογισμό της τιμής κατακερματισμού, κάθε χρήστης δημιουργεί μια κρυπτογραφική υπογραφή που συνδέει το δημόσιο μήνυμα με την ταυτότητά του.

Η υπογραφή ορίζεται ως:

$$Sig_i = sk_i \cdot h_i + sk_i \cdot M_i.$$

Η τιμή αυτή ενσωματώνει τόσο τη μυστική πληροφορία του χρήστη όσο και τη δημόσια συνεισφορά του.

Κατά συνέπεια, οποιαδήποτε προσπάθεια παραποίησης των δεδομένων καθίσταται ανιχνεύσιμη από τους υπόλοιπους συμμετέχοντες.

6.4.5 Ανταλλαγή Μηνυμάτων

Ο κάθε χρήστης μεταδίδει στο δίκτυο το ακόλουθο σύνολο πληροφοριών:

$$(ID_i, pk_i, M_i, Sig_i).$$

Τα δεδομένα αυτά μπορούν να μεταδοθούν είτε απευθείας στους υπόλοιπους κόμβους είτε μέσω της υποδομής Hyperledger Fabric.

Η καταγραφή των δεδομένων στο blockchain παρέχει χρονική σήμανση, ακεραιότητα και δυνατότητα μεταγενέστερης επαλήθευσης.

Μετά την ολοκλήρωση της ανταλλαγής μηνυμάτων, κάθε συμμετέχων διαθέτει το πλήρες σύνολο:

$$\{(ID_i, pk_i, M_i, Sig_i)\}_{i=1}^n.$$

Το σύνολο αυτό αποτελεί την είσοδο για τη διαδικασία επαλήθευσης και τον τελικό υπολογισμό του ομαδικού κλειδιού.

Στο επόμενο στάδιο, κάθε κόμβος επαληθεύει την εγκυρότητα όλων των μηνυμάτων που έλαβε πριν συμμετάσχει στον υπολογισμό του κοινού μυστικού κλειδιού.

6.5 Επαλήθευση Μηνυμάτων

Πριν από τον υπολογισμό του τελικού ομαδικού κλειδιού, κάθε συμμετέχων οφείλει να επαληθεύσει την εγκυρότητα όλων των μηνυμάτων που έλαβε κατά τη διάρκεια της εκτέλεσης του πρωτοκόλλου. Η διαδικασία αυτή είναι απαραίτητη ώστε να αποκλειστούν κακόβουλοι ή εσφαλμένοι κόμβοι που επιχειρούν να εισάγουν μη έγκυρες τιμές στον υπολογισμό του κοινού κλειδιού.

Η επαλήθευση πραγματοποιείται ανεξάρτητα από κάθε χρήστη και βασίζεται στις ιδιότητες της διγραμμικής απεικόνισης. Για κάθε συμμετέχοντα u_i , οι υπόλοιποι κόμβοι διαθέτουν τα ακόλουθα δημόσια δεδομένα:

$$(ID_i, pk_i, M_i, Sig_i).$$

Επιπλέον, κάθε κόμβος μπορεί να ανακατασκευάσει το αντίστοιχο ιστορικό ανταλλαγής μηνυμάτων και να υπολογίσει εκ νέου την τιμή

$$h_i = H_2(M_i \parallel pk_i \parallel transcript_i).$$

Η επαλήθευση πραγματοποιείται μέσω της σχέσης

$$e(Sig_i, g) = e(h_i, pk_i) \cdot e(M_i, g).$$

Εφόσον η παραπάνω ισότητα επαληθεύεται, το μήνυμα θεωρείται έγκυρο και ο χρήστης u_i αναγνωρίζεται ως νόμιμος συμμετέχων της τρέχουσας εκτέλεσης.

Αντίθετα, εάν η ισότητα δεν ισχύει, το μήνυμα απορρίπτεται και ο αντίστοιχος κόμβος εξαιρείται από τη διαδικασία παραγωγής του ομαδικού κλειδιού.

Η διαδικασία αυτή εκτελείται για όλους τους συμμετέχοντες:

$$u_1, u_2, \dots, u_n.$$

Μόνο όταν όλες οι επαληθεύσεις ολοκληρωθούν επιτυχώς επιτρέπεται η μετάβαση στο επόμενο στάδιο του πρωτοκόλλου.

Η χρήση της διγραμμικής απεικόνισης παρέχει έναν αποδοτικό μηχανισμό επαλήθευσης, καθώς επιτρέπει τη σύνδεση των δημόσιων παραμέτρων με τις αντίστοιχες υπογραφές χωρίς αποκάλυψη των ιδιωτικών κλειδιών των χρηστών.

Επιπλέον, η συμπερίληψη του ιστορικού ανταλλαγής μηνυμάτων στον υπολογισμό της τιμής κατακερματισμού αποτρέπει επιθέσεις επανάληψης (replay attacks). Κάθε εκτέλεση

του πρωτοκόλλου οδηγεί σε διαφορετική τιμή h_i , ακόμη και αν ο ίδιος χρήστης συμμετέχει πολλές φορές.

Η διαδικασία επαλήθευσης παρέχει τις ακόλουθες ιδιότητες ασφάλειας:

- αυθεντικότητα συμμετέχοντος,
- ακεραιότητα ανταλλασσόμενων δεδομένων,
- ανίχνευση παραποιημένων μηνυμάτων,
- προστασία από replay attacks,
- αποτροπή μη εξουσιοδοτημένης συμμετοχής.

Μετά την επιτυχή ολοκλήρωση όλων των ελέγχων, κάθε κόμβος διαθέτει ένα κοινό αποδεκτό και επαληθευμένο σύνολο δημόσιων συνεισφορών. Οι τιμές αυτές χρησιμοποιούνται στο επόμενο στάδιο για τον υπολογισμό του τελικού ομαδικού κλειδιού.

6.6 Παραγωγή Ομαδικού Κλειδιού

Μετά την επιτυχή ολοκλήρωση της διαδικασίας επαλήθευσης, όλοι οι συμμετέχοντες διαθέτουν ένα κοινό και αξιόπιστο σύνολο δημόσιων συνεισφορών:

$$\{M_1, M_2, \dots, M_n\}.$$

Οι τιμές αυτές αποτελούν τη βάση για τον υπολογισμό του τελικού ομαδικού κλειδιού.

Σε αντίθεση με τα κλασικά πρωτόκολλα Diffie--Hellman, όπου το τελικό μυστικό προκύπτει από διαδοχικές εκθετικοποιήσεις, στο προτεινόμενο σχήμα αξιοποιούνται οι ιδιότητες της διγραμμικής απεικόνισης ώστε να παραχθεί ένα κοινό μυστικό το οποίο εξαρτάται από τη συνεισφορά όλων των συμμετεχόντων.

6.6.1 Παραγωγή των Τιμών f_i

Μετά την ολοκλήρωση της αλυσίδας ανταλλαγής μηνυμάτων, ο τελευταίος κόμβος της εκτέλεσης, συμβολιζόμενος ως u_n , επιλέγει μία νέα τυχαία τιμή:

$$t_n \in \mathbb{Z}_q^*.$$

Η τιμή αυτή αποτελεί τον τελικό παράγοντα μυστικότητας της συνεδρίας και χρησιμοποιείται για τη δημιουργία του κοινού κλειδιού.

Στη συνέχεια ο κόμβος υπολογίζει:

$$f_1 = t_n M_1,$$

$$f_2 = t_n M_2,$$

$$\vdots$$

$$f_n = t_n M_n.$$

Γενικά:

$$f_i = t_n M_i, \quad 1 \leq i \leq n.$$

Επειδή κάθε δημόσια συνεισφορά έχει τη μορφή

$$M_i = m_i g,$$

προκύπτει:

$$f_i = t_n m_i g.$$

Οι τιμές αυτές ενσωματώνουν τόσο τη μυστική συμβολή του χρήστη u_i όσο και την τυχαία τιμή t_n που εισάγει ο τελευταίος κόμβος.

6.6.2 Υπολογισμός της Παραμέτρου β

Αφού παραχθούν όλες οι τιμές f_i , ο τελευταίος κόμβος υπολογίζει το άθροισμά τους:

$$\beta = \sum_{i=1}^n f_i.$$

Αναπτύσσοντας τον παραπάνω τύπο προκύπτει:

$$\beta = t_n \sum_{i=1}^n M_i.$$

Η παράμετρος β αποτελεί μια συλλογική αναπαράσταση της συνεισφοράς όλων των συμμετεχόντων και χρησιμοποιείται στη συνέχεια για την παραγωγή του τελικού κλειδιού.

Ο κόμβος u_n δημοσιεύει στο δίκτυο το σύνολο:

$$(f_1, f_2, \dots, f_n),$$

ώστε όλοι οι υπόλοιποι συμμετέχοντες να μπορούν να ολοκληρώσουν τοπικά τον υπολογισμό του ομαδικού κλειδιού.

6.6.3 Τοπικός Υπολογισμός της Παραμέτρου α_i

Αφού κάθε κόμβος λάβει τις τιμές f_i , χρησιμοποιεί τη δική του μυστική τιμή m_i για να υπολογίσει:

$$\alpha_i = m_i^{-1} f_i.$$

Αντικαθιστώντας το $f_i = t_n m_i g$, προκύπτει:

$$\alpha_i = m_i^{-1} (t_n m_i g).$$

Λόγω της ύπαρξης του αντιστρόφου του m_i στο $\mathbb{Z}_q^*$, έχουμε:

$$\alpha_i = t_n g.$$

Επομένως όλοι οι συμμετέχοντες καταλήγουν στην ίδια ακριβώς τιμή α_i , ανεξάρτητα από το ποια ήταν η αρχική τους τυχαία συνεισφορά.

6.6.4 Υπολογισμός του Τελικού Ομαδικού Κλειδιού

Μετά τον υπολογισμό των α_i και β , κάθε συμμετέχων υπολογίζει το τελικό ομαδικό κλειδί μέσω της διγραμμικής απεικόνισης:

$$GK_i = e(\alpha_i, \beta).$$

Αντικαθιστώντας τις προηγούμενες σχέσεις:

$$GK_i = e \left(t_n g, t_n \sum_{j=1}^n M_j \right).$$

Εφαρμόζοντας τη διγραμμικότητα της απεικόνισης:

$$GK_i = e \left(g, \sum_{j=1}^n M_j \right)^{t_n^2}.$$

Παρατηρούμε ότι το τελικό κλειδί εξαρτάται τόσο από την τυχαία τιμή t_n όσο και από το σύνολο των δημόσιων συνεισφορών όλων των συμμετεχόντων.

Επομένως:

$$GK_1 = GK_2 = \dots = GK_n.$$

Όλοι οι χρήστες παράγουν το ίδιο κοινό μυστικό χωρίς να απαιτείται περαιτέρω ανταλλαγή πληροφοριών.

6.6.5 Αυτοπιστοποίηση του Κλειδιού

Ένα ιδιαίτερο χαρακτηριστικό του πρωτοκόλλου είναι ότι το παραγόμενο ομαδικό κλειδί μπορεί να επαληθευθεί χωρίς επιπλέον επικοινωνία μεταξύ των συμμετεχόντων.

Κάθε χρήστης μπορεί να ελέγξει την ορθότητα του υπολογισμού του μέσω της σχέσης:

$$e \left(\alpha_i, \sum_{j=1}^n M_j \right) = e(\beta, g).$$

Εάν η παραπάνω ισότητα ισχύει, τότε ο χρήστης γνωρίζει ότι το παραγόμενο κλειδί είναι συνεπές με τις δημόσιες συνεισφορές όλων των μελών της ομάδας.

Η ιδιότητα αυτή είναι γνωστή ως *key self-certification* και μειώνει σημαντικά το επικοινωνιακό κόστος, καθώς δεν απαιτείται ανταλλαγή πρόσθετων μηνυμάτων επιβεβαίωσης.

6.7 Ενσωμάτωση Blockchain

Σε αντίθεση με τα κλασικά πρωτόκολλα Group Key Agreement, όπου η ανταλλαγή των μηνυμάτων πραγματοποιείται αποκλειστικά μέσω καναλιών επικοινωνίας μεταξύ των συμμετεχόντων, το προτεινόμενο πρωτόκολλο αξιοποιεί μια υποδομή blockchain για την καταγραφή και επαλήθευση κρίσιμων δεδομένων της εκτέλεσης.

Η υποδομή blockchain βασίζεται στο Hyperledger Fabric, ένα permissioned blockchain framework το οποίο επιτρέπει τον έλεγχο ταυτότητας των συμμετεχόντων, τη διαχείριση πολιτικών πρόσβασης και την αποθήκευση δεδομένων σε αμετάβλητο κατακευκτικό καθολικό.

Ο κύριος στόχος της ενσωμάτωσης του blockchain είναι η παροχή μηχανισμών ακεραιότητας, ιχνηλασιμότητας και μη αποποίησης ευθύνης κατά τη διάρκεια της διαδικασίας συμφωνίας του ομαδικού κλειδιού.

6.7.1 Καταγραφή Δημοσίων Παραμέτρων

Κατά την έναρξη του πρωτοκόλλου, οι συμμετέχοντες δημοσιεύουν τις δημόσιες πληροφορίες τους στο blockchain.

Για κάθε χρήστη u_i αποθηκεύονται:

$$(ID_i, pk_i).$$

Οι πληροφορίες αυτές καταγράφονται ως συναλλαγές στο καθολικό και είναι διαθέσιμες σε όλους τους εξουσιοδοτημένους συμμετέχοντες.

Με τον τρόπο αυτό αποτρέπεται η αντικατάσταση δημόσιων κλειδιών από κακόβουλους χρήστες και διασφαλίζεται η ύπαρξη ενός μοναδικού σημείου αναφοράς για την επαλήθευση των ταυτοτήτων.

6.7.2 Καταγραφή Δημοσίων Συνεισφορών

Κατά την εκτέλεση του πρωτοκόλλου, κάθε χρήστης δημιουργεί τη δημόσια συνεισφορά:

$$M_i = m_i g.$$

Η τιμή αυτή καταγράφεται στο blockchain μαζί με τα αντίστοιχα μεταδεδομένα της εκτέλεσης. Για κάθε συμμετέχοντα δημιουργείται μία εγγραφή της μορφής:

$$(ID_i, M_i, timestamp).$$

Η χρονική σήμανση επιτρέπει την ανακατασκευή της ακολουθίας των γεγονότων ακόμη και μετά την ολοκλήρωση της συνεδρίας.

6.7.3 Καταγραφή Υπογραφών

Εκτός από τις δημόσιες συνεισφορές, στο blockchain αποθηκεύονται και οι παραγόμενες υπογραφές:

$$Sig_i.$$

Οι υπογραφές χρησιμοποιούνται ως αποδεικτικά στοιχεία ότι ο αντίστοιχος χρήστης συμμετείχε στην εκτέλεση του πρωτοκόλλου.

Η ύπαρξη των υπογραφών στο καθολικό επιτρέπει τη μεταγενέστερη επαλήθευση της διαδικασίας χωρίς να απαιτείται πρόσβαση στους αρχικούς συμμετέχοντες.

6.7.4 Χρήση Chaincode

Η αλληλεπίδραση των συμμετεχόντων με το blockchain πραγματοποιείται μέσω chaincode υλοποιημένου στο Hyperledger Fabric.

Το chaincode είναι υπεύθυνο για:

- την καταχώριση δημόσιων κλειδιών,
- την αποθήκευση των τιμών M_i ,
- την αποθήκευση των υπογραφών Sig_i ,
- την καταγραφή των χρονικών σφραγίδων,
- την ανάκτηση δεδομένων προηγούμενων εκτελέσεων.

Κάθε συναλλαγή που υποβάλλεται στο chaincode επικυρώνεται από τους peers του δικτύου πριν καταγραφεί οριστικά στο ledger.

6.7.5 Ρόλος των Peers

Οι peer nodes του Hyperledger Fabric λειτουργούν ως κατανεμημένοι επαληθευτές των δεδομένων του πρωτοκόλλου.

Κάθε peer:

- διατηρεί αντίγραφο του ledger,
- εκτελεί το chaincode,
- επικυρώνει συναλλαγές,
- συμμετέχει στη διαδικασία endorsement.

Η ύπαρξη πολλαπλών peers εξαλείφει την εξάρτηση από έναν μοναδικό εξυπηρετητή και αυξάνει την ανθεκτικότητα του συστήματος.

6.7.6 Ρόλος των Orderers

Οι orderer nodes είναι υπεύθυνοι για τη δημιουργία της τελικής ακολουθίας των συναλλαγών. Οι συναλλαγές που αφορούν:

$$pk_i, M_i, Sig_i$$

συγκεντρώνονται από το ordering service και οργανώνονται σε blocks.

Στη συνέχεια τα blocks διανέμονται στους peers για τελική επικύρωση και αποθήκευση.

Η διαδικασία αυτή εξασφαλίζει ότι όλοι οι συμμετέχοντες διαθέτουν ακριβώς την ίδια εικόνα της εκτέλεσης του πρωτοκόλλου.

6.7.7 Ιχνηλασιμότητα και Μη Αποποίηση Ευθύνης

Η καταγραφή όλων των κρίσιμων γεγονότων στο blockchain παρέχει πλήρη δυνατότητα ελέγχου της εκτέλεσης.

Ένας εξουσιοδοτημένος ελεγκτής μπορεί να ανακτήσει:

$$(pk_i, M_i, Sig_i, timestamp)$$

για οποιονδήποτε συμμετέχοντα και να επαληθεύσει ανεξάρτητα την ορθότητα των ενεργειών του.

Επιπλέον, επειδή οι εγγραφές είναι αμετάβλητες μετά την επικύρωσή τους, κανένας συμμετέχων δεν μπορεί να αρνηθεί μεταγενέστερα ότι έλαβε μέρος στη διαδικασία.

Η ιδιότητα αυτή είναι ιδιαίτερα σημαντική σε επιχειρησιακά και βιομηχανικά περιβάλλοντα όπου απαιτείται λογοδοσία και πλήρες audit trail.

6.7.8 Πλεονεκτήματα της Ενσωμάτωσης Blockchain

Η χρήση του Hyperledger Fabric προσθέτει ένα επιπλέον επίπεδο ασφάλειας στο προτεινόμενο πρωτόκολλο.

Συγκεκριμένα παρέχει:

- αμετάβλητη αποθήκευση δεδομένων,
- αποκεντρωμένη επαλήθευση,
- προστασία από αλλοίωση μηνυμάτων,
- πλήρη ιχνηλασιμότητα,
- μη αποποίηση ευθύνης,
- δυνατότητα μεταγενέστερου ελέγχου της εκτέλεσης.

Ως αποτέλεσμα, το blockchain δεν λειτουργεί απλώς ως μηχανισμός αποθήκευσης, αλλά ως ενεργό συστατικό της αρχιτεκτονικής ασφάλειας του προτεινόμενου πρωτοκόλλου.

6.8 Ανάλυση Ορθότητας

Στην παρούσα ενότητα αποδεικνύεται ότι όλοι οι νόμιμοι συμμετέχοντες του πρωτοκόλλου καταλήγουν στο ίδιο ομαδικό κλειδί μετά την ολοκλήρωση της διαδικασίας ανταλλαγής μηνυμάτων και των αντίστοιχων υπολογισμών.

Η ορθότητα του πρωτοκόλλου βασίζεται στις ιδιότητες της διγραμμικής απεικόνισης και στη συνεπή χρήση των δημόσιων και ιδιωτικών παραμέτρων από όλους τους συμμετέχοντες.

Έστω ότι κάθε χρήστης u_i επιλέγει μία τυχαία τιμή:

$$m_i \in \mathbb{Z}_q^*$$

και υπολογίζει τη δημόσια συνεισφορά:

$$M_i = m_i g.$$

Μετά την ολοκλήρωση της διαδικασίας ανταλλαγής μηνυμάτων, ο τελευταίος κόμβος επιλέγει μία επιπλέον τυχαία τιμή:

$$t_n \in \mathbb{Z}_q^*.$$

Στη συνέχεια παράγει για κάθε συμμετέχοντα την τιμή:

$$f_i = t_n M_i.$$

Αντικαθιστώντας τον ορισμό του M_i προκύπτει:

$$f_i = t_n(m_i g).$$

Λόγω της προσεταιριστικής ιδιότητας του scalar multiplication έχουμε:

$$f_i = t_n m_i g.$$

Κάθε χρήστης παραλαμβάνει την αντίστοιχη τιμή f_i και υπολογίζει:

$$\alpha_i = m_i^{-1} f_i.$$

Αντικαθιστώντας το f_i προκύπτει:

$$\alpha_i = m_i^{-1} (t_n m_i g).$$

Επειδή το m_i ανήκει στο $\mathbb{Z}_q^*$ διαθέτει μοναδικό αντίστροφο modulo q .
Επομένως:

$$m_i^{-1} m_i = 1.$$

Άρα:

$$\alpha_i = t_n g.$$

Παρατηρούμε ότι το αποτέλεσμα αυτό είναι ανεξάρτητο από τον δείκτη i .
Συνεπώς:

$$\alpha_1 = \alpha_2 = \dots = \alpha_n = t_n g.$$

Στη συνέχεια ο τελευταίος κόμβος υπολογίζει:

$$\beta = \sum_{j=1}^n f_j.$$

Αντικαθιστώντας τους ορισμούς των f_j :

$$\beta = \sum_{j=1}^n t_n M_j.$$

Εξάγοντας τον κοινό παράγοντα:

$$\beta = t_n \sum_{j=1}^n M_j.$$

Όλοι οι συμμετέχοντες διαθέτουν πλέον τις ίδιες τιμές β και α_i .
Το τελικό ομαδικό κλειδί ορίζεται ως:

$$GK_i = e(\alpha_i, \beta).$$

Αντικαθιστώντας τα προηγούμενα αποτελέσματα:

$$GK_i = e \left(t_n g, t_n \sum_{j=1}^n M_j \right).$$

Από τη διγραμμικότητα της απεικόνισης προκύπτει:

$$GK_i = e \left(g, \sum_{j=1}^n M_j \right)^{t_n^2}.$$

Η παραπάνω έκφραση δεν περιέχει καμία μεταβλητή που να εξαρτάται από συγκεκριμένο συμμετέχοντα.

Επομένως για κάθε ζεύγος χρηστών u_i και u_j ισχύει:

$$GK_i = GK_j.$$

Άρα:

$$GK_1 = GK_2 = \dots = GK_n.$$

Συνεπώς όλοι οι νόμιμοι συμμετέχοντες παράγουν ακριβώς το ίδιο ομαδικό κλειδί. Επιπλέον, επειδή το τελικό κλειδί εξαρτάται από:

$$M_1, M_2, \dots, M_n$$

και από την τυχαία τιμή:

$$t_n,$$

η συνεισφορά κάθε συμμετέχοντα επηρεάζει άμεσα το τελικό αποτέλεσμα.

Η ιδιότητα αυτή εξασφαλίζει ότι το πρωτόκολλο ανήκει στην κατηγορία των contributory group key agreement protocols, καθώς κανένας συμμετέχων δεν μπορεί να καθορίσει μονομερώς το τελικό κλειδί.

Επομένως, υπό την προϋπόθεση ότι όλοι οι κόμβοι ακολουθούν πιστά τα βήματα του πρωτοκόλλου και ότι οι επαληθεύσεις ολοκληρώνονται επιτυχώς, το προτεινόμενο σχήμα οδηγεί πάντοτε στην παραγωγή ενός μοναδικού και κοινού ομαδικού κλειδιού για όλα τα μέλη της ομάδας.

6.9 Ανάλυση Ασφάλειας

Στην ενότητα αυτή αναλύονται οι ιδιότητες ασφάλειας του προτεινόμενου πρωτοκόλλου και αξιολογείται η ανθεκτικότητά του έναντι γνωστών επιθέσεων. Η ασφάλεια του συστήματος βασίζεται τόσο στις μαθηματικές ιδιότητες των διγραμμικών ζευγοποιήσεων όσο και στους μηχανισμούς ακεραιότητας και καταγραφής που προσφέρει το Hyperledger Fabric.

Υποθέτουμε ότι ο αντίπαλος έχει πλήρη πρόσβαση στο δημόσιο δίκτυο επικοινωνίας, μπορεί να παρακολουθεί, να καταγράφει και να επαναμεταδίδει μηνύματα, αλλά δεν μπορεί να επιλύσει αποδοτικά το πρόβλημα Bilinear Diffie--Hellman (BDH).

6.9.1 Αυθεντικοποίηση Συμμετεχόντων

Η αυθεντικοποίηση των χρηστών επιτυγχάνεται μέσω του συνδυασμού δημοσίων κλειδιών, ψηφιακών υπογραφών και πιστοποιητικών του Hyperledger Fabric.

Για κάθε χρήστη u_i ισχύει:

$$pk_i = sk_i g.$$

Οι υπογραφές που παράγονται κατά την εκτέλεση του πρωτοκόλλου συνδέονται άμεσα με το αντίστοιχο ιδιωτικό κλειδί. Κατά συνέπεια, ένας επιτιθέμενος που δεν γνωρίζει το sk_i αδυνατεί να δημιουργήσει έγκυρη υπογραφή για λογαριασμό του χρήστη.

Επιπλέον, η ύπαρξη πιστοποιητικών που εκδίδονται από την Certificate Authority του Fabric εξασφαλίζει ότι κάθε δημόσιο κλειδί αντιστοιχεί σε πραγματικό και εξουσιοδοτημένο συμμετέχοντα.

6.9.2 Ακεραιότητα Δεδομένων

Η ακεραιότητα των ανταλλασσόμενων δεδομένων εξασφαλίζεται μέσω της συνάρτησης κατακερματισμού:

$$h_i = H_2(M_i \parallel pk_i \parallel transcript_i).$$

Οποιαδήποτε τροποποίηση των δεδομένων οδηγεί σε διαφορετική τιμή hash, με αποτέλεσμα την αποτυχία της διαδικασίας επαλήθευσης.

Επομένως, ένας επιτιθέμενος δεν μπορεί να μεταβάλει τα ανταλλασσόμενα μηνύματα χωρίς να γίνει αντιληπτός από τους υπόλοιπους συμμετέχοντες.

6.9.3 Forward Secrecy

Η ιδιότητα forward secrecy απαιτεί η αποκάλυψη ενός τρέχοντος κλειδιού να μην επιτρέπει την ανάκτηση παλαιότερων κλειδιών συνεδρίας.

Στο προτεινόμενο πρωτόκολλο κάθε εκτέλεση χρησιμοποιεί νέες τυχαίες τιμές:

$$m_i \in \mathbb{Z}_q^*$$

και

$$t_n \in \mathbb{Z}_q^*.$$

Επειδή οι τιμές αυτές δημιουργούνται εκ νέου σε κάθε συνεδρία, η αποκάλυψη ενός μεταγενέστερου ομαδικού κλειδιού δεν παρέχει πληροφορία για προηγούμενα κλειδιά.

Κατά συνέπεια, το πρωτόκολλο παρέχει forward secrecy.

6.9.4 Backward Secrecy

Η backward secrecy προστατεύει τις προηγούμενες συνεδρίες από νέους συμμετέχοντες.

Όταν ένας νέος χρήστης εισέρχεται στην ομάδα, εκτελείται νέα διαδικασία δημιουργίας ομαδικού κλειδιού με νέες τυχαίες τιμές.

Ο νέος χρήστης δεν διαθέτει πρόσβαση στις τυχαίες συνεισφορές που χρησιμοποιήθηκαν σε παλαιότερες συνεδρίες και συνεπώς δεν μπορεί να υπολογίσει τα αντίστοιχα ομαδικά κλειδιά.

6.9.5 Key Independence

Η ιδιότητα key independence απαιτεί κάθε ομαδικό κλειδί να είναι ανεξάρτητο από προηγούμενα ή μελλοντικά κλειδιά.

Δεδομένου ότι κάθε συνεδρία χρησιμοποιεί διαφορετικές τιμές:

$$m_1, m_2, \dots, m_n$$

και διαφορετική τιμή:

$$t_n,$$

κάθε παραγόμενο κλειδί αποτελεί ανεξάρτητο αποτέλεσμα της εκτέλεσης.

Η γνώση ενός κλειδιού δεν επιτρέπει τον υπολογισμό οποιουδήποτε άλλου κλειδιού συνεδρίας.

6.9.6 Ανθεκτικότητα σε Replay Attacks

Σε μια επίθεση επανάληψης, ένας αντίπαλος επιχειρεί να επαναμεταδώσει παλαιά έγκυρα μηνύματα.

Στο προτεινόμενο πρωτόκολλο, η τιμή:

$$h_i = H_2(M_i \parallel pk_i \parallel transcript_i)$$

εξαρτάται από το ιστορικό της τρέχουσας εκτέλεσης.

Κατά συνέπεια, η επαναχρησιμοποίηση μηνυμάτων προηγούμενης συνεδρίας οδηγεί σε διαφορετική τιμή κατακερματισμού και αποτυχημένη επαλήθευση.

6.9.7 Ανθεκτικότητα σε Man-in-the-Middle Επιθέσεις

Ένας αντίπαλος που παρεμβάλλεται μεταξύ δύο χρηστών μπορεί να επιχειρήσει την αντικατάσταση δημοσίων συνεισφορών ή υπογραφών.

Ωστόσο, κάθε δημόσια συνεισφορά συνδέεται με:

$$pk_i$$

και με την αντίστοιχη υπογραφή:

$$Sig_i.$$

Η διαδικασία επαλήθευσης μέσω της διγραμμικής απεικόνισης απορρίπτει οποιαδήποτε αλλοιωμένη πληροφορία.

Επιπλέον, τα δημόσια κλειδιά είναι καταχωρημένα στο blockchain και δεν μπορούν να αντικατασταθούν χωρίς να εντοπιστεί η τροποποίηση.

6.9.8 Ανθεκτικότητα σε Επιθέσεις Συνωμοσίας

Ακόμη και αν ένα υποσύνολο χρηστών συνεργαστεί κακόβουλα, η παραγωγή του τελικού κλειδιού απαιτεί τη συνεισφορά όλων των συμμετεχόντων.

Το τελικό κλειδί εξαρτάται από:

$$M_1, M_2, \dots, M_n.$$

Η απουσία οποιασδήποτε συνεισφοράς οδηγεί σε διαφορετικό αποτέλεσμα και συνεπώς εμποδίζει τον υπολογισμό του πραγματικού ομαδικού κλειδιού.

6.9.9 Ακεραιότητα του Blockchain

Η χρήση του Hyperledger Fabric παρέχει πρόσθετη προστασία έναντι αλλοίωσης δεδομένων.

Οι πληροφορίες που αποθηκεύονται στο ledger επικυρώνονται μέσω των endorsement policies και καταγράφονται σε αμετάβλητη μορφή.

Οποιαδήποτε προσπάθεια μεταβολής παλαιότερων εγγραφών θα γινόταν άμεσα αντιληπτή από τους υπόλοιπους peers του δικτύου.

6.9.10 Σχέση με την Υπόθεση Bilinear Diffie--Hellman

Η υπολογιστική ασφάλεια του πρωτοκόλλου βασίζεται στη δυσκολία επίλυσης του προβλήματος Bilinear Diffie--Hellman.

Δοθέντων των στοιχείων:

$$(g, ag, bg, cg),$$

θεωρείται υπολογιστικά ανέφικτος ο υπολογισμός της τιμής:

$$e(g, g)^{abc}.$$

Το τελικό ομαδικό κλειδί εξαρτάται από αντίστοιχους εκθετικούς συνδυασμούς που προκύπτουν από τις τυχαίες συνεισφορές των χρηστών.

Κατά συνέπεια, ένας αντίπαλος που δεν μπορεί να επιλύσει το πρόβλημα BDH δεν μπορεί να υπολογίσει το ομαδικό κλειδί ακόμη και αν διαθέτει πλήρη γνώση όλων των δημόσιων παραμέτρων και των μηνυμάτων που ανταλλάχθηκαν κατά τη διάρκεια του πρωτοκόλλου.

Συνολικά, το προτεινόμενο πρωτόκολλο ικανοποιεί τις βασικές απαιτήσεις ασφάλειας των σύγχρονων συστημάτων Group Key Agreement και συνδυάζει τις ιδιότητες των bilinear pairings με τους μηχανισμούς ακεραιότητας και ιχνηλασιμότητας που προσφέρει το Hyperledger Fabric.

Κεφάλαιο 7

Τεχνολογίες Υλοποίησης

7.1 Spring Boot

Το Spring Boot αποτελεί ένα από τα δημοφιλέστερα πλαίσια ανάπτυξης (Java Frameworks) για τη δημιουργία εφαρμογών επιχειρησιακού επιπέδου. Αναπτύχθηκε από την ομάδα του Spring Framework με στόχο την απλοποίηση της διαδικασίας ανάπτυξης εφαρμογών μέσω της φιλοσοφίας ``Convention over Configuration".

Η βασική ιδέα του Spring Boot είναι η παροχή προκαθορισμένων ρυθμίσεων (auto-configuration), οι οποίες επιτρέπουν στον προγραμματιστή να επικεντρωθεί στην επιχειρησιακή λογική της εφαρμογής χωρίς να απαιτείται εκτεταμένη παραμετροποίηση.

Ένα από τα σημαντικότερα χαρακτηριστικά του είναι η ενσωμάτωση ενσωματωμένων web servers, όπως οι Tomcat, Jetty και Undertow, επιτρέποντας την εκτέλεση της εφαρμογής ως αυτόνομο εκτελέσιμο αρχείο χωρίς την ανάγκη εξωτερικού application server.

Στην παρούσα εργασία το Spring Boot χρησιμοποιήθηκε για την υλοποίηση τόσο του κεντρικού κόμβου (Master Node) όσο και των συμμετεχόντων κόμβων του πρωτοκόλλου. Μέσω του framework υλοποιήθηκαν τα REST APIs, οι JMS listeners, τα services και οι μηχανισμοί επικοινωνίας με το blockchain δίκτυο.

7.2 RESTful Web Services

Η αρχιτεκτονική REST (Representational State Transfer) αποτελεί ένα από τα επικρατέστερα πρότυπα σχεδίασης διαδικτυακών υπηρεσιών.

Στο μοντέλο αυτό οι λειτουργίες της εφαρμογής εκτίθενται μέσω HTTP endpoints και προσπελάζονται με χρήση μεθόδων όπως GET, POST, PUT και DELETE.

Τα δεδομένα μεταφέρονται συνήθως σε μορφή JSON, γεγονός που διευκολύνει τη διαλειτουργικότητα μεταξύ ετερογενών συστημάτων.

Στην παρούσα εργασία χρησιμοποιήθηκαν REST APIs για την επικοινωνία των κόμβων με το blockchain layer. Μέσω των APIs παρέχεται πρόσβαση στις λειτουργίες δημιουργίας, ανάγνωσης και ανάκτησης blocks από το Hyperledger Fabric δίκτυο.

7.3 Apache ActiveMQ Artemis

Το Apache ActiveMQ Artemis αποτελεί έναν σύγχρονο message broker υψηλών επιδόσεων, ο οποίος αναπτύσσεται στο πλαίσιο του Apache Software Foundation.

Ο Artemis παρέχει αξιόπιστη ασύγχρονη επικοινωνία μεταξύ κατανεμημένων εφαρμογών και υποστηρίζει πληθώρα πρωτοκόλλων, όπως AMQP, MQTT, STOMP και JMS.

Η λειτουργία του βασίζεται στην έννοια των μηνυμάτων (messages), τα οποία ανταλλάσσονται μέσω queues ή topics.

Στην παρούσα υλοποίηση ο Artemis χρησιμοποιείται ως μηχανισμός επικοινωνίας μεταξύ των κόμβων του πρωτοκόλλου. Κάθε συμμετέχων λαμβάνει και αποστέλλει μηνύματα μέσω ενός κοινού topic, επιτρέποντας τη διάδοση πληροφοριών χωρίς άμεσες συνδέσεις μεταξύ των κόμβων.

Η χρήση message broker αυξάνει την επεκτασιμότητα και την αξιοπιστία του συστήματος, καθώς αποσυνδέει λογικά τους αποστολείς από τους παραλήπτες.

7.4 Java Message Service (JMS)

Το Java Message Service (JMS) αποτελεί το πρότυπο ανταλλαγής μηνυμάτων της πλατφόρμας Java.

Μέσω του JMS οι εφαρμογές μπορούν να επικοινωνούν με ασύγχρονο τρόπο, χωρίς να απαιτείται ταυτόχρονη παρουσία αποστολέα και παραλήπτη.

Το πρότυπο υποστηρίζει δύο βασικά μοντέλα επικοινωνίας:

- Point-to-Point (Queues)
- Publish-Subscribe (Topics)

Στο μοντέλο Publish-Subscribe ένας παραγωγός μηνυμάτων δημοσιεύει πληροφορίες σε ένα topic και πολλαπλοί καταναλωτές μπορούν να λαμβάνουν τα ίδια μηνύματα.

Η συγκεκριμένη λειτουργία αξιοποιήθηκε στην παρούσα εργασία για τη διάδοση των μηνυμάτων του πρωτοκόλλου BDGKA. Τα μηνύματα τύπου START, PARTICIPANTS, GK-COMPUTATION και GK-FINAL διακινούνται μέσω JMS topics και παραλαμβάνονται από τους αντίστοιχους listeners κάθε κόμβου.

7.5 Docker

Το Docker αποτελεί πλατφόρμα εικονικοποίησης επιπέδου λειτουργικού συστήματος, η οποία επιτρέπει την εκτέλεση εφαρμογών μέσα σε απομονωμένα containers.

Κάθε container περιλαμβάνει όλες τις απαραίτητες βιβλιοθήκες, εξαρτήσεις και αρχεία ρυθμίσεων που απαιτούνται για την εκτέλεση της εφαρμογής.

Στην παρούσα εργασία το Docker χρησιμοποιήθηκε για την ανάπτυξη του δικτύου Hyperledger Fabric. Οι peer κόμβοι, οι orderers και οι certificate authorities εκτελούνται ως ανεξάρτητα containers, επιτρέποντας την εύκολη διαχείριση και ανάπτυξη του συστήματος.

7.6 JPBC Library

Η βιβλιοθήκη Java Pairing Based Cryptography (JPBC) αποτελεί μία από τις πλέον διαδεδομένες βιβλιοθήκες για την υλοποίηση pairings και πράξεων σε ελλειπτικές καμπύλες στη γλώσσα Java.

Η JPBC υποστηρίζει διάφορους τύπους pairings, καθώς και πράξεις στις ομάδες G_1 , G_2 , G_T και στο πεπερασμένο σώμα $\mathbb{Z}_r$.

Στην παρούσα εργασία η βιβλιοθήκη χρησιμοποιήθηκε για την παραγωγή κρυπτογραφικών κλειδιών, τη δημιουργία BLS υπογραφών, την επαλήθευση υπογραφών μέσω bilinear pairings και τον υπολογισμό του κοινού ομαδικού κλειδιού του πρωτοκόλλου BDGKA.

Κεφάλαιο 8

Σχεδιασμός Εφαρμογής

Η εφαρμογή που υλοποιήθηκε έχει ως στόχο την υλοποίηση ενός BDGKA πρωτοκόλλου και αποτελείται από πολλαπλά συνεργαζόμενα υποσυστήματα. Το συνολικό σύστημα περιλαμβάνει Hyperledger Fabric blockchain δίκτυο, smart contract, master Spring Boot κόμβο, peer nodes και βοηθητικά scripts διαχείρισης.

1. Hyperledger Fabric Blockchain test δίκτυο
2. Smart Contract
3. Master Spring Boot κόμβος
4. Spring Boot Peer κόμβοι
5. Scripts για την διαχείριση της εφαρμογής

8.1 Hyperledger Fabric Network

Το Hyperledger Fabric αποτελεί ένα open source blockchain framework με αρχιτεκτονική που βασίζεται σε peers, orderers και certificate authorities. Η πλατφόρμα λειτουργεί σε containerized περιβάλλον μέσω Docker και Docker Compose.

Οι οργανισμοί του δικτύου αντιπροσωπεύουν τους συμμετέχοντες φορείς. Στην παρούσα υλοποίηση χρησιμοποιείται test network με δύο οργανισμούς, όπου κάθε οργανισμός διαθέτει έναν Certificate Authority, έναν orderer και έναν peer.

Ο Certificate Authority είναι υπεύθυνος για την έκδοση πιστοποιητικών και τη διαχείριση ταυτοτήτων. Ο orderer συλλέγει συναλλαγές, τις ταξινομεί και δημιουργεί blocks. Ο peer διατηρεί το ledger και εκτελεί το smart contract.

Το σύστημα χρησιμοποιεί κανάλι με όνομα “myChannel”, το οποίο επιτρέπει ιδιωτική επικοινωνία μεταξύ των οργανισμών. Σε αυτό εγκαθίσταται το chaincode που υλοποιεί τη λογική του blockchain.

8.2 Organizations

Οι οργανισμοί είναι οι συμμετέχοντες στο δίκτυο, φερ'επιν οι εταιρίες. Στην εφαρμογή χρησιμοποιούμε το τεστ δίκτυο που παρέχεται από το hyperledger fabric. Το τεστ δίκτυο αποτελείται από δύο οργανισμούς οι οποίοι έχουν 1 Certificate Authority, 1 Orderer, 1 peer

- Certificate Authority: εκδίδει ψηφιακά πιστοποιητικά , Ταυτοποιεί χρήστες και κόμβους , διαχειρίζεται τις ταυτότητες
- Orderer: Συλλέγουν τις συναλλαγές , τις ταξινομούν και δημιουργούν τα μπλοκ στο blockchain
- Peer: διατηρούνε αντίγραφο του ledger και εκτελούν τα smart contracts

Channel

Η εφαρμογή χρησιμοποιεί και δημιουργεί το κανάλι "myChannel" , το κάθε κανάλι έχει το δικό του ledger και επιτρέπει στους οργανισμούς να επικοινωνούν (ιδιωτικά) Στο κανάλι καταθέτουμε το *contract – chaincode* που είναι η λογική της εφαρμογής όσο αφορά τις λειτουργίες του blockchain και εκτελείται στους peers

8.3 Ροή Εκτέλεσης του Πρωτοκόλλου

Η λειτουργία του συστήματος βασίζεται στη συνεργασία των συμμετεχόντων κόμβων, του blockchain δικτύου Hyperledger Fabric και του συστήματος ανταλλαγής μηνυμάτων. Σκοπός του πρωτοκόλλου είναι η παραγωγή ενός κοινού ομαδικού κλειδιού (Group Key), το οποίο υπολογίζεται συνεργατικά από όλους τους συμμετέχοντες χωρίς την ύπαρξη κεντρικού διαχειριστή κλειδιών.

Η εκτέλεση του πρωτοκόλλου χωρίζεται σε διακριτές φάσεις οι οποίες εκτελούνται διαδοχικά μέχρι την τελική παραγωγή και επαλήθευση του κοινού μυστικού.

8.3.1 Τύποι Μηνυμάτων

Η επικοινωνία μεταξύ των κόμβων βασίζεται σε μηνύματα JMS τα οποία μεταδίδονται μέσω του Artemis Message Broker.

Οι βασικοί τύποι μηνυμάτων που χρησιμοποιούνται είναι:

- START
- PARTICIPANTS
- INITIAL_PK_BROADCAST
- GK-COMPUTATION
- GK-FINAL

Κάθε τύπος αντιστοιχεί σε συγκεκριμένο στάδιο του πρωτοκόλλου και ενεργοποιεί διαφορετική λειτουργικότητα στον παραλήπτη.

8.3.2 Φάση Εκκίνησης (START)

Η πρώτη φάση του πρωτοκόλλου ξεκινά με την αποστολή ενός μηνύματος τύπου START προς όλους τους ενεργούς κόμβους.

Κατά τη φάση αυτή κάθε κόμβος δημιουργεί το ζεύγος των κρυπτογραφικών κλειδιών του. Συγκεκριμένα παράγεται ένα ιδιωτικό κλειδί το οποίο αποτελεί τυχαίο στοιχείο του συνόλου $\mathbb{Z}_r$ και το αντίστοιχο δημόσιο κλειδί το οποίο προκύπτει από πράξη βαθμωτού πολλαπλασιασμού πάνω στον γεννήτορα της ομάδας.

Μετά τη δημιουργία των κλειδιών κάθε κόμβος μεταδίδει το δημόσιο κλειδί του στους υπόλοιπους συμμετέχοντες μέσω του συστήματος ανταλλαγής μηνυμάτων.

8.3.3 Φάση Καθορισμού Συμμετεχόντων

Μετά την ανταλλαγή των δημοσίων κλειδιών οι κόμβοι αποκτούν πληροφορίες σχετικά με όλους τους συμμετέχοντες του πρωτοκόλλου.

Οι συμμετέχοντες ταξινομούνται με βάση το αναγνωριστικό τους και δημιουργείται μία λογική κυκλική τοπολογία επικοινωνίας. Για κάθε κόμβο καθορίζεται ο προηγούμενος και ο επόμενος γείτονας.

Η πληροφορία αυτή χρησιμοποιείται στη συνέχεια για τη σειριακή διάδοση των κρυπτογραφικών δεδομένων κατά τη διάρκεια του υπολογισμού του ομαδικού κλειδιού.

8.3.4 Φάση Ανταλλαγής Δημοσίων Κλειδιών

Κατά τη φάση αυτή πραγματοποιείται η ανταλλαγή των δημοσίων κλειδιών μεταξύ γειτονικών κόμβων.

Κάθε κόμβος αποθηκεύει τα δημόσια κλειδιά του προηγούμενου και του επόμενου συμμετέχοντα ώστε να είναι δυνατή η επαλήθευση των κρυπτογραφικών υπογραφών που θα ακολουθήσουν.

Η ύπαρξη των δημοσίων κλειδιών είναι απαραίτητη για την επαλήθευση της προέλευσης κάθε block που καταγράφεται στο blockchain.

8.3.5 Φάση Δημιουργίας Blocks

Ο πρώτος κόμβος της ακολουθίας δημιουργεί το αρχικό block του πρωτοκόλλου.

Το block περιλαμβάνει:

- Το αναγνωριστικό του block.
- Τη διεύθυνση του επόμενου κόμβου.
- Τη δημόσια πληροφορία M_i .
- Την τιμή H_{i2} .
- Την ψηφιακή υπογραφή.
- Μεταδεδομένα χρονικής σήμανσης.

Μετά τη δημιουργία του block πραγματοποιείται καταχώρηση στο blockchain μέσω του chaincode. Το block αποκτά μοναδικό hash και συνδέεται με το προηγούμενο block της αλυσίδας.

Η διαδικασία αυτή επαναλαμβάνεται από κάθε συμμετέχοντα καθώς το πρωτόκολλο εξελίσσεται.

8.3.6 Φάση Υπολογισμού Group Key

Η φάση αυτή αποτελεί τον πυρήνα του πρωτοκόλλου.

Κάθε κόμβος λαμβάνει το τελευταίο block από το blockchain και πραγματοποιεί επαλήθευση των κρυπτογραφικών υπογραφών μέσω διγραμμικών pairings.

Εφόσον η επαλήθευση ολοκληρωθεί επιτυχώς, ο κόμβος παράγει τη δική του συνεισφορά στο πρωτόκολλο μέσω της δημιουργίας νέου στοιχείου M_i και της αντίστοιχης υπογραφής.

Η νέα πληροφορία ενσωματώνεται σε νέο block το οποίο καταχωρείται στο blockchain και προωθείται στον επόμενο συμμετέχοντα.

Η διαδικασία συνεχίζεται μέχρι να φτάσει στον τελευταίο κόμβο της ομάδας.

8.3.7 Υπολογισμός του Τελικού Κλειδιού

Ο τελευταίος κόμβος της ακολουθίας διαθέτει πλέον όλες τις απαραίτητες συνεισφορές των συμμετεχόντων.

Χρησιμοποιώντας τις τιμές αυτές υπολογίζει το κοινό ομαδικό κλειδί μέσω πράξεων πάνω στις ομάδες του pairing.

Το παραγόμενο αποτέλεσμα ανήκει στην ομάδα στόχο G_T και αποτελεί το κοινό μυστικό όλων των συμμετεχόντων.

Για επιπλέον ασφάλεια εφαρμόζεται συνάρτηση κατακερματισμού SHA-512 πάνω στο παραγόμενο στοιχείο πριν χρησιμοποιηθεί από τις ανώτερες εφαρμογές.

8.3.8 Φάση Τελικής Επαλήθευσης

Μετά τον υπολογισμό του τελικού κλειδιού ο τελευταίος κόμβος διανέμει τις απαραίτητες παραμέτρους στους υπόλοιπους συμμετέχοντες.

Κάθε κόμβος επανακατασκευάζει το κοινό κλειδί χρησιμοποιώντας τις τοπικές του πληροφορίες και τις τιμές που έλαβε από τον τελευταίο συμμετέχοντα.

Η ορθότητα του αποτελέσματος επαληθεύεται μέσω σύγκρισης των κατακερματισμένων τιμών του group key.

Εφόσον όλοι οι κόμβοι παράγουν την ίδια τιμή κατακερματισμού, θεωρείται ότι το πρωτόκολλο ολοκληρώθηκε επιτυχώς και όλοι οι συμμετέχοντες διαθέτουν το ίδιο κοινό μυστικό.

8.3.9 Υλοποίηση Chaincode

Το chaincode αποτελεί το αποκεντρωμένο τμήμα της εφαρμογής και εκτελείται στους peer κόμβους του Hyperledger Fabric. Η κύρια αρμοδιότητά του είναι η αποθήκευση και διαχείριση των blocks που παράγονται κατά την εκτέλεση του πρωτοκόλλου.

Η υλοποίηση πραγματοποιήθηκε στη γλώσσα Go αξιοποιώντας το Fabric Contract API. Κάθε block αποθηκεύεται στο world state του Fabric σε μορφή JSON και συνοδεύεται από πληροφορίες όπως το αναγνωριστικό του block, το hash του προηγούμενου block, κρυπτογραφικές παραμέτρους και χρονικές σημάνσεις.

Η λειτουργία AppendBlock αποτελεί τη σημαντικότερη συναλλαγή του smart contract. Κατά την εκτέλεσή της δημιουργείται ένα νέο block, υπολογίζεται το κρυπτογραφικό του αποτύπωμα και αποθηκεύεται στο ledger.

Η ύπαρξη του πεδίου PrevHash δημιουργεί μία αλυσίδα εξαρτήσεων μεταξύ των blocks, εξασφαλίζοντας ότι οποιαδήποτε τροποποίηση ενός block θα επηρεάσει όλα τα επόμενα blocks της αλυσίδας.

8.4 Repository Layer

Η πρόσβαση στο blockchain πραγματοποιείται μέσω ενός Repository Layer το οποίο λειτουργεί ως ενδιάμεσο επίπεδο μεταξύ της εφαρμογής και του Hyperledger Fabric.

Το υποσύστημα αυτό αξιοποιεί το Fabric Gateway API και αναλαμβάνει τη δημιουργία ασφαλούς σύνδεσης με το δίκτυο. Κατά την αρχικοποίηση φορτώνονται τα απαραίτητα πιστοποιητικά, τα ιδιωτικά κλειδιά και οι πληροφορίες του οργανισμού.

Μέσω του Repository Layer αποκτάται πρόσβαση στο channel του δικτύου καθώς και στο αντίστοιχο chaincode.

Η προσέγγιση αυτή επιτρέπει τον διαχωρισμό της λογικής πρόσβασης στο blockchain από την επιχειρησιακή λογική της εφαρμογής, βελτιώνοντας τη συντηρησιμότητα του κώδικα.

8.5 Service Layer

Το Service Layer λειτουργεί ως αφαιρετικό επίπεδο πάνω από το blockchain και προσφέρει υπηρεσίες υψηλού επιπέδου προς το REST API.

Οι μέθοδοι του επιπέδου αυτού αντιστοιχούν στις βασικές λειτουργίες που παρέχει το chaincode, όπως η εισαγωγή νέων blocks, η ανάκτηση ολόκληρης της αλυσίδας και η αναζήτηση συγκεκριμένων blocks.

Η χρήση του Service Layer επιτρέπει την απόκρυψη των λεπτομερειών επικοινωνίας με το blockchain και την παροχή ενός απλού προγραμματιστικού περιβάλλοντος προς τα υπόλοιπα τμήματα της εφαρμογής.

8.6 Αρχικοποίηση Pairings

Η κρυπτογραφική λειτουργικότητα της εφαρμογής βασίζεται στη βιβλιοθήκη JPBC η οποία παρέχει υποστήριξη για διγραμμικές απεικονίσεις και πράξεις σε ελλειπτικές καμπύλες.

Κατά την εκκίνηση κάθε κόμβου δημιουργούνται ή φορτώνονται οι παράμετροι του pairing. Οι παράμετροι αυτές περιγράφουν τις ομάδες G_1 , G_T και το πεπερασμένο σώμα στο οποίο πραγματοποιούνται οι κρυπτογραφικές πράξεις.

Όλοι οι συμμετέχοντες χρησιμοποιούν κοινό γεννήτορα της ομάδας G_1 , γεγονός που επιτρέπει την εκτέλεση συμβατών πράξεων κατά τη διάρκεια του πρωτοκόλλου.

8.6.1 Υλοποίηση BLS Υπογραφών

Για την αυθεντικοποίηση των συμμετεχόντων χρησιμοποιούνται ψηφιακές υπογραφές τύπου BLS.

Κάθε κόμβος διαθέτει ιδιωτικό κλειδί sk και δημόσιο κλειδί pk . Κατά τη δημιουργία ενός block παράγεται υπογραφή πάνω στα δεδομένα του block η οποία αποθηκεύεται μαζί με τις υπόλοιπες κρυπτογραφικές παραμέτρους.

Η επαλήθευση πραγματοποιείται μέσω bilinear pairings, συγκρίνοντας δύο τιμές της ομάδας στόχου G_T .

Με τον τρόπο αυτό κάθε κόμβος μπορεί να επαληθεύσει την ταυτότητα του προηγούμενου συμμετέχοντα πριν συνεχίσει την εκτέλεση του πρωτοκόλλου.

Κεφάλαιο 9

Πειράματα και Μετρήσεις

Στο παρόν κεφάλαιο παρουσιάζεται η πειραματική αξιολόγηση της προτεινόμενης υλοποίησης του πρωτοκόλλου Blockchain Dynamic Group Key Agreement (BDGKA). Στόχος των πειραμάτων είναι η διερεύνηση της αποδοτικότητας, της επεκτασιμότητας και της πρακτικής εφαρμοσιμότητας του πρωτοκόλλου σε ένα πραγματικό περιβάλλον blockchain βασισμένο στο Hyperledger Fabric.

Η αξιολόγηση πραγματοποιήθηκε σε περιβάλλον Linux, με χρήση Docker containers για την ανάπτυξη του δικτύου Hyperledger Fabric. Το blockchain δίκτυο αποτελείται από δύο οργανισμούς (Organizations), έναν Orderer, δύο Peers και μία Certificate Authority για κάθε οργανισμό. Οι κόμβοι του πρωτοκόλλου υλοποιήθηκαν σε Java χρησιμοποιώντας το πλαίσιο ανάπτυξης Spring Boot, ενώ οι κρυπτογραφικές λειτουργίες βασίστηκαν στη βιβλιοθήκη JPBC για την υποστήριξη bilinear pairings και ελλειπτικών καμπυλών.

Για την αξιολόγηση του πρωτοκόλλου εξετάστηκαν τρεις βασικές κατηγορίες μετρικών: (i) ο συνολικός χρόνος δημιουργίας του ομαδικού κλειδιού, (ii) το επικοινωνιακό κόστος σε αριθμό ανταλλασσόμενων μηνυμάτων και

Ο συνολικός χρόνος δημιουργίας του ομαδικού κλειδιού αντιστοιχεί στο χρονικό διάστημα από την έναρξη του πρωτοκόλλου έως την επιτυχή παραγωγή και επαλήθευση του κοινού μυστικού κλειδιού από όλους τους συμμετέχοντες.

9.1 Περιβάλλον Πειραμάτων

Τα πειράματα πραγματοποιήθηκαν σε υπολογιστικό σύστημα με τα παρακάτω χαρακτηριστικά:

- Λειτουργικό Σύστημα: Arch Linux
- Επεξεργαστής: AMD Ryzen 3 3250U with Radeon Graphics
- Μνήμη RAM: 32
- Docker Engine: 28
- Hyperledger Fabric: 2.5.4
- Java: OpenJDK 17
- Spring Boot: 4.0.1
- JPBC: 2.0.0

Το δίκτυο Hyperledger Fabric αναπτύχθηκε τοπικά μέσω Docker containers. Κάθε peer, orderer και certificate authority εκτελούνταν σε ξεχωριστό container ώστε να προσομοιωθεί ρεαλιστικό καταναμημένο περιβάλλον.

9.2 Μεθοδολογία Μετρήσεων

Για την εξαγωγή αξιόπιστων αποτελεσμάτων κάθε πείραμα εκτελέστηκε πέντε φορές και καταγράφηκε η μέση τιμή των μετρήσεων.

Η χρονομέτρηση ξεκινούσε κατά την αποστολή του μηνύματος εκκίνησης του πρωτοκόλλου και ολοκληρωνόταν όταν όλοι οι συμμετέχοντες είχαν παράγει και επαληθεύσει επιτυχώς το κοινό ομαδικό κλειδί.

Οι μετρήσεις πραγματοποιήθηκαν για ομάδες μεγέθους 4, 8, 16 και 32 κόμβων.

9.3 Επεκτασιμότητα πρωτοκόλλου

Για την αξιολόγηση της επεκτασιμότητας πραγματοποιήθηκαν πολλαπλές εκτελέσεις του πρωτοκόλλου με διαφορετικό πλήθος συμμετεχόντων κόμβων. Ο Πίνακας 9.1 παρουσιάζει τον συνολικό χρόνο ολοκλήρωσης του πρωτοκόλλου για διαφορετικά μεγέθη ομάδων.

Πίνακας 9.1: Χρόνος ολοκλήρωσης πρωτοκόλλου

Αριθμός Κόμβων	Χρόνος (sec)
4	15.3
8	30.8
16	62.2
32	125.9

Παρατηρείται ότι ο χρόνος εκτέλεσης αυξάνεται σχεδόν γραμμικά με τον αριθμό των συμμετεχόντων κόμβων. Ο διπλασιασμός του πλήθους των κόμβων οδηγεί σε αντίστοιχο σχεδόν διπλασιασμό του συνολικού χρόνου ολοκλήρωσης. Η συμπεριφορά αυτή είναι αναμενόμενη, καθώς κάθε κόμβος συμμετέχει διαδοχικά στη διαδικασία υπολογισμού του ομαδικού κλειδιού, ενώ παράλληλα πραγματοποιούνται συναλλαγές που καταγράφονται στο blockchain.

Τα αποτελέσματα δείχνουν ότι το προτεινόμενο πρωτόκολλο παρουσιάζει προβλέψιμη κλιμάκωση και μπορεί να υποστηρίξει μεγαλύτερες ομάδες χρηστών χωρίς εκθετική αύξηση του κόστους. Η κύρια πηγή καθυστέρησης δεν εντοπίζεται στις κρυπτογραφικές πράξεις, αλλά στην αλληλουχία ανταλλαγής μηνυμάτων και στη διαδικασία καταχώρησης συναλλαγών στο blockchain.

9.4 Επικοινωνιακό κόστος

Ιδιαίτερο ενδιαφέρον παρουσιάζει και το επικοινωνιακό κόστος του πρωτοκόλλου. Συγκεκριμένα, μετρήθηκε ο μέγιστος αριθμός μηνυμάτων που ανταλλάσσονται μεταξύ των κόμβων κατά την εκτέλεση του πρωτοκόλλου.

Πίνακας 9.2: Επικοινωνιακό κόστος (μέγιστος αριθμός μηνυμάτων)

Αριθμός Κόμβων	Μέγιστος αριθμός μηνυμάτων
4	16
8	32
16	64
32	128

Παρατηρείται ότι ο αριθμός των μηνυμάτων αυξάνεται γραμμικά με το πλήθος των συμμετεχόντων κόμβων, γεγονός που είναι αναμενόμενο για πρωτόκολλα ομαδικής συμφωνίας κλειδιού βασισμένα σε αλληλεπιδράσεις peer-to-peer.

9.5 Συμπεράσματα

Συνολικά, τα πειραματικά αποτελέσματα καταδεικνύουν ότι το προτεινόμενο πρωτόκολλο επιτυγχάνει την ασφαλή δημιουργία ενός κοινού ομαδικού κλειδιού σε αποκεντρωμένο περιβάλλον, διατηρώντας παράλληλα αποδεκτό υπολογιστικό και επικοινωνιακό κόστος.

Η χρήση του Hyperledger Fabric εξασφαλίζει ακεραιότητα, διαφάνεια και δυνατότητα επαλήθευσης των ανταλλασσόμενων μηνυμάτων. Παράλληλα, το Spring Boot παρέχει ένα σταθερό πλαίσιο υλοποίησης των κόμβων, ενώ η βιβλιοθήκη JPBC υποστηρίζει αποδοτικά τις απαιτούμενες κρυπτογραφικές πράξεις.

Τα αποτελέσματα δείχνουν ότι η προτεινόμενη λύση μπορεί να εφαρμοστεί αποτελεσματικά σε περιβάλλοντα όπου απαιτείται ασφαλής διαχείριση ομαδικών κλειδιών και αποκεντρωμένη εμπιστοσύνη.

Κεφάλαιο 10

Πηγαίος Κώδικας

```
1 package main
2
3 import (
4     "crypto/sha256"
5     "encoding/hex"
6     "encoding/json"
7     "fmt"
8
9     "github.com/hyperledger/fabric-contract-api-go/contractapi"
10 )
11
12 type Block struct {
13     ID      string `json:"id"`
14     Header  string `json:"header"`
15     NextIP  string `json:"nextIp"`
16     Hi2     string `json:"hi2"`
17     PK      string `json:"pk"`
18     Ms      []string `json:"ms"`
19     Signature string `json:"signature"`
20     Timestamp int64 `json:"timestamp"`
21     PrevHash string `json:"prevHash"`
22     CurrentHash string `json:"currentHash"`
23 }
24
25 type BDGKAContract struct {
26     contractapi.Contract
27 }
28
29 func (block *Block) hash() error {
30     tempBlock := *block
31     tempBlock.CurrentHash = ""
32     tempBlockBytes, err := json.Marshal(tempBlock)
33     if err != nil {
34         return fmt.Errorf("failed to marshal block: %v", err)
35     }
36     hash_bytes := sha256.Sum256(tempBlockBytes)
```

```

37 block.CurrentHash = hex.EncodeToString(hash_bytes[:])
38 return nil
39 }
40 func (contract *BDGKAContract) AppendBlock(ctx contractapi.TransactionContextInterface,
41 id string,
42 header string,
43 nextIP string,
44 hi2 string,
45 pk string,
46 msJson string,
47 signature string,
48 ) error {
49 var ms []string
50 err := json.Unmarshal([]byte(msJson), &ms)
51 if err != nil {
52 return fmt.Errorf("json cannot be turned to list")
53 }
54 exists, err := contract.BlockExists(ctx, id)
55 if err != nil {
56 return err
57 }
58 if exists {
59 return fmt.Errorf("block %s already exists", id)
60 }
61
62 timestamp, err := ctx.GetStub().GetTxTimestamp()
63 if err != nil {
64 return fmt.Errorf("timestamp error: %vs", err)
65 }
66 prevBlockIdBytes, err := ctx.GetStub().GetState("LAST_BLOCK")
67 if err != nil {
68 return fmt.Errorf("previous block error: %vs", err)
69 }
70 var prevBlock *Block
71 var prevHash string
72 if prevBlockIdBytes != nil {
73 prevBlockId := string(prevBlockIdBytes)
74 prevBlock, err = contract.ReadBlock(ctx, prevBlockId)
75 if err != nil {
76 return fmt.Errorf("previous block error: %v", err)
77 }
78 prevHash = prevBlock.CurrentHash
79 } else {
80 prevHash = "GENESIS"
81 }
82 block := Block{
83 ID: id,
84 Header: header,
85 NextIP: nextIP,
86 Hi2: hi2,

```

```

87  PK:    pk,
88  Ms:    ms,
89  Signature: signature,
90  Timestamp: timestamp.Seconds,
91  PrevHash: prevHash,
92  }
93  err = block.hash()
94  if err != nil {
95      return fmt.Errorf("error calculating signature: %v", err)
96  }
97  data, err := json.Marshal(block)
98  if err != nil {
99      return fmt.Errorf("invalid Ms JSON: %v", err)
100  }
101  if err := ctx.GetStub().PutState(id, data); err != nil {
102      return fmt.Errorf("failed to put block state: %v", err)
103  }
104  if err := ctx.GetStub().PutState("LAST_BLOCK", []byte(id)); err != nil {
105      return fmt.Errorf("failed to update LAST_BLOCK %v", err)
106  }
107  return nil
108  }
109
110 func (c *BDGKAContract) ReadBlock(ctx contractapi.TransactionContextInterface, id string)
(*Block, error) {
111  data, err := ctx.GetStub().GetState(id)
112  if err != nil {
113      return nil, fmt.Errorf("read: %v", err)
114  }
115  if data == nil {
116      return nil, fmt.Errorf("block %s not found", id)
117  }
118  var block Block
119  err = json.Unmarshal(data, &block)
120  if err != nil {
121      return nil, fmt.Errorf("unmarshal: %v", err)
122  }
123  return &block, nil
124  }
125
126 func (c *BDGKAContract) ReadCurrentBlock(ctx contractapi.TransactionContextInterface)
(*Block, error) {
127
128  prevBlockIdBytes, err := ctx.GetStub().GetState("LAST_BLOCK")
129  prevBlockId := string(prevBlockIdBytes)
130  block, err := c.ReadBlock(ctx, prevBlockId)
131  if err != nil {
132      return nil, fmt.Errorf("previous block error: %v", err)
133  }
134  return block, nil

```

```

135 }
136 func (c *BDGKAContract) BlockExists(ctx contractapi.TransactionContextInterface, id string)
137 (bool, error) {
138     data, err := ctx.GetStub().GetState(id)
139     if err != nil {
140         return false, err
141     }
142     return data != nil, nil
143 }
144 func (c *BDGKAContract) GetAllBlocks(ctx contractapi.TransactionContextInterface) ([]*Block,
145 error) {
146     resultsIterator, err := ctx.GetStub().GetStateByRange("", "")
147     if err != nil {
148         return nil, fmt.Errorf("failed to get state by range: %v", err)
149     }
150     defer resultsIterator.Close()
151     var blocks []*Block
152     for resultsIterator.HasNext() {
153         queryResponse, err := resultsIterator.Next()
154         if err != nil {
155             return nil, fmt.Errorf("failed to iterate results: %v", err)
156         }
157         key := queryResponse.Key
158         if key == "LAST_BLOCK" {
159             continue
160         }
161         var block Block
162         err = json.Unmarshal(queryResponse.Value, &block)
163         if err != nil {
164             return nil, fmt.Errorf("failed to unmarshal block: %v", err)
165         }
166         blocks = append(blocks, &block)
167     }
168     return blocks, nil
169 }
170 func main() {
171     cc, err := contractapi.NewChaincode(new(BDGKAContract))
172     if err != nil {
173         panic(fmt.Sprintf("initialization error: %v", err))
174     }
175     err = cc.Start()
176     if err != nil {
177         panic(fmt.Sprintf("error starting chaincode: %v", err))
178     }
179 }

```

```

1 @Service
2 public class ChaincodeService {
3     private BDGKARepository repo;
4     public ChaincodeService(BDGKARepository repo){

```

```

5      this.repo = repo;
6  }
7  public List<Block> getAllBlocks() throws Exception {
8      var bytes = repo.getContract().evaluateTransaction("GetAllBlocks");
9      var mapper = new ObjectMapper();
10     return mapper.readValue(bytes, new TypeReference<List<Block>>() {});
11 }
12 public String appendBlock(Block block) throws Exception {
13     ObjectMapper mapper = new ObjectMapper();
14     System.out.println(block.getMs());
15
16     String msJson = mapper.writeValueAsString(block.getMs());
17
18     System.out.println("msJson "+msJson);
19     System.out.println(block);
20     repo.getContract().submitTransaction(
21         "AppendBlock",
22         block.getId(),
23         block.getHeader(),
24         block.getNextIp(),
25         block.getHi2(),
26         block.getPk(),
27         msJson,
28         block.getSignature()
29     );
30     return "OK";
31 }
32 public Block readBlock(String id) throws Exception {
33     var bytes = repo.getContract().evaluateTransaction("ReadBlock",id);
34     return new ObjectMapper().readValue(bytes, new TypeReference<Block>() {});
35 }
36 public Block getCurrent() throws Exception{
37     var bytes = repo.getContract().evaluateTransaction("ReadCurrentBlock");
38     return new ObjectMapper().readValue(bytes, new TypeReference<Block>() {});
39 }
40 }
41 }

```

```

1 package com.icsd16191.bdgka_app_client.crypto;
2
3 import java.io.BufferedReader;
4 import java.io.BufferedWriter;
5 import java.io.ByteArrayOutputStream;
6 import java.io.File;
7 import java.io.FileReader;
8 import java.io.FileWriter;
9 import java.io.IOException;
10 import java.nio.charset.StandardCharsets;
11 import java.nio.file.Files;
12 import java.nio.file.Path;

```

```

13 import java.security.MessageDigest;
14 import java.security.NoSuchAlgorithmException;
15 import java.util.ArrayList;
16 import java.util.Base64;
17 import java.util.List;
18
19 import org.springframework.stereotype.Service;
20
21 import com.icsd16191.bdgka_app_client.configuration.IdentityService.Identity;
22 import com.icsd16191.bdgka_app_client.entities.Block;
23 import com.icsd16191.bdgka_app_client.messages.CryptographicMessageParameters;
24 import com.icsd16191.bdgka_app_client.services.InMemoryKeyStoreService;
25
26 import it.unisa.dia.gas.jpbc.Element;
27 import it.unisa.dia.gas.jpbc.Pairing;
28 import it.unisa.dia.gas.plaf.jpbc.pairing.PairingFactory;
29 import it.unisa.dia.gas.plaf.jpbc.pairing.a.TypeACurveGenerator;
30 import lombok.RequiredArgsConstructor;
31 import tools.jackson.databind.ObjectMapper;
32
33 @Service
34 @RequiredArgsConstructor
35 public class CryptoService {
36
37     private TypeACurveGenerator gen = new TypeACurveGenerator(256, 1024); // group security, size
38                                     // of base field
39     private final InMemoryKeyStoreService keyStoreService;
40     private Pairing pairing;
41     private Element genitorElement;
42     private final Identity id;
43     {
44         try {
45             if (!Files.exists(Path.of("src/main/resources/parameters.properties"))) {
46                 System.out.println("generating parameters");
47                 generateECParameters();
48             }
49         } catch (Exception e) {
50             throw new RuntimeException(e);
51         }
52         pairing = PairingFactory.getPairing("src/main/resources/parameters.properties");
53         if (!Files.exists(Path.of("src/main/resources/genitor.element"))) {
54             genitorElement = pairing.getG1().newRandomElement().getImmutable();
55             try {
56                 BufferedWriter writer = new BufferedWriter(new FileWriter(new
57                     File("src/main/resources/genitor.element")));
58                 var genitorStr = Base64.getEncoder().encodeToString(genitorElement.toBytes());
59                 writer.write(genitorStr);
60                 writer.close();
61             } catch (IOException e) {
62                 e.printStackTrace();
63             }
64         }
65     }
66 }

```

```

61     }
62 } else {
63     try (BufferedReader reader = new BufferedReader(new
64         FileReader("src/main/resources/genitor.element"))) {
65         var genitorStr = reader.readLine().toString();
66         genitorElement = pairing.getG1().newElementFromBytes(Base64.getDecoder().decode(genito_
67             rStr)).getImmutable();
68         reader.close();
69     } catch (IOException e) {
70         e.printStackTrace();
71     }
72 }
73
74 private void generateECParameters() throws Exception {
75     BufferedWriter bw = new BufferedWriter(new FileWriter(new
76         File("src/main/resources/parameters.properties")));
77     var params = gen.generate();
78     bw.write(params.toString());
79     bw.flush();
80     bw.close();
81 }
82
83 public void generateMiKeys() {
84     var mi = pairing.getZr().newRandomElement().getImmutable();
85     var Mi = this.genitorElement.duplicate().powZn(mi).getImmutable();
86     keyStoreService.setMi_public(Mi);
87     keyStoreService.setMi_private(mi);
88 }
89
90 public void generateSignatureKeys() {
91     var mi = pairing.getZr().newRandomElement().getImmutable();
92     var Mi = this.genitorElement.duplicate().powZn(mi).getImmutable();
93     keyStoreService.setPublicKey(Mi);
94     keyStoreService.setPrivateKey(mi);
95 }
96
97 public String base64Encode(byte[] value) {
98     var encoder = Base64.getEncoder();
99     return encoder.encodeToString(value);
100 }
101
102 public byte[] base64Decode(String base64) {
103     var decoder = Base64.getDecoder();
104     return decoder.decode(base64);
105 }
106
107 public Element elementFromHash(byte[] hash) {
108     return pairing.getG1().newElementFromHash(hash, 0, hash.length).getImmutable();
109 }

```

```

109
110 public Element elementFromBytes(byte[] bytes) {
111     return pairing.getG1().newElementFromBytes(bytes);
112 }
113
114 public Element elementFromBase64(String base64) {
115     var bytes = base64Decode(base64);
116     var el = pairing.getG1().newElementFromBytes(bytes).getImmutable();
117     return el;
118 }
119
120 public String blsSignature(byte[] content) {
121     var h = elementFromHash(content);
122     var sig = h.duplicate().powZn(keyStoreService.getPrivateKey());
123     return base64Encode(sig.toBytes());
124 }
125
126 // this one gets used
127 public Element h2(Element h0, Element h1) {
128     var sig = h0.duplicate().powZn(keyStoreService.getPrivateKey()).add(
129         h1.duplicate().powZn(keyStoreService.getPrivateKey())).getImmutable();
130     return sig;
131 }
132
133
134 public Boolean blsVerify(byte[] content, byte[] signature, Element publicKey) {
135     var h = elementFromBytes(content);
136     var sig = pairing.getG1().newElementFromBytes(signature);
137
138     Element temp1 = pairing.pairing(sig, genitorElement);
139     Element temp2 = pairing.pairing(h, publicKey);
140
141     return temp1.isEqual(temp2);
142 }
143
144 public Boolean h2Verify(Element el1, Element el2, Element sig, Element publicKey) {
145     Element left = pairing.pairing(sig, this.genitorElement.duplicate()).getImmutable();
146     // System.out.println("left-> " + left);
147     Element right0 = pairing.pairing(el1, publicKey).getImmutable();
148     Element right1 = pairing.pairing(el2, publicKey).getImmutable();
149
150     // System.out.println("right-> " + right0.duplicate().mul(right1));
151     return left.isEqual(right0.duplicate().mul(right1));
152 }
153
154 public Block createBlock(String id, List<String> ms) {
155     var block = new Block();
156     ms.add(base64Encode(keyStoreService.getMi_public().toBytes()));
157     block.setMs(ms);
158     var bytes = new ByteArrayOutputStream();

```

```

159
160 try {
161     bytes.write(keyStoreService.getPublicKey().toBytes());
162     bytes.write(keyStoreService.getMi_public().toBytes());
163     // bytes.write(keyStoreService.getPrivateKey().toBytes());
164 } catch (IOException e) {
165     e.printStackTrace();
166     throw new RuntimeException(e);
167 }
168 byte[] hi2data = bytes.toByteArray();
169 var hi2 = elementFromHash(hi2data);
170 block.setHi2(base64Encode(hi2.toBytes()));
171
172 var signature = h2(hi2, keyStoreService.getMi_public());
173 block.setSignature(base64Encode(signature.toBytes()));
174
175 block.setId(id);
176 block.setTimestamp(System.currentTimeMillis());
177 block.setNextIp(keyStoreService.getNext());
178 block.setPk(base64Encode(keyStoreService.getPublicKey().toBytes()));
179 block.setHeader("Key-Computation");
180
181 // tests
182 if (keyStoreService.getMi_public().isEqual(this.genitorElement.powZn(keyStoreService.getMi_
private())) {
183     System.out.println("Mi test ok");
184 } else {
185     System.out.println("Mi test error");
186 }
187 if (elementFromBase64(block.getHi2()).isEqual(hi2)) {
188     System.out.println("Hi2 ok");
189 } else {
190     System.out.println("Hi2 error");
191 }
192
193 if (elementFromBase64(block.getSignature()).isEqual(signature)) {
194     System.out.println("signature ok");
195 } else {
196     System.out.println("signature error");
197 }
198
199 if (h2Verify(
200     elementFromBase64(block.getHi2()),
201     keyStoreService.getMi_public(),
202     elementFromBase64(block.getSignature()),
203     keyStoreService.getPublicKey())) {
204     System.out.println("Block " + block.getId() + " is ok");
205 } else {
206     System.out.println("Block " + block.getId() + " not verified");
207 }

```

```

208     return block;
209 }
210
211
212 public CryptographicMessageParameters produceMessageParametersFromBlock(Block block) {
213     var cm = new CryptographicMessageParameters();
214     cm.setHi2(block.getHi2());
215     cm.setHii(blsSignature(keyStoreService.getMi_public().toBytes()));
216     cm.setIp(block.getNextIp());
217     cm.setPku(block.getPk());
218     cm.setSign1(block.getSignature());
219     System.out.println(block.getSignature());
220     cm.setMi(base64Encode(keyStoreService.getMi_public().toBytes()));
221
222     if (!verifyUser(block, cm)) {
223         System.out.println("verification process error");
224     }
225
226     return cm;
227 }
228
229 public Boolean verifyUser(Block block, CryptographicMessageParameters params) {
230     return h2Verify(
231         elementFromBase64(params.getHi2()),
232         elementFromBase64(params.getMi()),
233         elementFromBase64(block.getSignature()),
234         elementFromBase64(params.getPku()));
235 }
236
237 public List<String> computeFsListAndGkForUn(List<String> ms) {
238     List<String> fs = new ArrayList<>();
239     Element tn = pairing.getZr().newRandomElement().getImmutable();
240
241     Element sumFr = null;
242
243     for (int i = 0; i < ms.size(); i++) {
244         var m = ms.get(i);
245         var fcurrent = elementFromBase64(m).duplicate().powZn(tn).getImmutable();
246         fs.add(base64Encode(fcurrent.toBytes()));
247
248         if (sumFr == null) {
249             sumFr = fcurrent.duplicate().getImmutable();
250         } else {
251             sumFr = sumFr.duplicate().add(fcurrent).getImmutable();
252         }
253     }
254
255     var betaN = this.genitorElement.duplicate().mul(sumFr).getImmutable();
256     var alpha = this.genitorElement.duplicate().powZn(tn).getImmutable();
257     System.out.println("\n\t" + alpha + "\n\t" + betaN);

```

```

258     keyStoreService.setGroupKey(pairing.pairing(
259         alpha,
260         betaN).getImmutable());
261     System.out.println("START");
262     System.out.println(id.getId());
263     System.out.println("group key for Un " + keyStoreService.getGroupKey());
264     System.out.println("END");
265
266     return fs;
267 }
268
269 public void computeGkForUi(List<String> fs, Integer index) {
270     var invertedMi = keyStoreService.getMi_private().duplicate().invert().getImmutable();
271     Element betaK = null;
272     var alphak = invertedMi.duplicate().getImmutable();
273     var fk = elementFromBase64(fs.get(fs.size() - 1 - index));
274     for (String f : fs) {
275         if (betaK == null) {
276             betaK = elementFromBase64(f).duplicate().getImmutable();
277         } else {
278             betaK = betaK.duplicate().add(elementFromBase64(f)).getImmutable();
279         }
280     }
281     alphak = fk.duplicate().powZn(alphak.duplicate()).getImmutable();
282     keyStoreService.setGroupKey(pairing.pairing(alphak,
283         this.genitorElement.duplicate().mul(betaK).getImmutable()));
284     System.out.println("\n\t" + alphak + "\n\t" + betaK);
285     System.out.println("START");
286     System.out.println(id.getId());
287     System.out.println("group key for Un-" + index + " " + keyStoreService.getGroupKey());
288     System.out.println("END");
289 }
290 public String h1Gk() throws NoSuchAlgorithmException{
291     var hash = MessageDigest.getInstance("SHA-512");
292     var gk = keyStoreService.getGroupKey().duplicate();
293     return base64Encode(hash.digest(gk.toBytes()));
294 }
295 public Boolean h1VerifyGk(String base64GkHash) throws NoSuchAlgorithmException{
296     return MessageDigest.isEqual(base64Decode(h1Gk()), base64Decode(base64GkHash));
297 }
298 }
299
300 ^I

```

```

1 package com.icsd16191.bdgka_app_client.messages;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.Map;

```

```

6  import java.util.Set;
7  import java.util.UUID;
8
9  import org.springframework.jms.annotation.JmsListener;
10 import org.springframework.stereotype.Component;
11
12 import com.icsd16191.bdgka_app_client.configuration.IdentityService;
13 import com.icsd16191.bdgka_app_client.crypto.CryptoService;
14 import com.icsd16191.bdgka_app_client.entities.Block;
15 import com.icsd16191.bdgka_app_client.services.InMemoryKeyStoreService;
16 import com.icsd16191.bdgka_app_client.services.ProtocolService;
17 import com.icsd16191.bdgka_app_client.web_client.BdgkaClient;
18
19 import lombok.AllArgsConstructor;
20 import tools.jackson.core.type.TypeReference;
21 import tools.jackson.databind.ObjectMapper;
22
23 @Component
24 @AllArgsConstructor
25 public class MessageListener {
26     private ObjectMapper mapper;
27     private ProtocolService protocolService;
28     private InMemoryKeyStoreService keyStoreService;
29     private IdentityService.Identity identity;
30     private CryptoService cryptoService;
31     private MessageProducer messageProducer;
32     private BdgkaClient blockchainClient;
33
34     @JmsListener(destination = "blockchain-topic", containerFactory =
35         "durableTopicListenerFactory")
36     public void receiveMessage(String message) {
37         var msg = mapper.readValue(message, Message.class);
38         try {
39             switch (msg.getOperation()) {
40                 case "START":
41                     try {
42                         protocolService.generateAndBroadcastPublicKey();
43                     } catch (Exception e) {
44                         e.printStackTrace();
45                     }
46                     break;
47                 case "PARTICIPANTS":
48                     var participants = mapper.readValue(msg.getPayload(), new TypeReference<Set<String>>() {
49                     }).stream().toList();
50                     protocolService.resolveParticipantsAndSaveNextPrevious(participants);
51                     System.out.println("ORDER\n\t" +
52                         keyStoreService.getPrevious() + "->(current)" +
53                         identity.getId() + "->" +
54                         keyStoreService.getNext());

```

```

55     break;
56     case "INITIAL_PK_BROADCAST":
57         String next = keyStoreService.getNext();
58         String previous = keyStoreService.getPrevious();
59         if (!next.equals("<None>") && next.equals(msg.getFrom())) {
60             keyStoreService.getPublicKeys().put(
61                 next,
62                 cryptoService.elementFromBase64(msg.getPayload());
63             if (previous.equals("<None>")) {
64                 try {
65                     if (keyStoreService.getMi_public() == null) {
66                         cryptoService.generateMiKeys();
67                     }
68                     var ms = new ArrayList<String>();
69                     var block = cryptoService.createBlock("U-Block-" + UUID.randomUUID().toString(), ms);
70
71                     System.out.println("produced " + block.getId());
72
73                     var msgParams = cryptoService.produceMessageParametersFromBlock(block);
74                     var result = blockchainClient.append(block);
75                     System.out.println("blockchain network " + result.block());
76
77                     var messageToSend = new Message();
78                     messageToSend.setFrom(identity.getId());
79                     messageToSend.setForWho(keyStoreService.getNext());
80                     messageToSend.setOperation("GK-COMPUTATION");
81                     messageToSend.setPayload(mapper.writeValueAsString(msgParams));
82                     messageProducer.sendMessage("blockchain-topic", messageToSend);
83
84                     } catch (Exception e) {
85                         e.printStackTrace();
86                     }
87                 }
88             }
89
90             if (!previous.equals("<None>") && previous.equals(msg.getFrom())) {
91                 keyStoreService.getPublicKeys().put(
92                     previous,
93                     cryptoService.elementFromBase64(msg.getPayload());
94                 }
95             break;
96             case "GK-COMPUTATION":
97                 if (msg.getForWho().equals(identity.getId())) {
98                     CryptographicMessageParameters params = mapper.readValue(msg.getPayload(),
99                         CryptographicMessageParameters.class);
100                     var lastBlockResponse = blockchainClient.current().block();
101                     if (lastBlockResponse.get("result").equals("OK") && params.getIp().equals(identity.getId()))
102                         {
103                             if (keyStoreService.getMi_public() == null) {

```

```

104     System.out.println("generating Mi keys");
105     cryptoService.generateMiKeys();
106 }
107
108 Block currentBlock =
109     mapper.readValue(mapper.writeValueAsString(lastBlockResponse.get("data")),
110         Block.class);
111     System.out.println("read " + currentBlock.getId());
112
113     if (cryptoService.verifyUser(currentBlock, params)) {
114
115         var ms = currentBlock.getMs();
116         if (!keyStoreService.getNext().equals("<None>")) {
117             var block = cryptoService.createBlock("CONTRIBUTE-Block-" +
118                 UUID.randomUUID().toString(), ms);
119             var result = blockchainClient.append(block).block();
120             System.out.println("blockchain-network " + result + "\n" + block.getId() + " produced");
121
122             var msgParams = cryptoService.produceMessageParametersFromBlock(block);
123             var messageToSend = new Message();
124             messageToSend.setFrom(identity.getId());
125             messageToSend.setForWho(keyStoreService.getNext());
126             messageToSend.setOperation("GK-COMPUTATION");
127             messageToSend.setPayload(mapper.writeValueAsString(msgParams));
128             messageProducer.sendMessage("blockchain-topic", messageToSend);
129         } else {
130             var block = cryptoService.createBlock("CONTRIBUTE-Last-Block-" +
131                 UUID.randomUUID().toString(), ms);
132
133             var fs = cryptoService.computeFsListAndGkForUn(block.getMs());
134             block.setHeader(cryptoService.h1Gk());
135             var result = blockchainClient.append(block).block();
136             System.out.println("blockchain-network final " + result);
137             var messageToSend = new Message();
138
139             messageToSend.setFrom(identity.getId());
140             messageToSend.setForWho(keyStoreService.getPrevious());
141             messageToSend.setOperation("GK-FINAL");
142             messageToSend.setPayload(mapper.writeValueAsString(Map.of(
143                 "fs", fs, "index", 1)));
144
145             messageProducer.sendMessage("blockchain-topic", messageToSend);
146         }
147     } else {
148         System.out.println("user " + identity.getId() + " not verifying previous");
149     }
150 }
151

```

```

152     break;
153     case "GK-FINAL":
154         if (msg.getForWho().equals(identity.getId())) {
155             System.out.println("GK-FINAL " + identity.getId());
156             Map<String, Object> fsAndIndex = mapper.readValue(msg.getPayload(),
157                 new TypeReference<Map<String, Object>>() {
158                 });
159             List<String> fs;
160             try {
161                 fs = (List<String>) fsAndIndex.get("fs");
162
163                 var index = (Integer) fsAndIndex.get("index");
164                 cryptoService.computeGkForUi(fs, index);
165
166                 var lastBlockResponse = blockchainClient.current().block();
167                 if (lastBlockResponse.get("result").equals("OK")) {
168                     Block currentBlock =
169                         mapper.readValue(mapper.writeValueAsString(lastBlockResponse.get("data")),
170                             Block.class);
171                     System.out.println("read " + currentBlock.getId());
172                     if (cryptoService.h1VerifyGk(currentBlock.getHeader())) {
173                         System.out.println("group key is ok");
174                     } else {
175                         System.out.println("group key not verified");
176                     }
177
178                     var messageToSend = new Message();
179                     if (!keyStoreService.getPrevious().equals("<None>")) {
180                         messageToSend.setFrom(identity.getId());
181                         messageToSend.setForWho(keyStoreService.getPrevious());
182                         messageToSend.setOperation("GK-FINAL");
183                         messageToSend.setPayload(mapper.writeValueAsString(Map.of(
184                             "fs", fs, "index", index + 1)));
185                         messageProducer.sendMessage("blockchain-topic", messageToSend);
186                     }
187                 } catch (Exception e) {
188                     e.printStackTrace();
189                 }
190             }
191         }
192         break;
193     }
194     } catch (Exception e) {
195         e.printStackTrace();
196         throw new RuntimeException(e);
197     }
198 }
199
200 }

```

201
202

```
1 public class BDGKARepository {
2     private final String mspId = "Org1MSP";
3     private final String channelName = "mychannel";
4     private final String chaincodeName = "bdgka_chaincode";
5
6     private final Path cryptoRootDir = Paths
7         .get("../fabric-samples/test-network/organizations/peerOrganizations/org1.example.com");
8     private final Path certDirPath =
9         cryptoRootDir.resolve("users/User1@org1.example.com/msp/signcerts");
10    private final Path keyDirPath =
11        cryptoRootDir.resolve("users/User1@org1.example.com/msp/keystore");
12    private final Path tlsCertPath = cryptoRootDir.resolve("peers/peer0.org1.example.com/tls/ca.crt");
13
14    private final Gateway gateway;
15    private final Network network;
16    private final Contract contract;
17    public BDGKARepository() throws Exception {
18        var channel = connect();
19
20        var builder = Gateway.newInstance();
21        builder.identity(newIdentity());
22        builder.signer(newSigner());
23        builder.hash(Hash::sha256);
24        builder.connection(channel);
25        gateway = builder.connect();
26        network = gateway.getNetwork(this.channelName);
27        contract = network.getContract(this.chaincodeName);
28    }
29
30    public String getMspId() {
31        return mspId;
32    }
33
34    public String getChannelName() {
35        return channelName;
36    }
37
38    public String getChaincodeName() {
39        return chaincodeName;
40    }
41
42    public Path getCryptoRootDir() {
43        return cryptoRootDir;
44    }
45
46    public Path getCertDirPath() {
47        return certDirPath;
48    }
49 }
```

```

46 }
47
48 public Path getKeyDirPath() {
49     return keyDirPath;
50 }
51
52 public Path getTlsCertPath() {
53     return tlsCertPath;
54 }
55
56 public Gateway getGateway() {
57     return gateway;
58 }
59
60 public Network getNetwork() {
61     return network;
62 }
63
64 public Contract getContract() {
65     return contract;
66 }
67
68 @PreDestroy
69 public void close() {
70     gateway.close();
71 }
72
73 private ManagedChannel connect() throws IOException {
74     var creds = TlsChannelCredentials.newBuilder()
75         .trustManager(tlsCertPath.toFile())
76         .build();
77     return Grpc.newChannelBuilder("localhost:7051", creds)
78         .overrideAuthority("peer0.org1.example.com")
79         .build();
80 }
81
82 private Identity newIdentity() throws Exception {
83     try (var certReader = Files.newBufferedReader(getFirstFilePath(certDirPath))) {
84         var cert = Identities.readX509Certificate(certReader);
85         return new X509Identity(mspId, cert);
86     }
87 }
88 private Path getFirstFilePath(Path dirPath) throws Exception {
89     try (var keyFiles = Files.list(dirPath)) {
90         return keyFiles.findFirst().orElseThrow();
91     }
92 }
93 private Signer newSigner() throws Exception {
94     try (var keyReader = Files.newBufferedReader(getFirstFilePath(keyDirPath))) {
95         var privateKey = Identities.readPrivateKey(keyReader);

```

```
96     return Signers.newPrivateKeySigner(privateKey);
97 }
98 }
99 }
```

Κεφάλαιο 11

Βιβλιογραφία

Βιβλιογραφία

- [1] W. Diffie and M. E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644--654, 1976.
- [2] M. Burmester and Y. Desmedt, "A Secure and Efficient Conference Key Distribution System," in *Advances in Cryptology -- EUROCRYPT 1994*, Lecture Notes in Computer Science, vol. 950, Springer, pp. 275--286, 1995.
- [3] M. Steiner, G. Tsudik, and M. Waidner, "Diffie-Hellman Key Distribution Extended to Group Communication," in *Proceedings of the 3rd ACM Conference on Computer and Communications Security*, pp. 31--37, 1996.
- [4] A. Joux, "A One Round Protocol for Tripartite Diffie-Hellman," *Journal of Cryptology*, vol. 17, no. 4, pp. 263--276, 2004.
- [5] D. Boneh and M. Franklin, "Identity-Based Encryption from the Weil Pairing," *SIAM Journal on Computing*, vol. 32, no. 3, pp. 586--615, 2003.
- [6] P. S. L. M. Barreto, H. Y. Kim, B. Lynn, and M. Scott, "Efficient Algorithms for Pairing-Based Cryptosystems," in *Advances in Cryptology -- CRYPTO 2002*, Lecture Notes in Computer Science, vol. 2442, Springer, pp. 354--369, 2002.
- [7] V. S. Miller, "The Weil Pairing, and Its Efficient Calculation," *Journal of Cryptology*, vol. 17, no. 4, pp. 235--261, 2004.
- [8] F. Hess, "Efficient Identity Based Signature Schemes Based on Pairings," in *Selected Areas in Cryptography -- SAC 2002*, Lecture Notes in Computer Science, vol. 2595, Springer, pp. 310--324, 2003.
- [9] A. Shamir, "Identity-Based Cryptosystems and Signature Schemes," in *Advances in Cryptology -- CRYPTO 1984*, Lecture Notes in Computer Science, vol. 196, Springer, pp. 47--53, 1985.
- [10] D. Boneh, B. Lynn, and H. Shacham, "Short Signatures from the Weil Pairing," *Journal of Cryptology*, vol. 17, no. 4, pp. 297--319, 2004.
- [11] Y. Amir, Y. Kim, C. Nita-Rotaru, and G. Tsudik, "On the Performance of Group Key Agreement Protocols," in *Proceedings of the 22nd International Conference on Distributed Computing Systems*, IEEE, pp. 463--464, 2002.
- [12] S. Zheng, J. Alves-Foss, and S. Lee, "Performance of Group Key Agreement Protocols over Multiple Operations," in *Proceedings of the 17th IASTED International Conference on Parallel and Distributed Computing and Systems*, pp. 600--606, 2005.

- [13] R. Dutta and R. Barua, "Overview of Key Agreement Protocols," Cryptology ePrint Archive, Report 2005/289, 2005.
- [14] G. Yao, H. Wang, and Q. Jiang, "An Authenticated 3-Round Identity-Based Group Key Agreement Protocol," in *Proceedings of the International Conference on Availability, Reliability and Security*, ACM, pp. 538--543, 2008.
- [15] L. Zhou, W. Susilo, and Y. Mu, "Efficient ID-Based Authenticated Group Key Agreement from Bilinear Pairings," *Information Processing Letters*, vol. 98, no. 5, pp. 206--211, 2006.
- [16] H. Wang, G. Yao, and Q. Jiang, "An Identity-Based Group Key Agreement Protocol from Pairing," in *Proceedings of the International Conference on Availability, Reliability and Security*, pp. 532--537, 2008.
- [17] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
- [18] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014.
- [19] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains," in *Proceedings of the Thirteenth EuroSys Conference*, ACM, 2018.
- [20] Hyperledger Foundation, "Hyperledger Fabric Documentation," Available: <https://hyperledger-fabric.readthedocs.io/>
- [21] Hyperledger Foundation, "Smart Contracts and Chaincode," Available: <https://hyperledger-fabric.readthedocs.io/en/latest/smartcontract/smartcontract.html>
- [22] Hyperledger Foundation, "Membership Service Provider (MSP)," Available: <https://hyperledger-fabric.readthedocs.io/en/latest/membership/membership.html>
- [23] A. De Caro and V. Iovino, "jPBC: Java Pairing Based Cryptography," in *Proceedings of the IEEE Symposium on Computers and Communications*, pp. 850--855, 2011.
- [24] VMware, "Spring Boot Reference Documentation," Available: <https://docs.spring.io/spring-boot/>
- [25] VMware, "Spring Framework Documentation," Available: <https://docs.spring.io/spring-framework/reference/>
- [26] Apache Software Foundation, "Apache ActiveMQ Artemis Documentation," Available: <https://activemq.apache.org/components/artemis/documentation/>
- [27] Oracle, "Java Message Service Specification," Available: <https://www.oracle.com/java/technologies/java-message-service.html>
- [28] Docker Inc., "Docker Documentation," Available: <https://docs.docker.com/>

- [29] Apache Software Foundation, ``Apache Maven Project Documentation," Available: <https://maven.apache.org/guides/>
- [30] Oracle, ``Java Platform Documentation," Available: <https://docs.oracle.com/en/java/>
- [31] R. Hasan, M. A. Amin, M. I. Abdullah, and A. B. M. S. Ali, ``BEdX25519: A Lightweight Authenticated Key Exchange Protocol for IoT Device-to-Device (D2D) Secure Communications based on Blockchain and Elliptic Curve Cryptography," *Journal of Information Security and Applications*, Manuscript Draft, Manuscript Number: JISAS-D-25-02232, 2025.
- [32] Q. Zhang, B. Wang, X. Zhang, J. Yuan, and R. Wang, ``Blockchain-Based Dynamic Group Key Agreement Protocol for Ad Hoc Network," *Chinese Journal of Electronics*, vol. 29, no. 3, pp. 447--454, May 2020. DOI: 10.1049/cje.2020.02.020.